VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

SPECIALIZED PROJECT (CO4029)

BK-MInD: A Multimodal Data Understanding System for HCMUT Lectures

Supervisors:	Nguyen An Khuong, PhD	
	Nguyen Trang Sy Lam	Motohashi's Research Assistant
Students:	Nguyen Quang Phu	- 2252621
	Nguyen Ngoc Khoi	- 2252378
	Nguyen Minh Khoi	- 2252377

COUNCIL INFORMATION

Chair:	Assoc. Prof. Dr. Lê Hồng Trang
Member 1:	Dr. Trần Tuấn Anh
Member 2:	Dr. Nguyễn Đức Dũng

HO CHI MINH CITY, DECEMBER 2025

Contents

List of Figures

List of Tables

Declaration of Authorship

We declare that this research was completed entirely by ourselves under the supervision and guidance of **Dr. Nguyen An Khuong** and **Mr. Nguyen Trang Sy Lam**. The results presented in this report are original and have not been published anywhere before. All materials used in this study were collected from various reliable sources and are properly cited in the bibliography. All figures and illustrations in this report were created by the authors.

This research also refers to findings from other authors, which have been previously published and are clearly acknowledged in the references.

If any plagiarism is found, we take full responsibility for it. Ho Chi Minh City University of Technology - Vietnam National University is not responsible for any copyright issues that may result from this work.

Ho Chi Minh City, December 2025

Abstract

Educational materials at HCMUT—including lecture videos, audio recordings, slides, and documents—are expanding rapidly, yet conventional text-based tools fail to capture their multimodal nature. These methods often rely on transcripts or extracted text, resulting in recognition errors, hallucinations, and loss of visual and auditory context.

This project proposes a web-based application powered by a Multimodal Retrieval-Augmented Generation (mRAG) pipeline to assist students in studying and exam preparation. The system integrates textual transcripts, Optical Character Recognition (OCR)-based slide and document extraction, visual information for diagrams and equations, and audio features that capture emphasis and speaker prosody. Retrieval combines sparse and dense methods with cross-modal verification, and retrieved evidence is grounded in a large language model for accurate and explainable responses.

The prototype includes a backend using FastAPI for multimodal vector storage and a frontend built with React for responsive search and summarization. Key functionalities include multimodal content retrieval, automatic lecture summarization, and personalized study roadmap generation. The system aims to enhance learning efficiency, reduce information loss, and provide a robust, adaptive educational assistant tailored to the HCMUT academic environment.

Chapter 1

Introduction

1.1 Motivations

Modern educational environments present two fundamental challenges related to information processing and retrieval. First, students conducting self-directed learning face the task of navigating distributed multimodal educational resources across multiple platforms. These resources include video lectures, interactive tutorials, research papers, and technical documentation. The process of locating and organizing these resources requires substantial time investment. Furthermore, students must perform manual information synthesis across different modalities—for instance, connecting conceptual explanations from video content with related textual materials. This manual synthesis process creates high cognitive load and represents a barrier to efficient learning [1, 2].

This information management challenge exists at institutional levels as well. At Ho Chi Minh City University of Technology (HCMUT), the volume of educational content continues to expand across multiple formats including lecture videos, audio recordings, presentation slides, and text documents. Traditional text-based processing methods for this content present limitations. These methods typically rely on transcript generation, which introduces transcription errors and removes important contextual information present in visual components such as diagrams and mathematical notation, as well as auditory elements including speaker emphasis and intonation patterns.

Recent developments in multimodal Large Language Models (LLMs) provide potential solutions to these challenges. Models and applications such as NotebookLM and Google's Gemini 2.5 Flash demonstrate capabilities for processing and reasoning across text, image, and audio modalities [1]. These advances suggest the feasibility of developing educational systems that can process lecture materials across multiple modalities without information loss. This technical foundation motivates our investigation into integrating Retrieval-Augmented Generation (RAG) techniques with multimodal representation learning for educational applications. The proposed system aims to address both institutional content

[1] https://blog.google/technology/ai/notebooklm-google-ai/

management needs at HCMUT and the broader requirements of self-directed multimodal learning.

1.2 Objectives

The primary objective of this project is to design and implement a web-based system that enables students to search, summarize, and study lecture materials through a mRAG pipeline. The proposed system aims to:

- Integrate textual, visual, and audio information from lecture videos, slides, and documents into a unified retrieval system.
- Apply Optical Character Recognition (OCR) and Automatic Speech Recognition (ASR) to extract text from slides and audio content, enabling content indexing and retrieval.
- Employ a hybrid retrieval mechanism that combines sparse and dense embeddings with cross-modal verification for reliable and accurate information retrieval.
- Utilize a Large Language Model (LLM) to ground retrieved evidence and generate coherent, contextually accurate responses.
- Provide personalized study tools, including lecture summarization and study roadmap generation, to improve review efficiency and exam readiness.

Through these objectives, the system aims to enhance accessibility and understanding of complex lecture content, serving as an intelligent assistant that bridges multimodal information with adaptive learning.

1.3 Scope

This project focuses on the development of a prototype system that processes lecture-related multimodal data and supports retrieval, summarization, and study roadmap generation. The scope includes:

- Data ingestion from multiple sources such as videos, slides, and documents.
- Extraction of textual and visual information through OCR and ASR techniques.
- Construction of a multimodal retrieval pipeline using both dense and sparse embeddings.
- Deployment of a RAG-based reasoning layer powered by an LLM.

- Design of a frontend web interface that allows students to search for relevant content and receive summaries or study guidance.

The project does not aim to develop new models from scratch but rather focuses on integrating existing multimodal models and retrieval techniques into a cohesive and practical educational application.

1.4 Challenges

The challenges encountered in this project can be divided into theoretical and technical categories.

Theoretical Challenges

The theoretical challenges stem from the wide range of concepts that must be understood and applied, including:

- Retrieval-Augmented Generation (RAG) and its adaptation to multimodal contexts.
- Prompt engineering strategies for reliable question-answering and summarization.
- Vision-Language Models (VLMs) for visual understanding and alignment.
- Optical Character Recognition (OCR) and Automatic Speech Recognition (ASR) for extracting multimodal input data.
- Retrieval techniques such as dense retrieval, sparse retrieval, and hybrid reranking.
- Multi-embedding approaches for representing diverse data types.
- Architectural design of multimodal pipelines for scalable, efficient processing.

Understanding and synthesizing these theoretical foundations require careful study and analysis to ensure that each component works coherently within the system.

Technical Challenges

The technical challenges lie in implementing and optimizing these theories into a robust and reproducible workflow. These include:

- Integrating heterogeneous processing modules (OCR, ASR, visual embedding, text embedding) into a unified pipeline.

- Ensuring data synchronization between different modalities and storage layers.
- Selecting suitable technologies, frameworks, and models for performance, scalability, and cost-effectiveness.
- Managing cross-modal retrieval and reranking efficiently to minimize latency and improve precision.
- Conducting experiments and evaluations to determine the most suitable configuration for retrieval and summarization.
- (Optional) Deploying the system with considerations for cloud environments, scalability, and real-time performance.

Addressing these challenges requires both theoretical understanding and practical engineering effort to ensure a functional mRAG system that supports real-world educational use cases.

1.5 Structure of the Report

The remainder of this report is organized as follows:

- **Declaration of Authorship** – Confirms that the work presented in this report is original and completed by the authors.
- **Abstract** – Summarizes the motivation, objectives, and contributions of the project in a concise form.
- **Chapter 1: Introduction** – Provides background, motivations, objectives, scope, challenges, and the structure of the report.
- **Chapter 2: Preliminary Study** – Presents foundational knowledge, key theories, and tools relevant to multimodal retrieval and large language models.
- **Chapter 3: Related Work** – Reviews existing literature and prior systems related to multimodal RAG, OCR, ASR, and visual-language integration.
- **Chapter 4: Our Approach** – Describes the proposed architecture, system design, and methodologies for integrating multimodal retrieval and reasoning.
- **Chapter 5: Initial Implementation** – Details the practical setup, model configurations, and partial implementation results.
- **Chapter 6: Evaluation Methods** – Defines the metrics, datasets, and procedures used to evaluate system performance and effectiveness.
- **Chapter 7: Conclusions** – Outlines future work, improvements, and long-term development strategies for the project.

Chapter 2

Preliminaries

2.1 Automatic Speech Recognition

Automatic Speech Recognition (ASR) is the process of converting spoken language into text and has undergone significant evolution, progressing from traditional statistical pipelines to modern end-to-end neural architectures [3, 4]. Its applications span a wide range of domains, including voice assistants, captioning, transcription, and accessibility technologies. Recent research has increasingly focused on enhancing robustness, supporting low-resource languages, and leveraging large-scale pretraining to improve generalization and reduce reliance on labeled data [5, 6]. A typical ASR pipeline is illustrated in Figure 2.1[1].

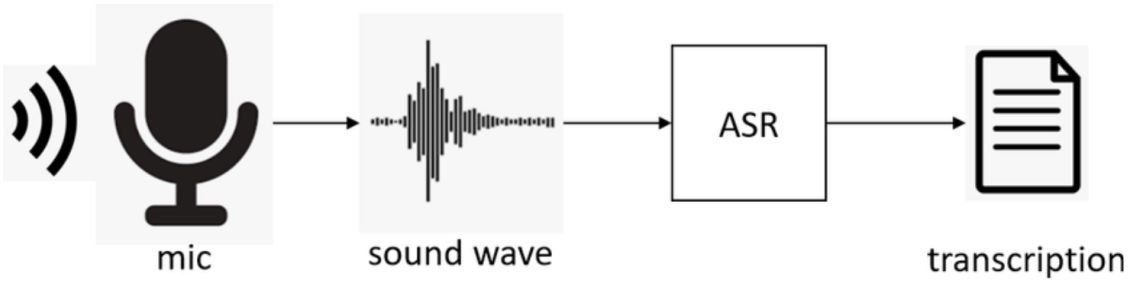

Figure 2.1: ASR pipeline

ASR is the task of converting an acoustic signal into a sequence of words. Formally, the goal of an ASR system is to determine the most probable word sequence $\hat{W}$ given an input acoustic observation X

$$\hat{W} = \arg\max_{W} P(W|X).$$

Using Bayes' theorem, this can be rewritten as

[1] https://www.researchgate.net/figure/ASR-application-pipeline_fig1_355070202

$$\hat{W} = \arg\max_{W} P(X|W)P(W)$$

where:

- $P(X|W)$ is the **acoustic model**, representing the probability of observing the audio features given a word sequence;
- $P(W)$ is the **language model**, representing the prior probability of a word sequence in the target language;
- $\hat{W}$ is the final predicted transcription.

The decoder combines both models to search for the optimal word sequence.

The earliest generation of ASR systems was based on Gaussian Mixture Models (GMMs) combined with Hidden Markov Models (HMMs), which provided temporal alignment and probabilistic modeling of speech signals [7, 8]. These systems relied on n-gram language models and lexicons to capture linguistic structure. Around 2010–2015, the hybrid Deep Neural Network–HMM (DNN–HMM) approach emerged, where deep neural networks replaced GMMs to improve acoustic modeling. This hybrid paradigm dominated production ASR systems for several years, supported by widely used toolkits such as Kaldi [9] and benchmark datasets like Switchboard and WSJ [10, 11], until end-to-end models began demonstrating superior performance.

End-to-end ASR methods simplify traditional pipelines by directly mapping audio signals to text with a single neural model, allowing joint optimization of components. Three major paradigms have shaped this shift. Connectionist Temporal Classification (CTC) enables training without frame-level alignment by marginalizing over possible alignments, making it effective for character or phoneme-based outputs [3]. Attention-based encoder–decoder models, exemplified by Listen, Attend and Spell (LAS), rely on an encoder to produce representations and an attention mechanism to guide token-level predictions, effectively modeling dependencies between outputs [4]. Transducer models, such as the Recurrent Neural Network Transducer (RNN-T) and Transformer Transducer, combine alignment and prediction into a unified probabilistic framework that supports streaming and low-latency applications [12, 13]. Each paradigm has its own trade-offs: CTC models are simple but constrained by conditional independence assumptions, attention-based models excel in accuracy but are less suitable for streaming, while transducer models balance accuracy and real-time usability.

End-to-End ASR using Connectionist Temporal Classification (CTC) models the probability of a label sequence Y by summing over all possible alignments π that collapse to Y

$$P(Y|X) = \sum_{\pi \in \mathcal{B}^{-1}(Y)} P(\pi|X)$$

where $\mathcal{B}$ is the mapping that removes blanks and repeated labels from the alignment path.

The introduction of transformer architectures has further advanced ASR by enabling global context modeling through attention mechanisms. Conformer, a convolution-augmented Transformer, improves upon the pure transformer design by integrating convolutional modules that capture local acoustic patterns while preserving global attention, achieving state-of-the-art performance on standard benchmarks such as LibriSpeech [14, 15]. These innovations have established transformers and conformers as the backbone of modern ASR systems.

In attention-based ASR models, the system generates the output transcription autoregressively, producing one token at a time conditioned on previous outputs and the encoded acoustic features. The probability of generating a transcription sequence Y given the input speech features X is formulated as

$$P(Y|X) = \prod_{t=1}^{T} P(y_t \mid y_{1:t-1}, X)$$

where:

- X is the input acoustic feature sequence extracted from the speech signal;
- $Y = (y_1, y_2, \ldots, y_T)$ is the output sequence of tokens (e.g., characters, subwords, or words);
- y_t is the token predicted at timestep t;
- $y_{1:t-1}$ denotes all previously generated tokens before timestep t;
- T is the length of the output sequence;
- $P(y_t \mid y_{1:t-1}, X)$ is the conditional probability of generating token y_t given the input features and prior predictions.

The decoder uses an attention mechanism to dynamically focus on relevant parts of the encoded speech representation at each timestep, enabling the model to learn soft alignments between acoustic frames and output tokens without explicit alignment labels.

Another major milestone is the rise of self-supervised learning and large-scale pretraining. Models such as wav2vec 2.0 learn powerful audio representations from unlabeled data using contrastive objectives and can be fine-tuned with limited labeled samples to achieve competitive Word Error Rates (WER). Large-scale efforts, including Whisper, demonstrate that scaling both data and model size improves robustness, generalization, and zero-shot transfer across languages and domains. This trend highlights the growing importance of pretraining as a foundation for ASR development.

Datasets and benchmarks play a central role in evaluating ASR progress. LibriSpeech has become the primary benchmark for reporting performance [15], while datasets such as

Switchboard [10], TED-LIUM [16], and Mozilla's Common Voice provide additional testbeds across different domains and languages [17]. Evaluation typically relies on Word Error Rate (WER) and Character Error Rate (CER), with WER remaining the most widely used metric for system comparison.

It measures the difference between the reference transcription and the hypothesis transcription produced by the system. WER is computed as

$$\text{WER} = \frac{S + D + I}{N}$$

where:

- S is the number of word substitutions;
- D is the number of word deletions;
- I is the number of word insertions;
- N is the total number of words in the reference transcript.

A lower WER indicates better recognition performance, with WER = 0 representing a perfect transcription.

Despite advances, practical deployment of ASR introduces further challenges. Many systems still integrate external language models or employ shallow fusion during decoding to improve accuracy. Streaming requirements necessitate architectures tailored for low-latency inference, such as transducer models or streaming variants of attention mechanisms. On-device ASR applications also demand model compression, quantization, and lightweight designs to meet computational and memory constraints.

Robustness, fairness, and safety remain open challenges in the field. ASR systems continue to struggle with noise, accents, microphone variability, and domain shifts. Although large-scale pretraining and data augmentation mitigate some issues, performance disparities persist for underrepresented languages and accents. Furthermore, large ASR models can generate hallucinated transcriptions that appear plausible but are incorrect, necessitating confidence estimation and caution in safety-critical applications.

Emerging directions in ASR research include the development of multilingual and code-switching systems, efficient streaming attention mechanisms for real-time transcription, privacy-preserving and on-device solutions, and the integration of diarization and alignment for richer transcription outputs. These efforts align with the broader trend toward building general-purpose, reliable, and accessible speech recognition technologies.

ASR has advanced from early GMM–HMM systems to end-to-end neural architectures that leverage transformers, conformers, and self-supervised pretraining. While remarkable improvements have been achieved in accuracy and efficiency, challenges in robustness, fairness, and real-time deployment remain pressing. Future progress will likely depend on scaling laws, architectural innovations, and greater diversity in datasets and evaluation

methodologies, ensuring that ASR systems become more equitable, reliable, and widely applicable.

2.2 Optical Character Recognition

Optical Character Recognition (OCR) has long been regarded as a cornerstone technology in the field of document analysis, enabling the automatic extraction of textual information from scanned documents and images. Over the years, its evolution has been marked by significant advancements, beginning with simple rule-based and template-matching approaches and progressing toward sophisticated deep learning models. Today, OCR systems achieve near-human performance across multiple domains, making them an essential component in automation, digitization, and intelligent data processing [18, 19].

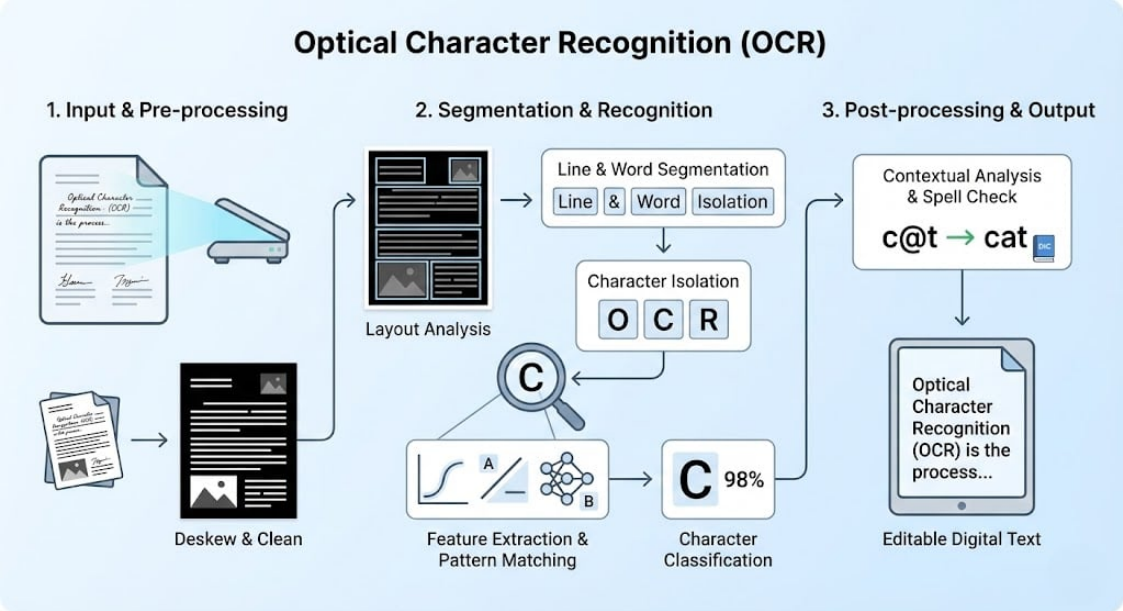

Figure 2.2: OCR pipeline.

The origins of OCR can be traced back to mechanical devices developed in the early 20th century, which were capable of recognizing characters in limited contexts. During the 1950s through the 1970s, template matching emerged as the dominant approach, though it was often constrained by sensitivity to font variations and document quality [20]. By the 1970s, OCR technology saw commercial adoption in industries such as banking and postal services, where it was applied to automate check processing and mail sorting. Despite these advances, early commercial systems lacked robustness, often struggling with noise and inconsistencies in printed text [21].

Conventional OCR pipelines were typically divided into sequential stages: segmentation, feature extraction, and classification. Early methods relied heavily on hand-crafted features such as zoning, projection profiles, and contour analysis, which were then processed using

statistical classifiers, including Support Vector Machines (SVMs) and Hidden Markov Models (HMMs) [22]. While these approaches performed adequately for printed text, handwriting recognition posed greater challenges due to variability in writing styles, cursiveness, and the presence of ligatures. To address this, researchers explored techniques such as elastic matching and Dynamic Time Warping (DTW), which offered improved flexibility. Gradual integration of statistical modeling further enhanced performance, paving the way for more advanced approaches [23].

The introduction of deep learning fundamentally transformed OCR research and practice. Convolutional Neural Networks (CNNs) proved especially effective for character recognition by learning spatial features directly from image data, eliminating the reliance on hand-crafted feature engineering [24]. Recurrent Neural Networks (RNNs) and their variants, particularly Long Short-Term Memory (LSTM) networks, enabled sequence modeling that supported recognition without explicit segmentation, addressing a key limitation of traditional systems [25]. Furthermore, the development of Connectionist Temporal Classification (CTC) provided alignment-free recognition of sequential text, significantly improving end-to-end performance [3]. More recently, transformer-based architectures such as TrOCR have established the state of the art, offering robust recognition across a wide range of scripts and environmental conditions [26].

With advances in deep learning, end-to-end models such as CRNN and Transformer-based architectures directly learn visual-to-text mappings from raw images [27–29]. These models estimate the output text sequence by maximizing the conditional probability given the input image

$$\hat{Y} = \arg\max_{Y} P(Y|X)$$

where X denotes the input image and Y is the predicted text sequence.

Although OCR has significantly improved, challenges remain, including handwritten documents, low resolution images, domain-specific noise, multilingual text, and complex page layouts. These challenges motivate continued research toward more robust recognition models.

To evaluate OCR performance, character-level edit distance is widely used. The Character Error Rate (CER) is defined as

$$\text{CER} = \frac{S + D + I}{N}$$

where S, D, and I are the number of substitutions, deletions, and insertions, respectively, and N is the total number of reference characters. Word Error Rate (WER) applies the same formulation at the word level

$$\text{WER} = \frac{S + D + I}{N}$$

Lower CER and WER values indicate better recognition performance, thus motivating improvements in model architecture, data augmentation, and domain adaptation.

In addition to scanned documents, OCR research has increasingly focused on scene text recognition (STR), which addresses the challenges of extracting text embedded within natural images. Unlike structured documents, STR involves handling irregular backgrounds, distortions, and varying lighting conditions. To meet these challenges, recent methods have combined object detection frameworks such as Faster R-CNN and YOLO with recognition pipelines, thereby improving both localization and recognition accuracy. This integration has enabled OCR systems to achieve strong performance in real-world applications, where text often appears in unconstrained environments [30].

The applications of OCR extend across diverse domains and industries, underscoring its versatility and impact. In the digital humanities, OCR plays a crucial role in large-scale digitization projects aimed at preserving and analyzing historical archives [31]. Within the financial sector, it supports automated check verification and identity document processing, streamlining transactional workflows [32]. In healthcare, OCR facilitates the digitization of medical prescriptions and patient records, enhancing accessibility and reducing manual entry errors [33]. Assistive technologies for visually impaired individuals also rely heavily on OCR, providing real-time access to printed information [34]. Additionally, intelligent transportation systems employ OCR for automatic license plate recognition, contributing to improved traffic management and security [35].

Despite remarkable progress, OCR continues to face several challenges that limit its universality. Multilingual recognition remains difficult, particularly for low-resource languages and scripts with limited annotated data [18]. Historical manuscripts and degraded documents introduce further complexity, as noise and damage hinder recognition accuracy [19]. Handwriting variability remains an unresolved issue, with cursive and unconstrained writing styles posing significant obstacles [23]. Looking ahead, promising research directions include self-supervised learning approaches that reduce dependence on labeled data, multimodal OCR systems that integrate visual and linguistic cues, and few-shot learning methods designed to handle underrepresented languages effectively [26].

OCR has undergone a profound transformation, evolving from rigid template-based recognition to flexible, data-driven deep learning and transformer-based systems. These advancements have enabled robust performance across a wide range of domains, from structured documents to complex natural scenes. As OCR continues to underpin automation, accessibility, and large-scale digitization efforts, ongoing research is likely to focus on improving its robustness, adaptability, and inclusivity. Future developments will ensure that OCR remains an indispensable technology in bridging the gap between visual and textual information.

2.3 Transformer

The Transformer model described in this section is based on the architecture proposed in *Attention Is All You Need* [36]. The Transformer architecture, introduced by Vaswani et al. in 2017, revolutionized sequence modeling by relying entirely on self-attention mechanisms, dispensing with recurrent and convolutional structures. This design enabled efficient parallelization during training and excelled at capturing long-range dependencies, leading to state-of-the-art performance in natural language processing tasks such as machine translation, text generation, and language understanding.

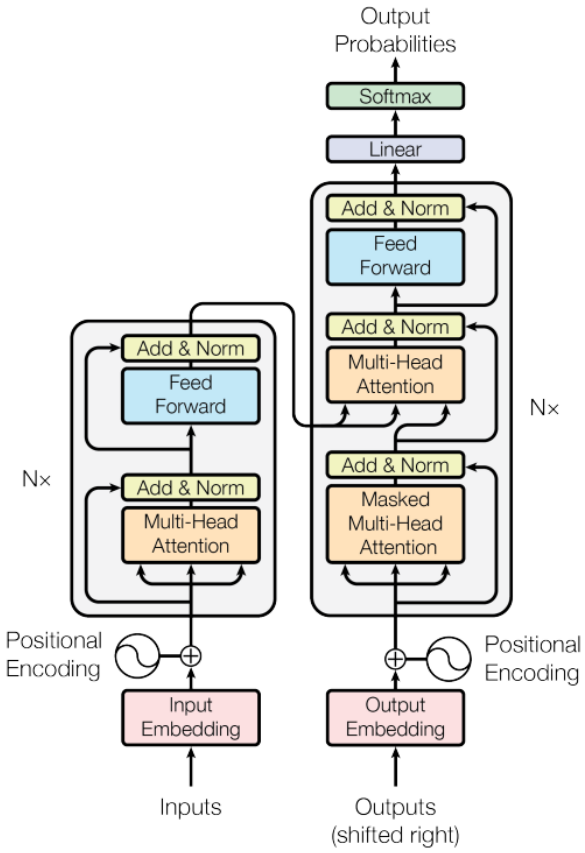

Figure 1: The Transformer - model architecture.

Figure 2.3: Transformer Architecture.

A Transformer consists of an **Encoder** and a **Decoder**, each composed of N identical layers ($N = 6$ in the original implementation).

Each Encoder layer contains two sub-layers:

1. Multi-head self-attention.
2. Position-wise fully connected feed-forward network.

Each sub-layer is wrapped with a residual connection. Formally, the output of each sub-layer is

$$\text{LayerNorm}(x + \text{Sublayer}(x)).$$

Each Decoder layer contains three sub-layers:

1. Masked multi-head self-attention, preventing each position from attending to future tokens.
2. Multi-head attention over the Encoder outputs.
3. Position-wise feed-forward network.

As in the Encoder, residual connections and layer normalization are applied to each sub-layer.

Because the Transformer contains no recurrence or convolution, positional information must be injected explicitly. The model adds Positional Encodings to input embeddings at the bottoms of both the Encoder and Decoder stacks. The original Transformer uses sinusoidal encodings defined as

$$\text{PE}(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right),$$
$$\text{PE}(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right),$$

where pos is the token position and i is the dimension index. Each dimension corresponds to a sinusoid at a different frequency, enabling the model to encode both absolute and relative positional patterns.

Self-attention computes an output for each position by mapping a query vector to a set of key-value pairs.

Similarity between queries and keys is computed via the dot product QK^T, followed by a softmax to obtain attention weights. Because large values can destabilize gradients when the dimensionality d_K is high, the dot products are scaled by $\sqrt{d_K}$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_K}}\right)V.$$

Instead of performing a single attention operation, the Transformer projects queries, keys, and values into h subspaces using learned linear transformations. Attention is computed independently in each head

$$\text{head}_i = \text{Attention}(QW_i^Q,\, KW_i^K,\, VW_i^V).$$

The outputs of all h heads are concatenated and linearly transformed

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O.$$

This mechanism allows the model to capture multiple types of relationships simultaneously by attending to diverse representation subspaces.

2.4 Multimodal Fusion and Cross-Attentional Models

For specific data types and stages within retrieval pipelines, more specialized architectures are necessary. In the context of video, which inherently combine visual frames, audio signals, motion dynamics, and transcripts generated through ASR, reliance on purely visual encoders such as CLIP fails to exploit the full range of available modalities. To address this limitation, the Multi-level Multi-modal Hybrid Fusion (M2HF) model has been proposed. This model integrates three complementary mechanisms. It first applies multi-stream feature extraction in order to capture representations from visual, audio, motion, and ASR-text channels. It then employs attentional fusion, where attention blocks are used to generate enriched cross-modal features, such as audio-guided visual representations, rather than relying on naive concatenation. Finally, it implements multi-level fusion strategies, combining modalities at both the feature level and the decision level, thereby producing richer and more informative retrieval signals [37].

Cross-attentional models constitute another important development, particularly in the reranking stage of retrieval pipelines. Unlike dual-encoder architectures, which encode queries and documents independently, cross-attentional models such as BERT-based cross-encoders compute deep, token-level interactions between queries and documents. This capacity allows them to capture fine-grained semantic distinctions, such as differentiating between "a slide explaining photosynthesis" and "a slide merely mentioning photosynthesis." Their ability to model these nuanced relationships has consistently led to state-of-the-art performance in reranking tasks. However, these gains come at the cost of high computational complexity, which renders such models impractical for large-scale, first-stage retrieval [38, 39]. Instead, their most effective application lies in second-stage reranking, where they refine a smaller set of candidate results returned by a more efficient retrieval mechanism[2]. The cross-attention mechanism is illustrated in Figure 2.4[3].

Cross-attention computes attention weights using queries from one sequence and key-value pairs from another. Given query matrix Q, key matrix K, and value matrix V

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where:

- $Q \in \mathbb{R}^{T_q \times d}$ are queries from the target modality (e.g., decoder text embeddings);
- $K, V \in \mathbb{R}^{T_k \times d}$ are keys and values from the source modality (e.g., encoder speech/image features);
- d_k is the dimension of the key vectors;

[2] https://www.pinecone.io/learn/series/rag/rerankers/
[3] https://magazine.sebastianraschka.com/p/understanding-multimodal-llms

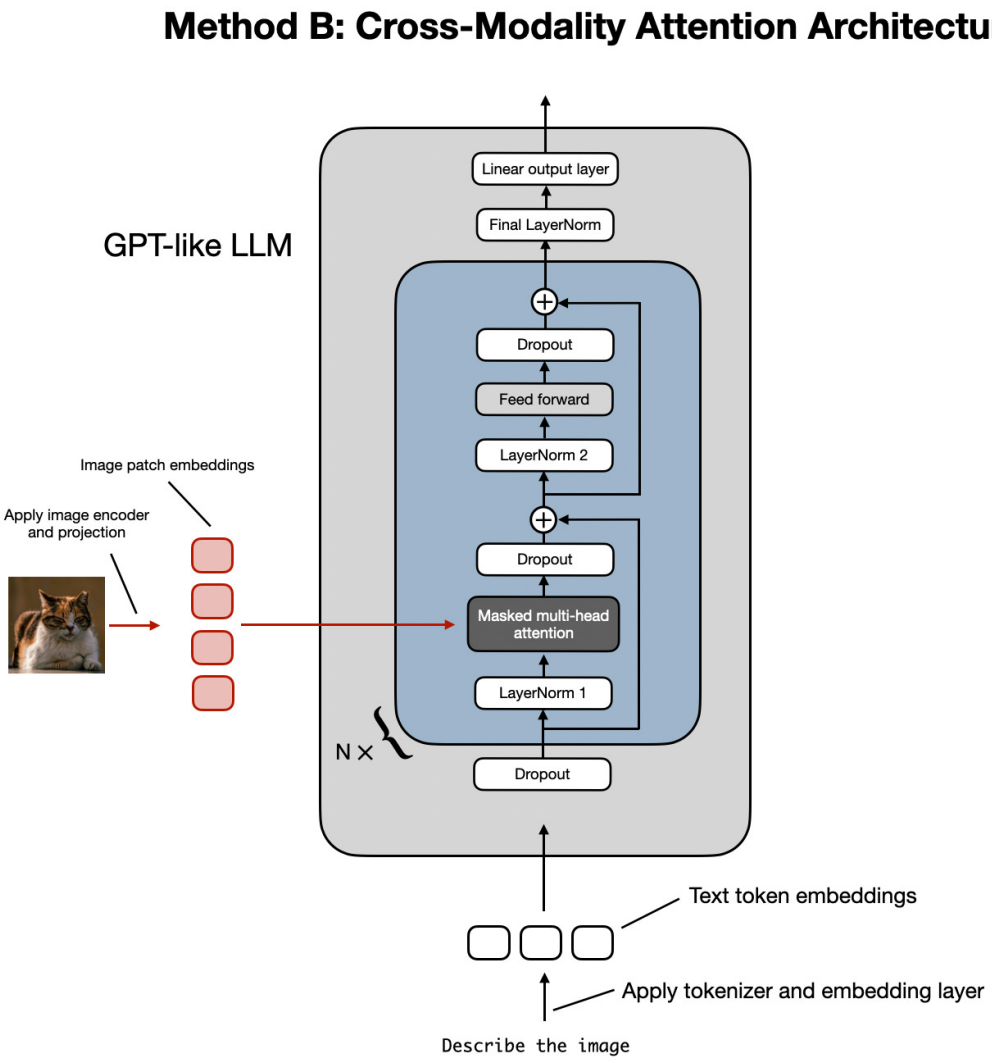

Figure 2.4: Cross-attention

- T_q, T_k are token lengths for target and source sequences.

Multimodal fusion via dual attention for two modalities A and B, cross-attention is computed independently:

$$H_A = \text{Attention}(Q, K_A, V_A);$$

$$H_B = \text{Attention}(Q, K_B, V_B).$$

The fused representation is then computed as

$$H = \alpha H_A + (1 - \alpha) H_B$$

where $0 \leq \alpha \leq 1$ is a learnable fusion coefficient.

2.5 Multimodal Embedding

The rapid development of multimodal embedding architectures has significantly broadened the scope of retrieval systems, enabling richer representations across heterogeneous data sources. Early foundation models such as CLIP (2021) established a strong baseline for cross-modal embeddings by jointly training on 400 million image–text pairs and learning a shared embedding space for both modalities [40]. CLIP's dual-encoder design not only facilitated zero-shot image–text retrieval but also became a widely adopted backbone for subsequent multimodal systems. Building upon this foundation, recent research has extended embeddings to additional modalities and specialized domains.

Each input modality $m \in \{\text{text}, \text{audio}, \text{vision}\}$ is encoded into a latent vector using a modality-specific encoder

$$z_m = f_m(x_m; \theta_m)$$

where x_m is the raw input of modality m, $f_m(\cdot)$ is the corresponding encoder (e.g., Transformer encoder for text, Conformer for audio, ViT for vision), and θ_m denotes its learnable parameters. An example of multimodal embedding is shown in Figure 2.5[4].

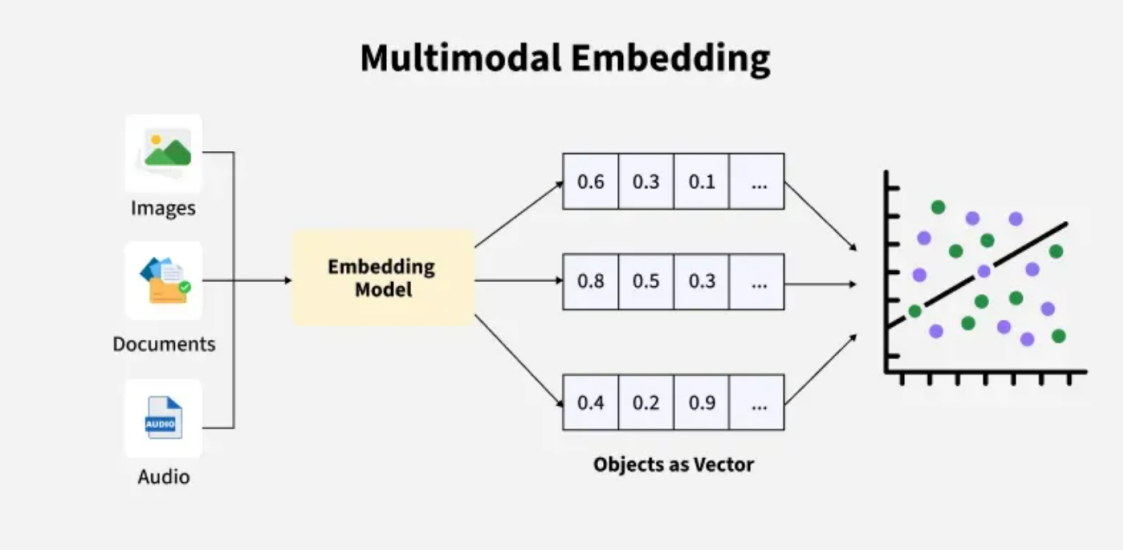

Figure 2.5: Multimodal embedding

We fuse modality embeddings into a shared representation

$$z = g([z_{\text{text}}; z_{\text{audio}}; z_{\text{vision}}]; \phi)$$

where $[z_{\text{text}}; z_{\text{audio}}; z_{\text{vision}}]$ is the concatenation operator, $g([z_{\text{text}}; z_{\text{audio}}; z_{\text{vision}}])$ is a fusion network (e.g., MLP or cross-attention module), and ϕ denotes fusion parameters.

[4] https://newsletter.theaiedge.io/p/how-to-build-a-multimodal-rag-pipeline-8d6

To align representations across modalities, we apply a contrastive loss

$$\mathcal{L}_{\text{contrast}} = -\sum_{i=1}^{N} \log \frac{\exp(\text{sim}(z_i^{(a)}, z_i^{(t)})/\tau)}{\sum_{j=1}^{N} \exp(\text{sim}(z_i^{(a)}, z_j^{(t)})/\tau)}$$

where $z_i^{(a)}$ and $z_i^{(t)}$ are the audio and text embeddings of paired sample i, $\text{sim}(z_i^{(a)}, z_j^{(t)})$ is a similarity function (e.g., cosine similarity), τ is the temperature scaling factor, and N is the batch size.

One line of progress has centered on multimodal large language models (MLLMs). These models, exemplified by LLaVA and LLaMA-2 Vision, can process both visual and textual input and have increasingly been repurposed for retrieval tasks. Lin et al. introduced MM-Embed, a multimodal LLM fine-tuned as a bi-encoder to support queries that combine images and text [41]. While naïve multimodal LLM retrievers displayed modality biases, such as a tendency to over-prioritize text results, MM-Embed reduced these issues by incorporating modality-aware hard negatives during training. This adjustment enabled the model to achieve state-of-the-art performance on the M-BEIR benchmark, a multi-domain multimodal retrieval suite, and it even outperformed leading text-only retrievers on standard text-focused benchmarks. These findings demonstrate that, with careful fine-tuning, large multimodal models can function as powerful and general-purpose retrievers across different modalities.

Another noteworthy development is the emergence of unified multimodal embedding architectures. Meta AI's ImageBind (2023) represents a landmark contribution by integrating six distinct modalities: images, video, text, audio, depth, thermal data, and IMU motion signals into a single shared embedding space [42]. Crucially, ImageBind does not require explicit pairing for every modality. Instead, it learns a joint representation that enables flexible cross-modal retrieval; for instance, an audio recording of a lecture can retrieve a relevant diagram, or a video clip can be matched with its spoken description. This holistic framework signals the potential for future educational systems in which any form of input, whether speech, text, or image, can be used to locate semantically related content across heterogeneous media.

Beyond general-purpose models, domain-specific designs have also gained traction. Educational content frequently blends textual explanations with complex visual structures such as diagrams, charts, and mathematical formulas. To address this, models such as jina-embeddings-v4 (2025) have been proposed [43]. With 3.8 billion parameters, this model integrates text and image data and supports both single-vector embeddings and multi-vector late-interaction embeddings. It further incorporates task-specific adapters, trained via low-rank adaptation (LoRA), for applications including code search and question answering. The model achieves state-of-the-art results on text and multimodal retrieval benchmarks, particularly excelling in tasks involving visually dense content such as tables and lecture slides. To evaluate these capabilities, the authors introduced Jina-VDR, a benchmark for visually rich document retrieval, which underscores the critical importance of handling diagrams and mixed-media content in educational settings.

Parallel efforts in industry have emphasized retrieval over multimodal documents such as PDFs and slides, which combine text layout with images. Nomic's Embed

Multimodal 7B (2025) exemplifies this trend by embedding interleaved text and images for retrieval-augmented generation over documents[5]. While a formal paper for this model is not yet publicly available, industry documentation and benchmark reports indicate that this model achieved a new state-of-the-art on the ViDoRe–v2 visual document retrieval benchmark, recording an NDCG@5 of 62.7 and surpassing prior models by 2.8 points.

Finally, an important trend is the narrowing performance gap between multimodal and single-modal models. For example, the MM-Embed retriever, initialized from the NV-Embed text model, not only excelled at cross-modal retrieval but also outperformed strong text-only retrievers on the text-focused MTEB benchmark. This convergence suggests that training with multimodal data enhances the semantic richness of representations even for unimodal text tasks. Such results are particularly encouraging for educational contexts, where queries often combine textual and visual references, for example, a request to locate "the graph of X" alongside a written description.

2.6 Retrieval-Augmented Generation

2.6.1 Historical Evolution of Retrieval-Augmented Generation

2017–2019: Pre-RAG foundations. Open-domain question answering prior to RAG largely followed a retrieve-then-read pipeline, where a sparse retriever fetched passages and a neural reader extracted answers. DrQA exemplified this architecture and established the modern baseline of TF-IDF/BM25 retrieval paired with a neural reader [44]. Neural methods then began to learn retrieval: ORQA jointly trained a dense retriever and reader from QA supervision, demonstrating that learned dual encoders could outperform purely sparse heuristics while remaining efficient at query time [45]. In parallel, kNN-LMs showed that a language model's predictions can be conditioned on a non-parametric, external datastore during inference, anticipating the later concept of "semi-parametric" models that fluidly combine internal parameters with external memory [46]. Together, these developments set the stage for retrieval and generation to be tightly coupled rather than treated as disjoint stages.

2020: Formalizing retrieval-augmented generation. REALM introduced end-to-end differentiable retrieval during pretraining so that a masked-LM objective could backpropagate through a large document index, demonstrating that external, updateable knowledge could be learned and consulted by the LM at scale [47]. Shortly afterward, RAG combined a dense retriever (DPR) with a sequence-to-sequence generator (BART), training the entire model so that gradients update both retriever and generator. This "general-purpose fine-tuning recipe" showed that a medium-sized LM augmented with learned retrieval can outperform substantially larger parametric-only models on knowledge-intensive tasks [48].

2021–2022: Stronger Retrievers, Multi-Document Reasoning, and Lighter

[5] https://www.nomic.ai/blog/posts/nomic-embed-multimodal

Readers

Dense retrieval advanced on several fronts: RocketQA introduced denoised hard negatives and cross-batch training [49]; RocketQAv2 refined knowledge distillation from cross-encoders to bi-encoders for latency-aware quality [50]. Retriever-specific pretraining (Condenser) made language-model pretraining explicitly serve dense retrieval [51], while unsupervised contrastive pretraining (Contriever) yielded robust, domain-transferable retrievers without labels [52]. Scaling T5 dual encoders (GTR) further improved zero-shot transfer [53]. Beyond bi-encoders, late-interaction architectures (ColBERTv2) compressed token-level matching to make interaction models practical at scale, often used as a re-ranking stage [54]. In sparse-dense hybrids, SPLADE and SPLADE-v2 showed that learned sparsity pairs naturally with dense vectors; hybrid fusion became a "default" for out-of-distribution robustness [55, 56].

At the same time, RAG pipelines learned to retrieve multiple supporting documents: EMDR pushed learning signals across the retrieve→read boundary with an EM-style objective [57], and MDR operationalized multi-hop chains of evidence for compositional questions [58]. On the reader side, Fusion-in-Decoder (FiD) became a standard for fusing many passages, while KG-FiD injected graph structure and GNN-based re-ranking to pass only high-value evidence to the decoder [59, 60]. FiD-Light then reduces context while constraining encoder→decoder flow and source-aware re-ranking, improving the accuracy–latency Pareto frontier [61]. Retrieval-on-the-fly matured via RETRO, which matched the quality of a model roughly 25× larger by consulting a massive index during generation, reinforcing the promise of semi-parametric scaling [62].

Bridging retriever and generator also simplified: Retrieval-as-Attention (ReAtt) folded retrieval into a single transformer trained end-to-end [63], and Re2G unified initial retrieval, re-ranking, and generation in one seq2seq framework [64]. Meanwhile, RAG-oriented query reformulation rose to prominence: GAR enriched queries with generated contexts; Self-Ask and ReAct interleaved decomposition/reasoning with search; WebGPT introduced browser-assisted QA with feedback; and HyDE generated "hypothetical documents" to boost first-stage recall—techniques that remain plug-and-play modules in modern RAG stacks [65–69].

2023–2024: From fixed pipelines to adaptive, verifiable, and structure-aware systems.

Active/adaptive retrieval made when and what to retrieve a learned, closed-loop decision. FLARE triggered retrieval mid-decoding when uncertainty spiked; Adaptive-RAG learned to route among no-retrieval vs. single- or multi-step strategies; SEAKR and DRAGIN dynamically chose whether/what to retrieve conditioned on evolving information needs, cutting cost while improving grounding [70–73]. Robustness moved from "more context fewer hallucinations" to controlling what gets used. SELF-RAG alternated retrieve→generate→critique cycles; CRAG filtered low-quality evidence or refused to answer; CoV-RAG added a chain-of-verification to revise queries/answers; and independent analysis documented the tug-of-war between an LM's priors and retrieved facts motivating verification-aware decoding [74–77].

Context selection and compression became first-class: RankRAG instruction-tuned a single LLM to rank contexts and generate answers, while LLMLingua and a 2024 survey systematized aggressive yet faithful compression to stay within prompt budgets [78–80]. Retrieval broadened to multi-source/agentic settings: AU-RAG introduced source-aware routing among heterogeneous corpora (web/wiki/vertical DBs), and MSPR learned policies for which source to consult and how much to retrieve [81, 82]. Structure-aware retrieval emerged: GraphRAG and GRAG indexed entities/communities and retrieved subgraphs instead of IID chunks, enabling corpus-level synthesis and multi-hop reasoning over relationships [83, 84]. Finally, long-context LLMs met RAG: LongRAG combined long retriever + long reader to keep fewer but larger 4K-token units, and self-routing decided between long-context reading and targeted RAG per query [85, 86].

2025: Agentic control, standardized modules, faithful decoding, evaluation, and memory. By 2025, "Agentic RAG" is crystallized in surveys that codify reflection, planning, tool use, and multi-agent collaboration as control loops over retrieval; HopRAG exemplifies logic-guided hops over a passage graph before generation [87–89]. Complementary surveys standardize the pipeline into four tunable modules (1) retrieval optimization, (2) context processing (re-rank/filter/compress), (3) generation optimization (faithfulness/citation-aware decoding), and (4) evaluation/monitoring collecting robustness and efficiency tactics in each [90, 91]. Generation now explicitly enforces use of evidence: FaithfulRAG models LM-vs-retrieval conflicts and inserts self-thinking before decoding; ReCLAIM constrains decoding with per-sentence citations; CiteFix repairs/validates citations post-hoc; and new studies plus CiteBench/CiteEval move toward principled attribution metrics [92–96]. Evaluation became first-class: ARES provides automated judge-based scoring of context relevance, answer faithfulness, and answer relevance; a 2025 survey consolidates datasets/metrics including abstention and multi-hop fidelity; and competitions like "live RAG" benchmarks (e.g., RAGtifier) standardize end-to-end evaluation on time-varying corpora [97–99].

Finally, RAG is reframed as memory and continual learning: HippoRAG-2 strengthens associative, graph-traversal memory; MemoRAG proposes a dual-system (light long-range LM for global memory + expressive LM for final answers); and agent-memory systems (Mem0, MIRIX, MemOS) treat persistent, structured memories as read/write stores that RAG can consult blurring retrieval, memory, and learning with provenance and versioning [100–104].

2.6.2 mRAG: The Convergence

Pre-RAG signals for vision–language tasks. Before mRAG (mRAG) was named, factual VQA exposed the limits of image-only reasoning. "Straight to the Facts" explicitly learned KB retrieval for VQA, and OK-VQA required outside knowledge, catalyzing retrieval-style VQA where images must be grounded in world facts [105].

2022–2023: Early mRAG patterns. Two threads converged. First, VQA pipelines adopted RAG-style structures: TRiG translated images to text, retrieved passages, and answered purely in NL space pragmatically reusing text-RAG infrastructure [106]; and

differentiable retrieval models, such as entity-focused dense passage retrieval, improved retrieval for OK-VQA [107]. Second, cross-modal embeddings provided the retrieval substrate: CLIP and ALIGN learned shared image–text spaces enabling text-image nearest-neighbor search at scale. REVEAL then pushed end-to-end retrieval-augmented visual-language pre-training with a multi-source multimodal memory, tying a retriever and generator together during pretraining [108].

In generation, retrieval-augmented captioning appeared: Ramos et al. retrieved captions for stronger descriptions, and EVCap added an external "visual-name" memory to keep captions up-to-date in open-world settings. FLMR aligned vision encoders to text retrievers and used multi-dimensional embeddings to capture finer-grained query–document relevance, improving cross-modal matching. Across these systems, the unit of retrieval was predominantly textual (captions/Wikipedia/OCR), even when the queries were visual; the fusion mechanism typically concatenated retrieved text as extra tokens to a VL encoder/decoder.

2024: Maturation – better retrieval and fusion for VLMs, and broader modality bindings. VisRAG explicitly treats visually rich documents as first-class evidence by combining OCR text with visual region cues for retrieval and grounding, bridging VQA-style methods to real-world documents [109]. RAVEN frames retrieval-augmented VLM fine-tuning as a general recipe across tasks [110], while DIR improves both image→text retrieval and the utilization of retrieved text for more complete visual understanding.

In parallel, ImageBind shows a single embedding space for six modalities (image, text, audio, depth, thermal, IMU), widely cited to justify one-index, cross-modal retrieval for mRAG systems that must unify heterogeneous educational assets (slides, lab audio, sensor feeds) [42].

2025: Advanced mRAG – domains, video, documents, and built-in retrieval. Domain-specific mRAG shows substantial gains when retrieval and verification are tuned to the domain. In remote sensing, RS-RAG integrates satellite imagery with world knowledge through a multimodal index and reports large improvements on captioning/classification/VQA [111].

Video becomes a first-class citizen: Multimodal Retrieval-Augmented Generation systems unify information across text, image, table, and video modalities in a single pipeline [112].

Evaluation and instruction-tuning keep pace. A recent survey consolidates components, tasks, and datasets for mRAG and argues empirically that mRAG reduces hallucinations on visually grounded queries compared to text-only RAG [113].

2.6.3 Pipeline

A multimodal RAG pipeline is illustrated in Figure 2.6[6].

The RAG pipeline integrates retrieval mechanisms with generative language models

[6] https://newsletter.theaiedge.io/p/how-to-build-a-multimodal-rag-pipeline-8d6

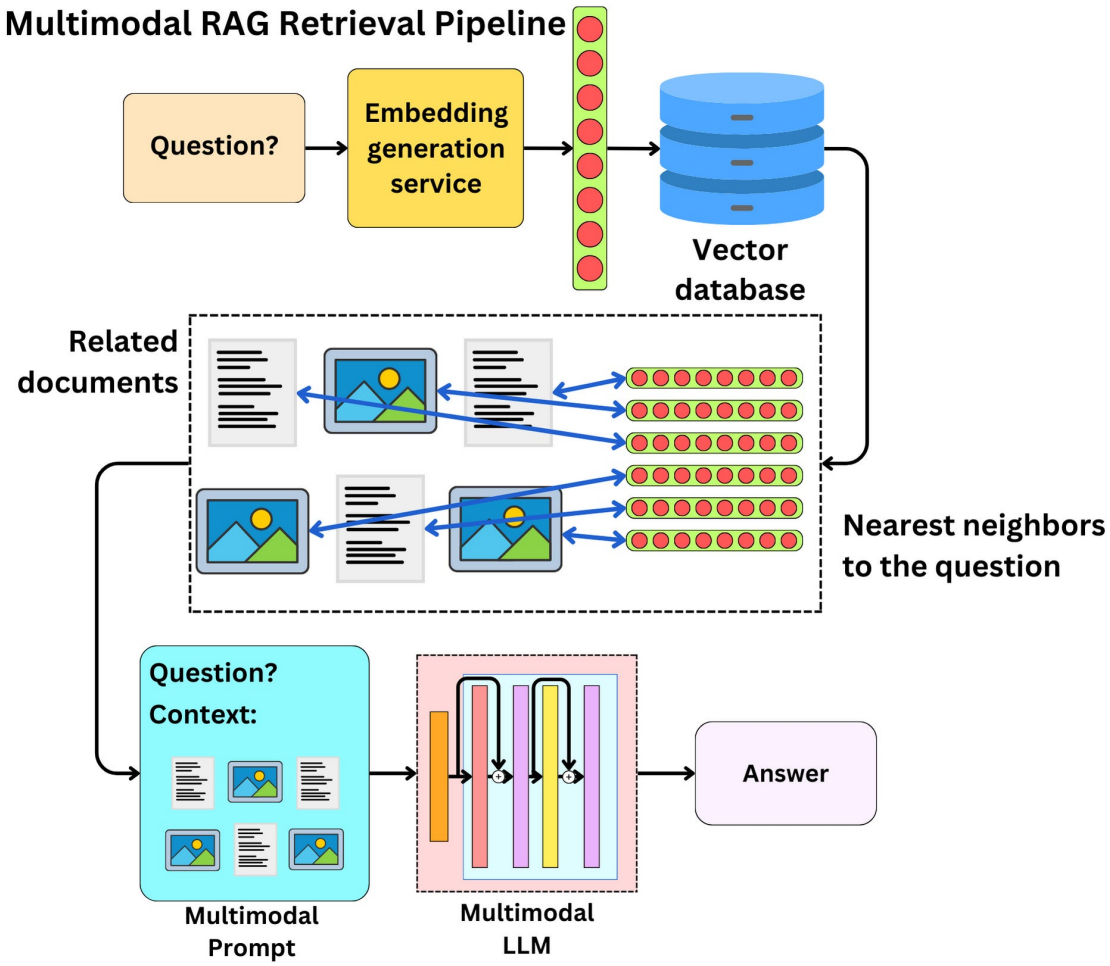

Figure 2.6: Multimodal RAG pipeline

to produce accurate and contextually grounded outputs. The process begins with query vectorization, where the user input is encoded into a high-dimensional vector representation using models such as BERT or Sentence Transformers. This query vector is then matched against a pre-indexed vector database to identify semantically similar entries, allowing the system to retrieve the most relevant documents or passages [48, 114]. After retrieval, the selected data may undergo preprocessing steps for example, summarization, filtering, or restructuring to ensure compatibility with the downstream language model. The processed information is then incorporated into the input context of a large language model (LLM), enabling it to generate responses that combine the retrieved evidence with its own pre-trained knowledge [115].

The Retrieval-Augmented Generation architecture consists of two primary modules: the retriever and the generator. The retriever is responsible for identifying relevant documents or passages from an external knowledge source. It typically operates using methods such as sparse retrieval (e.g., BM25), dense vector retrieval based on embedding similarity, or hybrid/neural retrieval models [48, 116]. These techniques allow the system to efficiently search large-scale corpora and return the most semantically relevant information.

The generator then consumes both the user query and the retrieved context to produce the final output. Modern RAG systems usually rely on transformer-based language models, which condition their decoding process on the retrieved evidence to ensure that the generated response is grounded in factual information and remains aligned with the input query [48, 117].

By combining retrieval with generation, RAG reduces hallucination rates, improves factual accuracy, and enables the model to access up-to-date or domain-specific knowledge without needing to retrain the entire language model. This makes RAG a practical and scalable solution for knowledge-intensive tasks in machine learning and natural language processing [48, 115, 118].

Moreover, recent advances in mRAG further expand its capabilities. For example, mRAG systematically explores different modality configurations (text, image, video, table), retrieval strategies, reranking, and how to integrate retrieved evidence into generation, achieving significant improvements without fine-tuning [119]. Another variant, HM-RAG, proposes a hierarchical multi-agent architecture where multiple retrieval agents (text, graph, web) work collaboratively and feed into a decision agent, improving answer accuracy on multimodal benchmarks [120]. In visually rich QA settings, models like mR^{2}AG adapt retrieval dynamically and reflect on the relevance of retrieved multimodal evidence to guide generation [121]. These innovations illustrate that RAG combined with multimodal retrieval and generation is becoming increasingly powerful especially for complex, knowledge-intensive, multimodal educational applications.

2.6.4 Retrieval

Sparse retrieval models represent the classical approach to information retrieval, mapping texts into high-dimensional, sparse vectors based on lexical features [122, 123]. These methods, such as TF-IDF and BM25, rely on keyword frequency statistics to determine relevance. The resulting vectors excel at exact keyword matching, making them highly effective and interpretable for queries where specific terms are crucial [122]. More advanced models like SPLADE are also based on this sparse representation paradigm [123]. The underlying scoring function for these models can typically be expressed as an inner product between the sparse vector representations of the query and the document. Retrieval is traditionally performed efficiently using an inverted index [123]. However, they can struggle with keyword mismatches, which means they cannot effectively handle semantic similarity where the exact keywords are not present in the document [123]. The BM25 algorithm is illustrated in Figure 2.7[7].

Dense retrieval emerged with the advent of deep learning, employing models to learn low-dimensional, dense vector representations that capture semantic meaning [114]. This approach, often implemented as a dual encoder, maps queries and documents into a shared embedding space where semantic similarity corresponds to vector proximity, typically measured by inner product [114]. Models like BERT-based DPR and SPECTER2 (for

[7] https://www.geeksforgeeks.org/what-is-bm25-best-matching-25-algorithm/

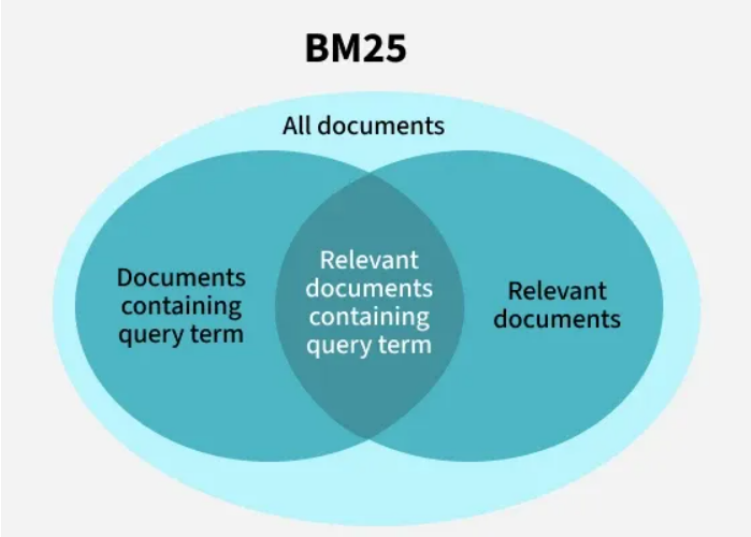

Figure 2.7: BM25

scientific documents) are trained to encode contextual information, allowing them to excel at semantic matching even when keywords differ [114, 124]. These dense vectors can be indexed for efficient large-scale retrieval using Approximate Nearest Neighbor Search (ANNS) algorithms, with graph-based methods like Hierarchical Navigable Small World (HNSW) being particularly effective [125]. While powerful for semantic understanding, dense models can lack the precision needed for exact term matching. Furthermore, theoretical and empirical analyses show that the capacity of fixed-length dense encoders can be limited, with performance potentially degrading as document length increases. This is because longer documents introduce smaller normalized margins between relevant and non-relevant items, requiring a larger encoding dimension to maintain fidelity with sparse models [123].

To leverage the complementary strengths of both paradigms, hybrid retrieval has become a widespread and effective practice, particularly in systems like Retrieval-Augmented Generation [126]. The core idea is to fuse sparse and dense signals to enhance both effectiveness and robustness [114, 123]. A simple yet effective approach is to linearly combine the scores from separate sparse and dense retrievers [114, 123]. Another common strategy is the two-route method, where sparse and dense searches are conducted separately and the results are merged [123]. However, this increases system complexity and can compromise accuracy, as the top hybrid results may not be in the top-k results of either individual system [123]. A more advanced approach is to build a unified index for concatenated dense-sparse hybrid vectors. This presents two key challenges: (1) the significant difference in data distribution between dense and sparse vectors makes it difficult to combine them optimally, potentially hurting accuracy; and (2) the high dimensionality and computational cost of sparse vectors can degrade retrieval efficiency. To address this, recent work proposes methods like distribution alignment (using pre-sampling to determine optimal fusion weights) and an adaptive two-stage computation strategy (initially using only dense distances for coarse search and later refining with hybrid distances) to improve both accuracy and speed [123, 127].

Once embeddings have been generated, the retrieval module identifies and returns the most relevant documents for a given query. In a RAG pipeline, this step is essential because it supplies the language model with context that guides accurate response generation. Formally, given a query q in language x, the retriever returns a document d in language y (which may or may not match x) from a corpus D_y. The retrieval function f_n^* may be implemented using dense, lexical, or multi-vector retrieval methods [128]

$$d_y \leftarrow f_n^*(q_x, D_y).$$

Dense Retrieval: A query q is encoded into hidden states H_q using a text encoder. The representation of the query is the normalized hidden state of the special token `[CLS]`

$$e_q = \text{norm}(H_q[0]).$$

Similarly, for a passage p

$$e_p = \text{norm}(H_p[0]).$$

Relevance is computed as the inner product

$$s_{\text{dense}} = \langle e_p, e_q \rangle.$$

Lexical Retrieval: For each term t in the query

$$w_{qt} = \text{ReLU}(W_{\text{lex}}^\top H_q[i]),$$

and similarly for a passage

$$w_{pt} = \text{ReLU}(W_{\text{lex}}^\top H_p[i]).$$

Where:

$$W_{\text{lex}} \in \mathbb{R}^{d \times 1}; \qquad \text{ReLU}(x) = \max(0, x).$$

Multi-vector Retrieval: Both queries and passages are represented using multiple vectors.

$$E_q = \text{norm}(W_{\text{mul}}^\top H_q), \qquad E_p = \text{norm}(W_{\text{mul}}^\top H_p),$$

where:

$$W_{\text{mul}} \in \mathbb{R}^{d \times d},$$

and the normalization function is applied column-wise.

2.6.5 Rerankers

Rerankers, commonly implemented as cross-encoders, refine search results by jointly encoding a query–document pair to compute a relevance score. Unlike bi-encoders, which independently embed queries and documents into fixed-length vectors, rerankers leverage full token-level interactions, enabling more precise semantic matching. They are

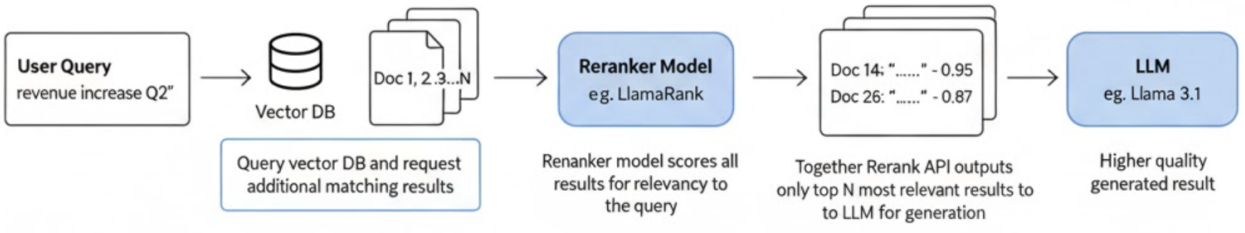

Figure 2.8: Reranker

typically integrated into two-stage retrieval pipelines, where the first-stage retriever generates candidate documents and the reranker refines their ordering to achieve higher accuracy. The reranking process is illustrated in Figure 2.8[8].

Rerankers model token-level cross-attention between the query and document, allowing nuanced reasoning such as entity correspondence, negation detection, factual verification, semantic alignment, and multi-hop reasoning. This yields high relevance precision, especially in long-form document retrieval, question answering, and factual verification tasks.

Rerankers are computationally expensive and therefore operate on a narrowed candidate set rather than the full corpus. This architecture is decomposed as follows:

- Stage 1 (Efficient Retrieval): A bi-encoder or sparse retriever (e.g., BM25, DPR, ColBERT) selects the top-k candidates efficiently using vector similarity or lexical matching.
- Stage 2 (Precision Reranking): A cross-encoder transformer takes (q, d) jointly as input and outputs a scalar relevance score used to reorder the candidate list.

This separation balances scalability and accuracy: fast retrieval reduces search space, while rerankers optimize precision at the top ranks.

Bi-encoders compute representations independently:

$$\text{doc} \to \mathbf{v}_d \in \mathbb{R}^d, \quad \text{query} \to \mathbf{v}_q \in \mathbb{R}^d$$

Similarity is computed via vector metrics (e.g., dot product), which is efficient but compresses document semantics into a single vector without query-context information.

Rerankers instead compute relevance jointly:

$$\text{Reranker}(q, d) \to \text{score} \in \mathbb{R}$$

This allows the model to consider local alignments and contextual dependencies, improving performance on tasks requiring fine-grained semantic reasoning. However, the computation happens online for each query-document pair and cannot be precomputed.

[8] https://www.together.ai/blog/together-rerank-api-and-salesforce-llamarank

Bi-encoder indexing is performed offline, and inference requires only a single query encoding followed by approximate nearest neighbor search. This enables millisecond-scale retrieval over large corpora.

Rerankers require full transformer inference for every candidate document:

$$q + d \rightarrow \text{Transformer} \rightarrow \text{score}$$

As corpus size grows, reranking becomes increasingly expensive. For example, reranking 40M documents with a BERT-based model on a V100 GPU may require over 50 hours per query, while bi-encoder retrieval completes in under 100 ms. Thus, rerankers are typically applied to only 10–1000 candidates rather than the full index.

Impact on Retrieval Quality

Empirically, rerankers consistently improve performance on metrics such as Recall and Mean Reciprocal Rank. They are particularly effective in:

- open-domain question answering
- legal and biomedical retrieval where precision is critical
- multi-step reasoning and claim verification
- reranking passages in RAG pipelines for LLM-based generation

Their ability to leverage token-level attention enables them to differentiate between subtle semantic variations (e.g., contradiction vs. support), which bi-encoders often fail to distinguish due to embedding compression.

- **Bi-encoders:** Scalable retrieval with precomputed embeddings, but limited by lossy compression.
- **Rerankers:** High precision via joint computation, but expensive at query time.
- **Two-stage retrieval:** Combines scalability and precision, forming the standard architecture for modern retrieval systems.

Early neural reranking approaches focused on applying cross-encoders to improve lexical retrieval methods such as BM25. ColBERT-v2 introduced a hybrid architecture combining late interaction with approximate vector search, enabling high recall with reduced latency while preserving token-level semantics [129]. Unlike traditional bi-encoders, ColBERT performs interaction at inference time but maintains efficiency through compressed token embeddings and multi-stage pruning.

MonoT5 advanced reranking by framing document relevance as a sequence-to-sequence task using a generative T5-based model [130]. Instead of predicting similarity scores

directly, MonoT5 reformulates ranking as a text generation problem, allowing language models to leverage pretraining objectives for improved relevance judgment. This approach demonstrated state-of-the-art performance on MS MARCO and BEIR benchmarks, highlighting the effectiveness of generative rerankers.

More recent developments explore instruction-tuned and task-adaptive reranking. TART [131] proposes a unified instruction interface that enables strong zero-shot retrieval across numerous downstream datasets without dataset-specific finetuning. By conditioning reranking on natural language task descriptions, TART generalizes beyond traditional supervised ranking settings.

Recent LLM-based methods push reranking further by using large language models for alignment, reasoning, and verification. RankGPT [132] employs ChatGPT as a reranker through a listwise ranking formulation, enabling the model to score and reorder multiple documents simultaneously. Rather than computing independent pairwise scores, RankGPT uses global contextual reasoning across the candidate set to mitigate position bias and improve ranking coherence.

These methods collectively highlight two major trends in reranking research: (1) scaling token-level interaction while maintaining computational feasibility (e.g., ColBERT-v2), and (2) leveraging generative and instruction-tuned LLMs to improve reasoning and generalization (e.g., MonoT5, TART, RankGPT). In modern RAG pipelines, rerankers increasingly serve not only as precision filters but also as semantic evaluators supporting long-context reasoning, multi-hop retrieval, and factual consistency.

2.6.6 Augmentation

The augmentation process in RAG is illustrated in Figure 2.9[9].

Augmentation in Retrieval-Augmented Generation refers to the process of expanding or transforming retrieved knowledge prior to generation, enabling the model to leverage richer, more structured evidence. Unlike raw retrieval pipelines, augmentation enhances retrieved data through summarization, compression, restructuring, or synthetic expansion, improving relevance and reducing noise [133, 134].

Given a retrieved document set $D = \{d_1, \ldots, d_k\}$, augmentation produces a transformed set $D' = \mathcal{A}(D)$ where $\mathcal{A}$ is an augmentation operator such as summarization, chunk merging, entity extraction, or reasoning-based synthesis. Formally:

$$D' = \mathcal{A}(D) = \{a_1, \ldots, a_m\}, \quad m \leq k \text{ or } m \geq k,$$

where each augmented element a_i may be compressed, reformatted, or newly generated from retrieval.

To preserve semantic coverage while reducing redundancy, augmentation can be framed as an optimization objective:

$$\max_{D'} \ \mathrm{Rel}(D', q) - \lambda \, \mathrm{Redund}(D'),$$

[9] https://obot.ai/resources/learning-center/retrieval-augmented-generation/

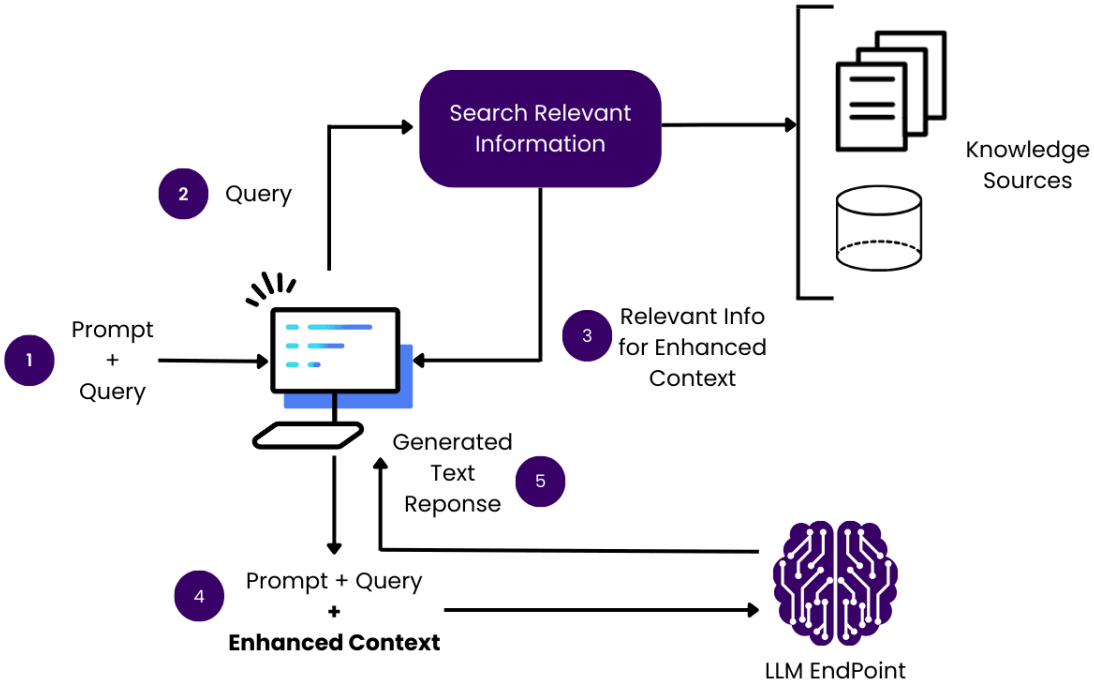

Figure 2.9: Augmentation

where Rel measures relevance to query q and Redund quantifies informational overlap.

A common augmentation strategy is active document compression, where generative models summarize retrieved passages while preserving answer-critical semantics [133]. This can be modeled as:

$$a_i = \arg\max_a P(a \mid d_i, q),$$

and scored using a utility function:

$$s(a_i, q) = \alpha \cdot \text{Sim}(a_i, q) + \beta \cdot \text{Coverage}(a_i, d_i),$$

where:

$$\text{Sim}(a_i, q) = \cos(g(q), h(a_i)), \quad \text{Coverage}(a_i, d_i) = \cos(h(a_i), h(d_i)).$$

Augmentation also enables iterative cycles of retrieval and rewriting. Self-reflection frameworks refine evidence through reasoned critique:

$$D^{(t+1)} = \mathcal{A}\Big(D^{(t)}, y^{(t)}, c^{(t)}\Big),$$

where $y^{(t)}$ is the generated answer and $c^{(t)}$ is a critique model output guiding refinement [134].

Another augmentation paradigm improves retrieval precision for programming tasks, where demonstrations are selected and reformatted into template-based exemplars:

$$a_i = \text{Format}\Big(\text{Demo}(d_i)\Big),$$

supporting structural learning in applications [135].

Augmentation thus serves as a knowledge refinement stage that restructures retrieved content before generation, improving factual grounding, reducing hallucination risk, and increasing domain robustness across tasks such as scientific question answering [118].

2.6.7 Generation

The generator in Retrieval-Augmented Generation serves as the synthesis module that produces the final output by conditioning on both the user query and retrieved evidence. Unlike standalone parametric language models that rely solely on internal representations, RAG generators incorporate retrieved documents to enhance factual grounding and reduce hallucinations [118]. Retrieved knowledge is integrated directly into the decoding process, ensuring that outputs are grounded in external sources rather than purely statistical predictions.

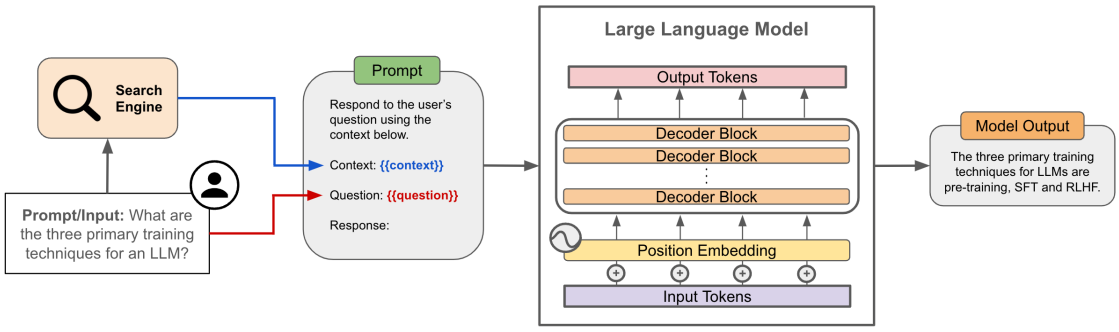

Figure 2.10: Generation

Formally, given a query q retrieves a document set $D = \{d_1, \ldots, d_k\}$, the generator models the output sequence $y = (y_1, \ldots, y_T)$ as:

$$P(y \mid q, D) = \prod_{t=1}^{T} P(y_t \mid y_{<t}, q, D).$$

Instead of treating retrieved documents as a single concatenated input, marginalization across documents can be applied:

$$P(y_t \mid y_{<t}, q, D) = \sum_{d_i \in D} P(y_t \mid y_{<t}, q, d_i)\, P(d_i \mid q).$$

Each retrieved document is encoded into a latent representation:

$$h_i = f_{\text{enc}}(d_i),$$

and decoding attends to these representations:

$$P(y_t \mid y_{<t}, q, d_i) = \text{softmax}(W \cdot \text{Dec}(y_{<t}, h_i)).$$

Modern RAG systems typically adopt large language models built upon autoregressive or encoder-decoder transformer architectures, with retrieved content integrated through concatenation, cross-attention, or late fusion mechanisms [133, 134]. Cross-attention approaches encode retrieved contexts independently, allowing the decoder to selectively attend to relevant passages during token generation and improving evidence utilization at scale.

- **Concatenation-based generation:** Retrieved passages are appended to the prompt; simple but limited by context length and prone to noisy retrieval.
- **Cross-attention generation:** Retrieved representations act as separate attention sources to improve semantic alignment and reduce positional bias [135].

In cross-attention generation, retrieved representations form a secondary attention memory:

$$\text{Attention}(Q, K_D, V_D) = \text{softmax}\left(\frac{QK_D^\top}{\sqrt{d_k}}\right)V_D,$$

where K_D, V_D are keys and values derived from retrieval encodings.

- **Rerank-and-generate pipelines:** Only top-ranked chunks are passed to the generator after evidence filtration [136].
- **Self-reflective iterative generation:** Retrieval, generation, and critique occur recursively to enhance factual accuracy and reasoning [134].

To further improve faithfulness, contrastive losses can be incorporated to prefer evidence-grounded explanations:

$$\mathcal{L}_{\text{faith}} = -\log \frac{\exp(s(y, D^+))}{\exp(s(y, D^+)) + \exp(s(y, D^-))},$$

where D^+ contains supporting evidence and D^- contains retrieved distractors. The score function $s(y, D)$ measures how well a generated output y is grounded in a document set D, commonly implemented as a similarity function such as:

$$s(y, D) = \cos\left(g(y),\ h(D)\right),$$

where $g(y)$ encodes the generated text and $h(D)$ aggregates document embeddings (e.g., mean pooling or attention-weighted fusion).

2.6.8 Prompt Engineering

Prompt engineering refers to the design and optimization of input prompts used to condition and control the behavior of large language models (LLMs) during inference. Since prompts

act as the primary interface between users and pretrained models, they provide a mechanism for steering responses without modifying model parameters [137].

A foundational line of work explores how different prompting formats affect task performance. **Zero-shot prompting** relies solely on natural language instructions, enabling the model to generalize based on pretrained knowledge without additional examples [138]. **One-shot** and **few-shot prompting** extend this by including one or multiple demonstration pairs, allowing models to infer task structure through in-context examples rather than fine-tuning [137]. This form of in-context learning has proven effective across classification, extraction, and reasoning tasks. More advanced prompting strategies target reasoning quality. **Chain-of-Thought (CoT) prompting** encourages models to generate intermediate reasoning steps before producing a final answer, improving performance on multi-step logical and mathematical tasks [139]. Subsequent techniques such as self-consistency aggregate multiple sampled reasoning paths to enhance stability and accuracy [140].

Prompt engineering also underpins efforts to produce **structured and reliable outputs**. Schema-based prompts can enforce formats such as JSON, bullet lists, and domain-specific templates, enabling seamless integration with downstream systems[10]. These constraints reduce hallucinations and parsing failures in production settings. Modern APIs further extend this idea via function calling interfaces, where models output well-defined data structures rather than free-form text, improving determinism and safety in tool-augmented pipelines. Early prompt engineering relied heavily on manual experimentation, making it labor-intensive and heuristic-driven. Recent research has shifted toward automated strategies that systematically search, optimize, or learn prompts using algorithmic techniques. A direct approach to **automated prompt optimization** is searching over a space of candidate prompts and selecting the best-performing one based on validation metrics. This includes searching over instruction phrasing, ordering of few-shot examples, or choice of examples. Since exhaustive search is infeasible due to a large combinatorial space, recent work frames prompt selection as a multi-armed bandit or best-arm identification problem, where each prompt is an "arm" with an unknown reward. For example, TRIPLE [141] uses bandit algorithms to balance exploration and exploitation under a limited query budget, yielding better prompts with fewer evaluations. Other approaches employ evolutionary search or Monte Carlo tree search, demonstrating that automated discovery can outperform manually engineered prompts. Beyond discrete text search, another line of work treats prompts as continuous parameters. Prefix-Tuning [142] learns soft prompt vectors that steer model behavior without modifying model weights. AutoPrompt [143] performs gradient-guided search over discrete tokens, iteratively updating trigger tokens based on gradient signals. AutoPrompt showed that masked language models can perform tasks such as sentiment classification and natural language inference near state-of-the-art accuracy without fine-tuning, and also demonstrated improved factual recall on probing benchmarks. These approaches bridge classic training with prompt design, though continuous prompts may sacrifice interpretability. Other methods refine an initial human-written prompt through iterative edits. GrIPS [144] applies gradient-free heuristic edits adding, removing,

[10]OpenAI GPT-4 Technical Report, https://openai.com/research/gpt-4

or reordering phrases and retains only modifications that improve performance, making it suitable for closed-source models. Notably, optimized prompts sometimes appear nonsensical to humans yet perform better, revealing model-specific quirks. Instruction Induction and Automatic Prompt Engineer (APE) [145] generate candidate instructions using an LLM itself, then score and select the best. APE notably discovered a more effective variant of chain-of-thought prompting ("Let's work this out in a step-by-step way to be sure we have the right answer.") that exceeded human-designed prompts on tasks. Recent work such as OPRO [146] frames prompt optimization as a task performed by the LLM itself. A meta-prompt maintains a history of candidate prompts and their performance, and the model proposes new candidates iteratively. This closes the optimization loop and enables the model to generate self-improving prompts. While promising, meta-prompting must avoid trivial variations and overfitting to validation data. Reinforcement learning (RL) offers another avenue by treating prompts as actions optimized for a reward function, such as validation accuracy or human preference. RLPrompt [147] applies policy gradients to optimize discrete prompt tokens, achieving improvements over manual baselines. Although less widely adopted due to large search spaces and sparse rewards, RL-based approaches align naturally with interactive and user-feedback-driven prompting scenarios. These developments illustrate a shift from heuristic prompt crafting toward principled optimization pipelines, leveraging search, gradients, RL, and LLM-driven meta-optimization to discover high-performing prompts beyond human intuition.

Overall, prompt engineering has matured from ad-hoc phrasing tricks into a systematic methodology spanning demonstrations, reasoning scaffolds, structural constraints, and automated optimization. As LLMs scale and deploy into real-world applications, prompt engineering remains central to achieving reliable, controllable, and domain-aligned model behavior.

Chapter 3

Related Work

3.1 Retrieval Methodologies for Educational Content

Designing an effective search or question-answering system for educational settings requires adapting retrieval methodologies to the type of content involved, whether it be plain text such as lecture notes and transcripts, slides containing both text and figures, or video recordings of lectures. The current best practice favors a two-stage retrieval architecture that balances scalability with precision.

In the first stage, retrieval is broad and efficient, designed to quickly narrow down a large collection to a smaller candidate set. Hybrid sparse–dense retrieval has proven especially effective for text-heavy materials such as lecture transcripts, textbooks, or slide decks. In this approach, a sparse retriever such as BM25 or SPLADE is combined with a dense retriever such as a bi-encoder BERT or SciBERT. Sparse retrieval ensures that exact technical terms, formulas, or names are not overlooked, while dense retrieval provides semantic matching that can capture paraphrases or rewordings of concepts. For instance, a query about a "photosynthesis diagram" would benefit from sparse retrieval if the exact term appears in the document, while a dense retriever could still surface relevant passages describing plant energy processes even when phrased differently. Empirical evidence supports the robustness of the hybrid approach; Luan et al. (2021) demonstrated that combining sparse and dense signals yields superior performance over either method alone, particularly in scientific domains [116]. To scale hybrid retrieval, efficient vector indices such as hierarchical navigable small-world (HNSW) graphs can be extended to support hybrid vectors or to efficiently merge results [148]. Many retrieval-augmented generation (RAG) systems for education adopt this hybrid method on a knowledge base of lessons or FAQs to ensure high recall.

For video lectures or other multimodal content, dense retrieval based on multimodal embeddings is increasingly common. In this section, we recall some backgrounds about multimodal fusion approaches. Models such as M2HF index video segments into dense vectors that encode visual frames, audio cues, and ASR transcript information [37]. Text queries are mapped into the same embedding space, enabling cross-modal retrieval. This approach is advantageous over relying solely on transcript text, as it allows the retrieval of

segments in which the visual modality is decisive for example, locating "the slide showing the cell cycle" even when the transcript contains only a vague reference. The challenge of index size is addressed by segmenting videos into short windows, each represented by high-dimensional embeddings, and using approximate nearest neighbor search (e.g., FAISS or Annoy) to maintain efficiency. In addition, if transcripts or slide text are available, learned sparse retrieval can be combined with multimodal signals. Research on multimodal learned sparse retrieval has shown that purely visual features are not easily expressed as sparse signals, but combining OCR-extracted text and captions with visual tags produces more effective retrieval. In practice, educational video retrieval often relies on maintaining both dense indices for semantic matches and inverted indices for textual signals.

Query adaptation constitutes another important dimension. For specialized educational queries, preprocessing or expansion may be required. For example, mathematical queries may be normalized by converting LaTeX into textual equivalents to ensure matches across both formats. Large language models (LLMs) can also be used to generate paraphrases of queries or, more experimentally, to produce synthetic images from text queries that can serve as additional search modalities. Although still exploratory, such query-driven modality augmentation holds promise for educational scenarios, such as automatically generating diagrams from student questions to facilitate diagram-based retrieval.

In the second stage, reranking is applied to the top candidates typically the top 50 or 100 using slower but more precise methods. Cross-attentional rerankers, often based on transformer models such as BERT, take both the query and the candidate as input and compute a joint relevance score. This approach captures subtle distinctions that bi-encoder methods miss, such as differentiating between “a slide that mentions photosynthesis” and “a slide that explains photosynthesis in detail.” While computationally expensive, reranking remains feasible at this stage since only a limited number of candidates are evaluated. Multimodal cross-attentional rerankers further enrich this process by incorporating visual or layout information alongside text.

More recently, LLM-based reranking has emerged as a promising paradigm. Instead of relying on a fixed, pre-trained reranker, a multimodal LLM can be prompted to evaluate the relevance of retrieved passages directly or to synthesize an answer using retrieved evidence. For instance, DynamicRAG uses outputs from an LLM as feedback for reranking in a RAG pipeline, dynamically adjusting which passages to keep [149]. Similarly, in a mRAG setup, more advanced relevancy measures have been proposed to better select which images, captions, or text to feed to the LLM, thereby reducing noise in the context [150]. Such methods blur the line between retrieval and generation, providing both reranking and contextualized responses, and have been increasingly applied to educational chatbots and tutoring systems.

In practice, educational retrieval pipelines combine these methods to maximize performance. A typical system might retrieve textbook passages through hybrid retrieval, video segments through multimodal dense retrieval, and then rerank all candidates using either a cross-attentional model or a multimodal LLM. This two-stage strategy has become the prevailing standard because it delivers an effective balance between recall in the first stage and precision in the final ranking, thereby ensuring that the retrieved answers are

both comprehensive and contextually relevant.

3.2 Educational applications

3.2.1 Multimodal Retrieval

To support evaluation and foster progress in multimodal retrieval for educational applications, researchers have assembled several new datasets that capture the multimodal nature of academic content. Between 2023 and 2025, multiple benchmarks have been introduced to reflect the diversity of lecture materials, textbooks, and visually rich documents. The Lecture Presentations Multimodal (LPM) dataset, released at ICCV 2023, is a notable example [151]. It contains approximately 180 hours of lecture videos delivered by ten lecturers across multiple subjects, aligned with more than 9,000 lecture slides. The dataset provides high-quality annotations linking slide content, including both figures and text, with corresponding speech segments. LPM defines three tasks that are directly relevant to education: figure-to-text retrieval, where a figure from a slide must be matched to the appropriate spoken segment; text-to-figure retrieval, where a segment of lecture speech or transcript is used to identify the relevant illustrative figure; and the generation of slide explanations. Baselines and human performance have been reported, and even fine-tuned vision–language models encountered significant difficulty on these tasks. This dataset has already inspired follow-up work, such as PolyViLT, which seeks to improve cross-modal alignment between slides and speech.

Another dataset is CK-12 Textbook QA (CK12-QA), which, although originally designed for question answering, doubles as a testbed for retrieval-augmented reasoning. Alawwad et al. (2025) propose a multimodal document ranking and retrieval supervision method for CK12-QA, showing that retrieval-augmented generation significantly helps when retrieving both text and diagrams from lessons [152]. Their findings revealed that retrieval substantially improves reasoning performance, though challenges remain, particularly in aligning question references with the correct diagram.

Beyond these examples, additional academic lecture video datasets have been introduced. The M^{3}AV dataset (ACL 2024) comprises 367 hours of university lectures in computer science, mathematics, and medicine, with transcripts and OCR-extracted slide text, including mathematical formulas, aligned with the videos [153]. Although M^{3}AV emphasizes recognition tasks such as ASR and slide generation, it provides rich multimodal data that can be repurposed for retrieval research. Earlier resources, such as lecture collections from VideoLectures.NET, contained smaller-scale alignments of subtitles and slides, but LPM and M^{3}AV substantially scale up both the size and the quality of annotations, enabling more rigorous evaluations of lecture segment retrieval.

Another relevant area involves benchmarks for visually rich document retrieval. The Jina-VDR (Visually Rich Document Retrieval) benchmark suite was introduced alongside Jina-Embeddings-v4, covering multimodal and multilingual document types (charts, tables, scanned pages, etc.) [43]. Improved performance on these tasks directly translates into

more effective educational resource search, such as retrieving the precise slide containing a particular chart referenced in a query.

General-purpose cross-modal retrieval benchmarks also continue to play a role. Datasets such as MSR-VTT, MSVD, LSMDC, Flickr30k, and COCO provide open-domain evaluation tasks, while BEIR and MTEB offer text-focused retrieval suites. For example, the M^{3}HF model reported strong results on MSR-VTT, MSVD, DiDeMo, and ActivityNet Captions, demonstrating robustness across popular video–text retrieval benchmarks. More recently, the introduction of M-BEIR extended BEIR into multimodal domains, offering a broad evaluation suite that includes both text-only and cross-modal retrieval tasks. This extension has been adopted in the training of models such as MM-Embed, which benefits from exposure to multiple retrieval domains, some of which share similarities with educational materials.

In summary, the ecosystem of benchmarks has rapidly expanded to reflect educational scenarios, including lecture alignment datasets such as LPM, textbook-based reasoning datasets such as CK12-QA, and visually rich document retrieval datasets such as Jina-VDR. Nevertheless, important gaps remain. In particular, a large-scale benchmark for within-video moment retrieval in lecture recordings designed to locate the exact timestamp at which a given concept is explained has yet to be developed. As multimodal applications in education continue to grow, it is likely that the research community will address these gaps by creating more task-specific datasets.

3.2.2 Modality Fusion

A central challenge in multimodal retrieval is determining how to effectively fuse information from different modalities. The choice of fusion strategy whether early fusion, late fusion, or a hybrid approach directly influences the quality of the learned embeddings and the effectiveness of the retriever. Recent work has explored these strategies through multi-stage pipelines that aim to capture both coarse- and fine-grained alignments across modalities.

Early fusion, or feature-level fusion, merges modalities prior to computing similarity. This approach yields joint representations that more richly encode cross-modal interactions. For example, the M2HF model for video retrieval combines CLIP-based visual features with audio features to generate audio-guided visual representations that highlight sound-producing regions within frames, and fuses motion features derived from optical flow with visual features to emphasize movement. In this way, a video segment can be embedded as a single vector capturing synchronized cues across modalities, which is particularly useful in lecture settings where spoken language, slide text, and gestures jointly define relevance.

Late fusion, or decision-level fusion, instead computes similarity scores separately for each modality and combines them at the output stage, typically through weighted score averaging or rank fusion. In an educational search context, a query may be matched independently against slide text and slide images, with the results subsequently aggregated. Late fusion offers robustness and modularity, as demonstrated by cases in which a keyword hit in the ASR transcript can elevate a candidate despite weak visual similarity. Many retrieval pipelines adopt late fusion as part of a two-stage process, where candidates are retrieved using one

modality and reranked using a cross-modal model.

Hybrid fusion strategies have recently gained attention as they combine the strengths of both approaches. M2HF exemplifies such a design by performing early fusion within video streams (e.g., audio–visual and motion–visual) and then applying text–video retrieval at multiple levels, followed by late fusion of the resulting scores. This multi-level approach significantly improved recall across diverse video retrieval tasks and allowed M2HF to achieve strong performance on benchmarks such as MSR-VTT, MSVD, LSMDC, DiDeMo, and ActivityNet.

A further refinement is cross-attentional fusion, which departs from dual-encoder architectures that generate independent embeddings per modality. Instead, cross-encoder models compute direct attention between modalities and between query and item, enabling deep, token-level fusion. For example, the PolyViLT model ingests aligned slide text, figures, and speech transcripts and applies cross-attention with a multi-instance learning (MIL) loss to align slide figures with specific spoken segments [151]. By capturing these fine-grained many-to-many alignments such as a single spoken segment referring to one figure among many on a slide cross-attentional models outperform independent embedding methods for retrieval of figure–speech pairs.

However, the computational cost of cross-attention makes such models impractical for large-scale first-stage retrieval, though they are highly effective in reranking top candidates. An important practical dimension of fusion is modality balancing, which ensures that no single modality dominates the relevance scoring process. Techniques such as modality-specific weighting and loss balancing are commonly used to achieve this. In M2HF, a multimodal balance loss was introduced to encourage the model to utilize audio and motion signals alongside visual cues. This balancing is essential in educational contexts for instance, in mathematics lectures, transcripts may not explicitly describe the content of diagrams, making the visual modality indispensable for accurate retrieval.

3.2.3 NotebookLM

NotebookLM is an AI-powered note-taking and research assistant developed by Google Labs[1], built on the Gemini family of models[2]. Designed to synthesize information from user-uploaded materials, the system emphasizes grounded responses, multimodal comprehension, and support for research workflows. While NotebookLM is marketed as a general-purpose productivity tool, its capabilities overlap with multimodal educational assistants such as our proposed system for HCMUT lectures.

NotebookLM supports diverse input formats, including PDF documents, Google Docs, Google Slides, web URLs, text files, and YouTube links [3]. The system performs structured ingestion and can interpret textual content as well as visual elements such as tables, diagrams, and slide images. Recent updates introduce multimodal reasoning, enabling question

[1] https://blog.google/technology/ai/notebooklm-google-ai/
[2] https://deepmind.google/technologies/gemini/
[3] https://notebooklm.google.com/get-started

answering based on charts, images, and transcripts rather than purely textual content [4].

This multimodal capability aligns with academic materials that combine formulas, diagrams, and spoken explanations. However, NotebookLM treats uploaded materials as isolated documents rather than components of a structured pedagogical sequence, limiting its effectiveness for course-level organization.

NotebookLM employs Retrieval-Augmented Generation to ensure responses are grounded in uploaded documents rather than solely in parametric model memory [**rag-framework**]. When answering a query, the system retrieves relevant text segments and conditions generation on them, frequently accompanied by inline citations that reference specific sections of the source material. This reduces hallucinations and supports academic verification [5].

Despite this grounding, complex analytical tasks such as mathematical reasoning, algorithm evaluation, or code execution may still rely on internal model reasoning and can produce incorrect interpretations without external validation layers. The NotebookLM interface is shown in Figure 3.1[6].

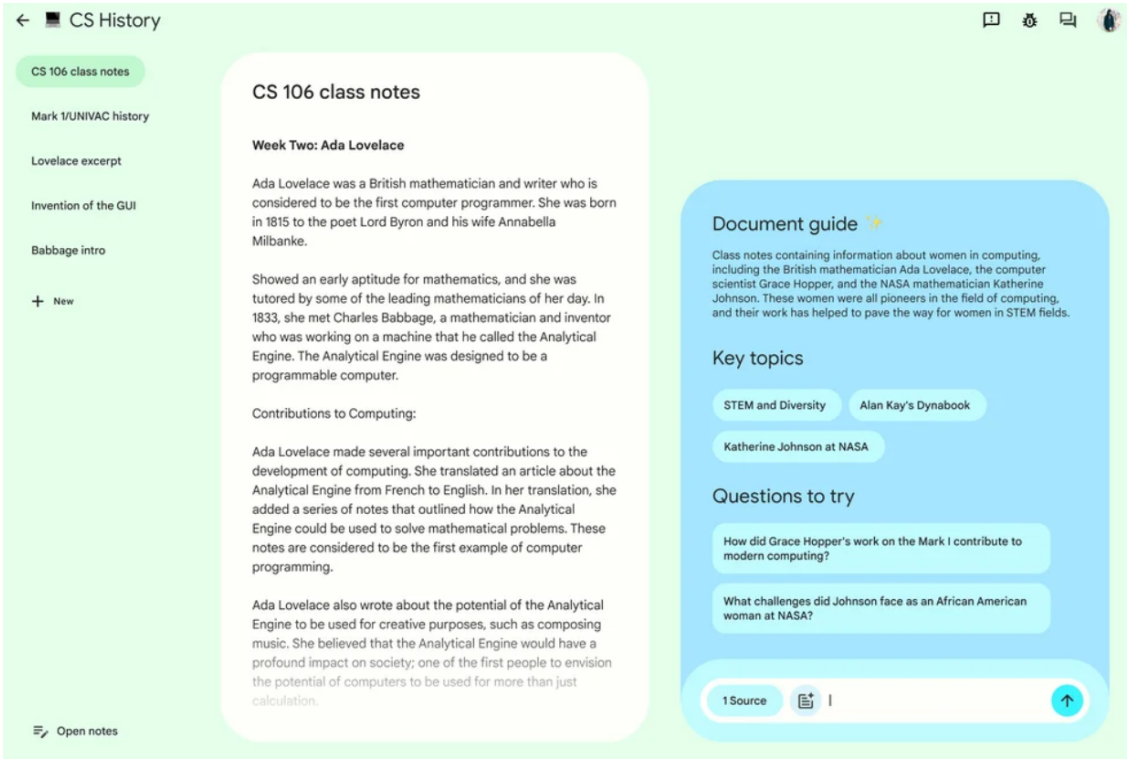

Figure 3.1: NotebookLM

Beyond static summarization and Q&A, NotebookLM introduces several learning-focused features[7]:

[4] https://blog.google/technology/google-labs/notebooklm-updates/
[5] https://blog.google/technology/ai/notebooklm-citations/
[6] https://blog.google/technology/ai/notebooklm-google-ai/
[7] Features include structured summaries (https://blog.google/technology/ai/notebooklm-summaries/),

- Structured document summaries [8],
- AI-generated flashcards and quizzes for active recall [9],
- Podcast-style audio overviews generated from uploaded materials [10],
- Narrated visual summaries similar to lecture recap videos [11]

These features support multiple learning modalities (reading, listening, reviewing). However, generated study resources depend on document completeness and may not capture contextual elements conveyed verbally during lectures.

NotebookLM enforces privacy by not using uploaded materials for model training and storing sources privately unless shared [12]. This makes the tool suitable for proprietary lecture materials. The system is deployed on both web and mobile platforms, supporting flexible access [13].

In contrast, an institutional deployment such as our proposed system may require deeper integration with internal platforms, including lecture capture pipelines, LMS authentication, and course-specific permissions.

Although NotebookLM provides strong baseline functionality, several limitations motivate our specialized system:

Table 3.1: Comparison between NotebookLM and our proposed system.

NotebookLM Limitation	**Opportunity for Our System**
Manual document uploads	Automatic ingestion from slides, recordings, LMS
Document-centric representation	Course-aware structuring by lecture/week/topic
Limited semantic reasoning for math, code, circuits	Domain-specific multimodal understanding (STEM-focused)
Generic responses without instructor oversight	Instructor feedback loops and domain-tuned prompting
Summaries not pedagogically sequenced	Curriculum-aligned learning path generation

flashcards and quizzes (https://blog.google/technology/ai/notebooklm-learning-tools/), audio overviews (https://blog.google/technology/ai/notebooklm-audio-overviews/), and visual summaries (https://blog.google/technology/ai/notebooklm-video-overviews/).

[8] https://blog.google/technology/ai/notebooklm-summaries/

[9] https://blog.google/technology/ai/notebooklm-learning-tools/

[10] https://blog.google/technology/ai/notebooklm-audio-overviews/

[11] https://blog.google/technology/ai/notebooklm-audio-overviews/

[12] https://notebooklm.google.com/privacy

[13] https://blog.google/technology/apps/notebooklm-mobile/

Thus, while NotebookLM is an effective multimodal research assistant, it is not optimized for end-to-end lecture understanding. Our system aims to bridge this gap through structured course-centric knowledge modeling, domain-specific reasoning, and automated ingestion of lecture materials.

Chapter 4

Our Approach

This chapter presents the methodological framework and empirical evaluation of the Retrieval-Augmented Generation pipelines developed for this project. We investigate the efficacy of various retrieval strategies, ranging from lexical matching to advanced late-interaction neural architectures.

4.1 Experiments on Embeddings and Retrieval Methods

4.1.1 Experimental Methodology

Dataset and Computing Environment To benchmark the performance of different retrieval architectures, we utilized the **Microsoft MS MARCO** (Machine Reading Comprehension) dataset[1]. This dataset is a standard benchmark for information retrieval tasks.

Due to computational constraints and to facilitate rapid iterative testing, we curated a representative subset of the data:

- **Corpus Size:** 50,000 documents (reduced from the original ≈1 million).
- **Test Queries:** 556 distinct evaluation queries.
- **Hardware Infrastructure:** All experiments were conducted on Google Colab utilizing a single **Tesla T4 GPU**.

Scoring Formulations Different pipelines utilize distinct mathematical formulations to calculate the relevance score $S(Q, D)$ between a query Q and a document D.

1. Lexical Scoring (BM25)

[1] https://huggingface.co/datasets/microsoft/ms_marco

For the sparse retrieval baseline, we employed the BM25 algorithm. This probabilistic model estimates relevance based on term frequency (TF) and inverse document frequency (IDF), normalizing for document length

$$\text{Score}_{\text{BM25}}(Q, D) = \sum_{t \in Q} \text{IDF}(t) \cdot \frac{\text{TF}(t, D) \cdot (k_1 + 1)}{\text{TF}(t, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

This method focuses on exact keyword matching and is highly effective for queries containing specific identifiers (e.g., proper nouns), though it lacks semantic understanding.

2. Semantic Scoring (Dense Retrieval)

For dense retrieval (e.g., MiniLM, BGE), we encode both query and document into fixed-size vectors ($\vec{V_Q}, \vec{V_D}$) and calculate the Cosine Similarity. This captures semantic meaning beyond exact keyword overlap

$$\text{Score}_{\text{Dense}}(Q, D) = \frac{\vec{V_Q} \cdot \vec{V_D}}{|\vec{V_Q}| \cdot |\vec{V_D}|}$$

The score range is $[-1, 1]$, representing the semantic proximity in the vector space.

3. Late Interaction Scoring (ColBERT/ColQwen)

Unlike dense retrievers that compress a document into a single vector, the Late Interaction architecture (used in ColBERTv2) encodes queries and documents into bags of embedding vectors. The relevance score is the sum of the maximum similarity (MaxSim) for each query token against all document tokens

$$\text{Score}_{\text{MaxSim}}(Q, D) = \sum_{q \in \vec{Q}} \max_{d \in \vec{D}} (\vec{q} \cdot \vec{d})$$

This allows for fine-grained matching, ensuring that specific details in a document are not lost during vector compression.

Evaluation Metrics To quantitatively assess the pipelines, we adhere to two standard information retrieval metrics:

1. **Recall@k:** Measures the percentage of relevant documents successfully retrieved within the top k results.

$$\text{Recall@k} = \frac{|\{\text{Relevant Docs}\} \cap \{\text{Retrieved Top-k}\}|}{|\{\text{Total Relevant Docs}\}|}$$

2. **nDCG@k (Normalized Discounted Cumulative Gain):** This metric evaluates the *ranking quality*. It rewards algorithms that place relevant documents at the top of the list.

$$\text{nDCG@k} = \frac{DCG@k}{IDCG@k} \quad \text{where} \quad DCG@k = \sum_{i=1}^{k} \frac{2^{rel_i} - 1}{\log_2(i + 1)}$$

4.1.2 Pipeline Analysis and Results

We conducted a comprehensive evaluation of Lexical, Dense, Hybrid, and Reranking pipelines. The aggregate results are presented in Table 4.1.

Table 4.1: Comparative performance of Retrieval and Reranking pipelines on the MS MARCO subset (50k docs, 556 queries).

Pipeline Strategy	Model / Configuration	Latency	nDCG@3	Recall@3
Baseline Retrievers (No Rerank)				
Sparse	BM25 (rank-bm25)	3m	0.2821	0.3085
Dense	MiniLM-L6 (Raw)	∼1s	0.8658	0.9125
Dense	**BGE-Small-en-v1.5**	∼1s	**0.8855**	**0.9257**
Hybrid Retrievers (Reciprocal Rank Fusion)				
Hybrid	BM25 + MiniLM ($k = 60$)	∼1s	0.5786	0.6885
Hybrid	BM25 + BGE ($k = 60$)	∼1s	0.5830	0.6787
Reranking Pipelines (Retrieve + Cross-Encoder)				
Retrieve + Rerank	BM25 + BGE-Large	1m 43s	0.4469	0.4592
Retrieve + Rerank	MiniLM + BGE-Large	1m 43s	0.9196	0.9526
Retrieve + Rerank	**BGE-Small + BGE-Large**	1m 43s	**0.9248**	**0.9598**
Late Interaction				
Late Interaction	**ColBERTv2 (RAGatouille)**	6.5s	**0.9638**	**0.9412**

Question 1: Is Hybrid Retrieval superior to single-mode retrieval? Our experiments demonstrate that Hybrid Retrieval (combining BM25 and Dense vectors via Reciprocal Rank Fusion) offers a significant improvement over purely lexical methods, providing a balanced approach to robustness.

- **Limitations of BM25:** The raw BM25 baseline yielded a low nDCG@3 of 0.2821. This confirms that the MS MARCO dataset relies heavily on semantic understanding rather than exact keyword matches, causing BM25 to fail when vocabulary mismatches occur.
- **The Hybrid Improvement:** By fusing BM25 with Dense retrieval (Hybrid BGE), the nDCG@3 score nearly doubled to 0.5830.
- **Significance:** While pure dense models performed exceptionally well on this specific dataset (0.8855), relying solely on dense retrieval in production carries the risk of "semantic hallucination" (retrieving conceptually similar but factually incorrect documents). Hybrid retrieval mitigates this by anchoring the semantic search with the keyword precision of BM25. It provides a "safety net" that ensures documents containing critical entity names are not discarded, making it a more robust strategy for real-world applications where query intent varies.

Question 2: Does the addition of a Reranker significantly improve ranking quality? The results strongly support the hypothesis that a two-stage "Retrieve-then-Rerank" pipeline outperforms single-stage retrieval.

- **Architectural Advantage:** We employed Cross-Encoders (specifically **BAAI/bge-reranker-large**) as the second stage. Unlike bi-encoders, cross-encoders process the query and document simultaneously in the Transformer, capturing deep interaction signals.
- **Performance Gain:** Applying the BGE-Large reranker to the BGE-Small retriever candidates improved nDCG@3 from 0.8855 to **0.9248** and Recall@3 from 0.9257 to **0.9598**.
- **Model Comparison:** We observed that larger rerankers yield better quality. The **bge-reranker-large** consistently outperformed the smaller **ms-marco-MiniLM-L-12** (e.g., 0.9248 vs 0.9107). However, this comes with a trade-off: the large reranker is approximately 6× slower ($1m43s$ vs $17s$).
- **Conclusion:** For applications prioritizing accuracy, the combination of a high-performance dense retriever (BGE-Small) and a large cross-encoder (BGE-Large) is the optimal two-stage configuration.

Question 3: Is ColBERT (Late Interaction) the most effective approach? Our data indicates that ColBERTv2 provides the highest retrieval quality, surpassing even the best Retrieve-then-Rerank pipelines.

- **Superior Metrics:** ColBERTv2 achieved the highest **nDCG@3 of 0.9638**, outperforming the best reranking pipeline (0.9248).
- **Solving the Information Bottleneck:** Standard two-stage pipelines suffer from a bottleneck: if the first-stage retriever (e.g., BGE-Small) fails to find the relevant document in the top 30 candidates, the Reranker never sees it. ColBERT avoids this by using a "Late Interaction" mechanism. It calculates the MaxSim between query tokens and document tokens directly.
- **Granularity:** As shown in Equation (3.3), ColBERT matches at the token level. This allows it to identify a relevant document even if only a small paragraph within it matches the query a nuance often lost when compressing a whole document into a single vector (Dense Retrieval).
- **Operational Note:** It is worth noting that while ColBERT's retrieval time is efficient ($6.5s$ on GPU), the indexing process is resource-intensive. In our environment, indexing required $\approx$ 35 minutes on CPU[2] compared to mere seconds/minutes for standard dense models. Despite this setup cost, ColBERT represents the state-of-the-art for retrieval accuracy on this dataset.

[2]Due to FAISS-GPU limitations on the Colab instance, indexing fell back to CPU.

4.2 Justification for Retrieval-Augmented Generation

While Large Language Models (LLMs) possess remarkable generative capabilities, they suffer from inherent limitations such as knowledge cutoffs, hallucinations, and a lack of access to proprietary data. To address these issues, several methodologies exist. This section analyzes alternative approaches - Fine-Tuning, Semantic Search, Prompt Engineering, and Domain-Specific Pre-training - and justifies the selection of RAG as the superior architectural choice for this project.

4.2.1 Comparative Analysis of Alternatives

RAG vs. Fine-Tuning Fine-tuning involves retraining a pre-trained LLM on a specific dataset to adjust its internal parameters (weights). While this method tailors the model to a specific domain style, it was rejected for the following reasons:

- **Static Knowledge Base:** Fine-tuned models immediately become outdated upon training completion. Updating the knowledge requires a computationally expensive retraining process. In contrast, RAG allows for *dynamic updates* - simply adding a document to the database makes it immediately retrievable without modifying the model weights.
- **Hallucination Risk:** Fine-tuning increases the probability of the model adopting a specific tone, but it does not guarantee factual accuracy. The model can still hallucinate information effectively. RAG mitigates this by grounding the generation in retrieved evidence.
- **Explainability:** A fine-tuned model outputs an answer based on "black box" internal weights. RAG provides citation and source attribution (retrieved chunks), which is critical for verifying accuracy in our application.

RAG vs. Prompt Engineering (Context Stuffing) Prompt engineering involves optimizing the textual input to guide the LLM's output, often by manually pasting context. While simple, it is insufficient for our scale:

- **Context Window Limitations:** Although context windows are growing (e.g., 128k tokens), it is infeasible to pass an entire corpus (e.g., 50,000 documents) into a single prompt. RAG acts as a filter, selecting only the most relevant snippets to fit within the model's constraints.
- **Performance Degradation:** Studies show that as the context length increases, the model's ability to reason over the data ("Lost in the Middle" phenomenon) decreases. RAG ensures the model focuses only on highly relevant, condensed information.

RAG vs. Semantic Search (Traditional Information Retrieval) Semantic search utilizes vector databases to retrieve documents based on meaning rather than keywords. However, it functions solely as a retrieval engine, not a generation engine.

- **Synthesis vs. Retrieval:** Semantic search returns a list of document links or snippets. The user is then forced to read and synthesize the answer manually. RAG automates this cognitive load by employing the LLM to read the retrieved documents and generate a coherent, natural language answer.
- **Completeness:** A user's query might require synthesizing information from three different documents. Semantic search provides the three documents; RAG provides the unified answer derived from them.

RAG vs. Domain-Specific Pre-training Pre-training involves training a model from scratch or continuing pre-training on a massive domain-specific corpus.

- **Computational Cost:** This approach is prohibitively expensive, requiring massive GPU clusters and weeks of training time.
- **Flexibility:** Like fine-tuning, a pre-trained model fails in dynamic environments where data changes daily. RAG decouples the *reasoning engine* (the LLM) from the *knowledge base* (the Vector DB), allowing us to swap or update the knowledge base instantly without touching the neural network.

4.2.2 Summary of Trade-offs

The following table summarizes the key architectural decisions that led to the adoption of RAG.

Table 4.2: Comparison of RAG against alternative LLM enhancement strategies.

Feature	RAG (Ours)	Fine-Tuning	Prompt Engineering	Semantic Search
Knowledge Update	*Real-time (Dynamic)*	Slow (Retraining)	Manual	Real-time
Hallucination Risk	Low (Grounded)	High	Medium	N/A (No Generation)
Context Capacity	Infinite (via Retrieval)	Limited by Training	Limited by Window	Infinite
Computational Cost	Low (Inference only)	High (Training)	Very Low	Low
Answer Synthesis	**Yes**	Yes	Yes	**No**

Conclusion: RAG is chosen because it offers the optimal balance of accuracy, cost-efficiency, and dynamic adaptability. It leverages the reasoning power of pre-trained LLMs while overcoming their memory limitations through external retrieval, making it the most suitable architecture for a document-based Question Answering system.

4.3 Technical Requirements

4.3.1 Hardware Requirements

- **GPU**: NVIDIA GPU with at least 4GB of VRAM, but when using 8GB VRAM, we recommend (RTX 4060, RTX 4070, or equivalent) for efficient embedding generation and inference. For larger models or batch processing, 16GB+ VRAM is recommended.
- **CPU**: Multi-core processor (8+ cores) to support parallel processing of documents and concurrent API requests.
- **Memory (RAM)**: Minimum 16GB of system RAM for running the entire pipeline, including model caching and document processing.
- **Storage**: At least 100GB of available disk space for storing document artifacts, indices, and model weights.

4.3.2 Software Dependencies

- **Python**: Version 3.10 or higher.
- **Core Libraries**:
  - FastAPI for the REST API framework.
  - PyTorch for deep learning model inference.
  - FAISS for efficient vector similarity search.
  - BM25Okapi for sparse text retrieval.
  - Pillow and OpenCV for image processing.
  - PyMuPDF (fitz) for PDF document handling.
- **Model Weights**:
  - `all-MiniLM-L6-v2` embedding model (384-dimensional, ∼30MB).
  - `bge-large` cross-encoder for reranking (∼500MB).
  - ColQwen2 vision-language model for visual retrieval (∼5GB).
  - LLM backbone (e.g., GPT-4 API or local model such as Mixtral, Qwen, or Llama 2) for generation.

4.3.3 System Architecture Constraints

- **Data Privacy**: All document artifacts are stored locally on the file system. No external cloud services are required for document storage, ensuring complete data privacy and compliance with institutional policies.

- **Concurrency**: The system is designed to handle multiple concurrent API requests using FastAPI's asynchronous architecture, supporting up to N simultaneous users (where N is bounded by available memory and GPU scheduling).
- **Scalability**: The modular pipeline design allows for independent scaling of retrieval and generation components. Additional indexing nodes can be added for large document corpora without modifying the core system.
- **Portability**: The entire system can be containerized using Docker, enabling seamless deployment across different environments (development, staging, production).

4.4 Main Architecture

This section presents the comprehensive architecture of the Multimodal Retrieval-Augmented Generation (mRAG) system. We structure the discussion in three layers: first, the pipeline architecture defining data transformation workflows; second, the high-level system architecture illustrating component interactions; and third, the detailed implementation of individual components.

4.4.1 Pipeline Architecture

The system implements a sophisticated dual-pipeline architecture designed to handle heterogeneous document formats and produce RAG-ready outputs for both text-based and image-based retrieval. The architecture is divided into two primary pipelines: the Data Processing Pipeline and the Embedding-Indexing Pipeline.

4.4.1.1 Data Processing Pipeline

The Data Processing Pipeline transforms raw input files through three sequential stages, each optimized for specific document types and processing requirements. As illustrated in Figure **??**, this pipeline ensures that diverse formats are normalized into standardized representations suitable for downstream retrieval tasks.

Stage 1: Normalization

The normalization stage converts heterogeneous input formats into two canonical forms: PDF (for visual representation) and Markdown (for semantic text content). This dual-output strategy is essential for supporting multimodal retrieval.

Input Format	Processing Method	Output (PDF)	Output (Markdown)
DOCX	LibreOffice conversion	(layout preserved)	(extracted text)
PPTX	LibreOffice conversion	(slides as pages)	(slide text)
HTML	BeautifulSoup parsing	(rendered)	(cleaned HTML)
Images (JPG, PNG)	img2pdf conversion	Yes (1 image = 1 page)	Yes (via OCR in Stage 3)
Excel/CSV	Pandas processing	No (tables only)	Yes (Markdown tables)
PDF	Copy as-is	Yes (original)	— (processed in Stage 3)
Markdown	Copy as-is	—	Yes (original)

Table 4.3: Stage 1 normalization mapping for different input formats.

Stage 2: Media Processing

For multimedia inputs (video and audio), Stage 2 extracts temporal information and converts it to text format. This stage is critical for enabling search over lecture recordings, presentations, and audiovisual content.

Input Format	Extraction Tool	Intermediate Output	Final Output
Video (MP4, AVI)	MoviePy	Audio (WAV)	Markdown transcript
	OpenCV (optional)	Video frames (JPG)	—
Audio (WAV, MP3)	OpenAI Whisper	—	Markdown transcript
			Timestamped subtitles (SRT/VTT)

Table 4.4: Stage 2 media processing transformation mapping.

The transcription process utilizes **OpenAI Whisper**, a state-of-the-art Automatic Speech Recognition (ASR) system. Whisper processes audio waveforms and generates timestamped transcripts, which are immediately serialized to Markdown format for consistency with other document types.

Stage 3: Document Processing

The final stage employs **Docling**, a specialized document understanding framework, to perform deep structural analysis. Unlike simple text extraction, Docling reconstructs document hierarchy, extracts tables with structure preservation, and isolates images for visual indexing.

Input	Docling Processing	Markdown	Images	Tables
DOCX	Layout + structure analysis	Yes	Yes (extracted)	Yes (CSV + MD)
PPTX	Slide decomposition	Yes	Yes (per slide)	Yes
PDF (digital)	Text layer extraction	Yes	Yes (embedded)	Yes
PDF (scanned)	OCR + VLM	Yes	Yes	Yes
Images	OCR + VLM description	Yes (captions)	Yes (original)	—
HTML	DOM parsing	Yes	Yes (embedded)	Yes

Table 4.5: Stage 3 document processing with Docling outputs.

Docling integrates multiple vision backends (OCR engines: RapidOCR, Tesseract, EasyOCR; Vision-Language Models for semantic understanding) to ensure high-fidelity content extraction from both born-digital and scanned documents.

4.4.1.2 Embedding-Indexing Pipeline

Following data processing, the system employs a dual-pathway embedding architecture to enable multimodal retrieval. As illustrated in Figure **??**, this architecture allows simultaneous retrieval of granular text segments and holistic document pages.

Text Embedding Pathway

The text pathway processes structured Markdown files generated from Stage 3. The workflow consists of:

1. **Chunking**: Markdown content is segmented using `RecursiveCharacterTextSplitter` with a chunk size of 1024 characters and 200-character overlap. This overlap ensures semantic continuity across chunk boundaries.
2. **Embedding**: Each chunk is encoded into a 384-dimensional vector using the `all-MiniLM-L6-v2` model. This bi-encoder architecture enables efficient similarity search via cosine distance.
3. **Indexing**: The system maintains three parallel indices:
   - **BM25 Index**: Inverted index for keyword-based sparse retrieval.
   - **Dense Index**: FAISS IndexFlatIP for semantic vector search.
   - **Hybrid Index**: Fusion of BM25 and dense scores via Reciprocal Rank Fusion (RRF).
4. **Metadata Storage**: Each indexed chunk includes metadata: `doc_id`, `chunk_idx`, `total_chunks`, and `source` file path.

Stage	Input	Process	Output	Storage	Index Type
Chunking	Markdown	RecursiveSplitter	Text chunks	—	—
Embedding	Text chunks	MiniLM-L6-v2	384-dim vectors	—	—
BM25 Index	Text chunks	TF-IDF scoring	Inverted index	.pkl file	Sparse
Dense Index	Vector embeddings	FAISS IndexFlatIP	Vector store	.bin file	Dense
Hybrid Index	BM25 + Dense	RRF fusion	Unified scores	JSON	Hybrid

Table 4.6: Text embedding and indexing pipeline stages.

Visual Embedding Pathway

The visual pathway processes PDF files to enable image-based retrieval. Instead of relying on OCR, this pathway treats each page as an image, preserving layout, charts, and diagrams.

1. **Rasterization**: PDF pages are converted to high-resolution JPEG images (150 DPI) using `pdf2image`. The system implements a "1 Page = 1 Image" strategy to maintain document structure.

2. **Visual Embedding**: Images are processed by **ColQwen2**, a Vision-Language Model based on the Late Interaction paradigm. Unlike traditional dense embeddings, ColQwen generates multi-vector representations, allowing fine-grained matching between query tokens and visual patches.

3. **Visual Indexing**: Visual embeddings are stored in a specialized ColQwen index with metadata: `source`, `source_path`, `page` number, and `total_pages`.

4. **Just-in-Time Rendering**: During retrieval, specific retrieved pages are rendered on-demand and passed directly to the Multimodal LLM, enabling the model to "see" diagrams and charts contextually.

Stage	Input	Process	Output	Storage	Index Type
Rasterization	PDF pages	pdf2image (150 DPI)	JPEG images	Image files	—
Visual Embedding	JPEG images	ColQwen2 (ViT)	Multi-vector tensors	Tensor store	Late Interaction
Visual Index	Visual vectors	ColQwen indexing	Searchable vectors	.bin file	Visual
Metadata	Per-page info	JSON serialization	Page metadata	.json file	Lookup table

Table 4.7: Visual embedding and indexing pipeline stages.

Pipeline Output Summary

The complete pipeline produces the following RAG-ready artifacts:

- **Text-Based Retrieval**: Markdown files with three indices (BM25, Dense, Hybrid) enabling keyword and semantic search over chunked content.
- **Image-Based Retrieval**: Rasterized PDF pages with ColQwen visual embeddings enabling layout-aware and diagram-sensitive search.
- **Structured Data**: Extracted tables (CSV + Markdown), isolated images, and comprehensive metadata (JSON) for provenance tracking.
- **Multimedia Transcripts**: Time-stamped text transcriptions from video/audio in multiple formats (TXT, JSON, SRT, VTT).

4.4.2 High-Level System Architecture

The overall system architecture follows a modular design with clear separation of concerns across three primary subsystems: Input Processing, Dual-Stream Indexing, and Generative Synthesis. This section describes the high-level interactions between components and the end-to-end data flow.

System Overview

As illustrated in Figure 4.2, the system implements a Multimodal Retrieval-Augmented Generation (RAG) architecture designed to synthesize answers from diverse unstructured data sources. The workflow integrates three distinct processing layers that operate sequentially during indexing and in parallel during query execution.

Data Flow Architecture

The system's operation can be divided into two phases: the offline indexing phase and the online retrieval phase.

Phase 1: Offline Indexing (Document Ingestion)

During the indexing phase, raw documents are transformed into searchable artifacts through the multi-stage pipeline described in the previous subsection. Figure **??** illustrates the normalization process where heterogeneous inputs (DOCX, PPTX, HTML, Images, Video, Audio) are converted into standardized PDF and Markdown representations.

The normalization layer employs specialized processors for each document type:

- **Office Documents**: LibreOffice converts DOCX/PPTX to layout-preserving PDFs.
- **Multimedia**: MoviePy extracts audio from video; Whisper transcribes audio to text.
- **Complex Documents**: Docling analyzes PDF/HTML structure, extracts tables, and isolates images.

Following normalization, the dual-stream indexing architecture processes the outputs in parallel. As shown in Figure **??**, the system maintains two independent retrieval pathways:

1. **Text Stream**: Markdown content → Chunking → Embedding (MiniLM-L6-v2) → Triple Indexing (BM25 + Dense + Hybrid).
2. **Visual Stream**: PDF pages → Rasterization → Visual Embedding (ColQwen2) → Visual Index.

This dual-pathway design ensures that the system can retrieve both semantically relevant text passages and visually similar document pages, addressing the limitations of text-only RAG systems that discard layout and diagram information.

Phase 2: Online Retrieval (Query Processing)

During query execution, the system operates in a retrieve-then-generate paradigm. The user submits a natural language query via a chat interface, which triggers parallel retrieval across both streams:

1. **Text Retrieval**: The query is processed by the Hybrid Search engine, which combines BM25 (keyword matching) and Dense Retrieval (semantic similarity). A Reciprocal Rank Fusion (RRF) algorithm merges the results. Optionally, a Cross-Encoder reranker (BGE-Large) refines the top-K candidates for improved precision.

2. **Visual Retrieval**: Simultaneously, the query is encoded by ColQwen's query encoder and matched against the visual index using Late Interaction scoring (MaxSim). This retrieves pages containing relevant diagrams, charts, or tables that may not be adequately represented in extracted text.

3. **Context Aggregation**: The top-K text chunks and top-K visual pages are aggregated into a unified context window. For visual results, the system performs just-in-time rendering of the retrieved PDF pages.

4. **Answer Generation**: The aggregated context (text + rendered images) is passed to a Multimodal Large Language Model (supporting OpenAI GPT-4o, Gemini, Anthropic Claude, or locally hosted models via Ollama). The LLM synthesizes a natural language answer with citations referencing the source documents and page numbers.

Architectural Components

The high-level architecture is composed of four logical layers:

1. **Interface Layer**: FastAPI-based REST API providing endpoints for document upload (`/api/upload`), processing (`/api/process`), and querying (`/api/query`). Asynchronous I/O and background task execution prevent blocking during heavy operations.

2. **Orchestration Layer**: The `UnifiedRAGPipeline` class acts as the central controller, coordinating document processing, retrieval, and generation. Dynamic YAML configuration allows switching between Text-Only, Image-Only, or Multimodal modes without code changes.

3. **Storage Layer**: File-system-based artifact storage organized into three directories:

   - `processing/`: Intermediate ETL artifacts from normalization, media processing, and Docling outputs.
   - `retrieval/`: Text indices (BM25 inverted index, FAISS vector store, metadata catalog).
   - `image_retrieval/`: Visual embeddings and ColQwen tensor store.

4. **Retrieval & Generation Layer**: Implements the hybrid text retrieval engine (BM25 + Dense + Reranker) and the visual retrieval engine (ColQwen Late Interaction). The generation module interfaces with multiple LLM providers via a unified API abstraction.

Key Architectural Decisions

Several design choices distinguish this architecture from traditional RAG systems:

- **Local-First Storage**: Unlike cloud-native solutions (Pinecone, Milvus), the system stores all artifacts locally, ensuring data privacy and compliance with institutional policies. This file-system-based approach enables portability and eliminates external dependencies.
- **Dual-Mode Retrieval**: By maintaining separate text and visual indices, the system overcomes the fundamental limitation of text-only RAG, which fails when critical information is encoded in charts, diagrams, or tables that OCR cannot accurately represent.
- **Late Interaction for Visual Search**: Traditional visual retrieval compresses entire pages into single vectors, losing fine-grained details. ColQwen's multi-vector representation enables token-level matching between queries and visual patches, preserving spatial and semantic nuances.
- **Hybrid Retrieval with Reranking**: The text pathway combines sparse (BM25) and dense (MiniLM) retrieval via RRF, then applies cross-encoder reranking. This two-stage architecture balances recall (via broad retrieval) and precision (via deep interaction modeling).

4.4.3 Detailed Component Architecture of Multimodal RAG System

Having established the pipeline workflows and high-level system design, this subsection provides comprehensive descriptions of all components that constitute the Multimodal Retrieval-Augmented Generation system. We organize the discussion across three operational layers: Input Processing, Dual-Stream Indexing, and Generative Synthesis.

4.4.3.1 Input Processing Layer

The first stage of the pipeline is responsible for normalizing diverse raw file formats into a standardized set of artifacts. This layer utilizes specific open-source tools for each data modality, as illustrated in Figure **??**.

Document Parsing with Docling

To address the challenge of ingesting complex, semi-structured data, the pipeline integrates **Docling**, a state-of-the-art document parsing framework. Docling serves as the central transformation engine, converting diverse input formats into structured, semantic representations ideal for RAG applications.

Docling represents a paradigm shift from traditional "text extraction" to "document understanding." While standard libraries (e.g., PyPDF2) merely scrape text strings, Docling analyzes the visual and structural hierarchy of a document. It reconstructs the logical flow of information, distinguishing between headers, paragraphs, footnotes, and tabular data, ensuring that the semantic context is preserved during the serialization to Markdown.

One of Docling's primary architectural advantages is its ability to unify heterogenous inputs into a single processing stream. As shown in the system architecture, Docling handles a wide array of formats natively:

- **PDF Documents**: Handles both native digital PDFs (with text layers) and scanned image-based PDFs.
- **Office Suites**: Directly parses Microsoft Excel (.XLS/XLSX) spreadsheets to extract tabular data, as well as PowerPoint (.PPTX) and Word (.DOCX) files.
- **Web & Code**: Ingests HTML and raw text formats.
- **Images**: Processes standalone diagrams or document screenshots.

Docling employs a hybrid approach combining computer vision and natural language processing to achieve high-fidelity extraction.

Before text extraction begins, Docling performs a segmentation pass. It identifies physical page elements such as columns, text blocks, and floating figures. This ensures that multi-column PDFs are read in logical reading order (left-column down, then right-column) rather than strictly left-to-right, which often garbles sentences in standard extractors.

For scanned documents or image-heavy PDFs, Docling integrates specialized vision backends:

- **OCR**: For standard scanned text, high-performance OCR engines recover the textual layer.
- **VLM**: For complex layouts, Docling leverages Vision-Language Models. These models "look" at the page to understand semantic structures that regex or rule-based parsers miss, such as distinguishing a figure caption from the main body text.

A critical feature highlighted in the architecture is the explicit extraction of **Tables**. Docling reconstructs table borders and cell relationships, converting them into structured Markdown tables or HTML representations. This allows the downstream embedding model to interpret row-column relationships correctly, which is essential for answering quantitative queries.

The result of the Docling process is a set of clean, normalized artifacts ready for indexing:

1. **Structured Markdown**: The primary text payload, with headers marked ('#', '##') to denote hierarchy.
2. **Isolated Images**: Figures and diagrams extracted for the visual indexing stream.
3. **Table Objects**: Structured data specifically preserved for precise retrieval.

Audio Transcription with Whisper

To extend the system's retrieval capabilities beyond static documents, the pipeline integrates **OpenAI Whisper**, a robust Automatic Speech Recognition (ASR) system. This component allows the RAG engine to *listen* to multimedia content, converting spoken words into structured, searchable text.

The transcription process follows a linear transformation pipeline designed to normalize audio into the system's standard text format:

1. **Input Ingestion**: The subsystem accepts raw audio inputs, specifically optimized for **.WAV** formats as depicted in the workflow diagram.
2. **Speech Processing**: The **Whisper** model processes the audio waveform, performing language identification and timestamped transcription.
3. **Text Serialization**: The raw transcript is first captured as plain text. Crucially, the system immediately serializes this text into **Markdown** format. This standardization ensures that audio transcripts are treated identically to PDF or DOCX content during the downstream chunking and embedding phases.

Whisper acts as a critical bridge for video content within the broader input processing layer. For video inputs (e.g., MP4), the system utilizes **MoviePy** to strip the audio track from the visual container. This extracted audio is then passed to Whisper, ensuring that lecture recordings or meeting videos are fully indexed and retrievable via text search queries.

Office Document Normalization

Legacy office formats (DOC, DOCX, PPT, PPTX, XLS) are converted using **LibreOffice** in headless mode. These documents are normalized into standard PDF format to preserve layout, fonts, and slide structures for the visual retrieval engine.

Output Artifacts

The Input Processing Layer produces a unified set of artifacts stored in the `rag_ready` directory:

1. **PDF**: Used for visual embedding (ColQwen) and user preview.
2. **Markdown**: Structured text for semantic chunking and text retrieval.
3. **Extracted Metadata**: JSON sidecars containing file provenance, page counts, and processing timestamps.

4.4.3.2 Dual-Stream Indexing Layer

Following normalization, the data is split into two specialized indexing streams to handle the unique requirements of text and images, as illustrated in Figure 4.1.

Textual Indexing Stream

The Markdown output undergoes a semantic vectorization process:

- **Chunking Strategy**: The system employs a `RecursiveCharacterTextSplitter` configured with a chunk size of **1024** characters and an overlap of **200** characters to maintain context across boundaries.
- **Embedding Model**: Chunks are encoded using `all-MiniLM-L6-v2`, producing 384-dimensional dense vectors.
- **Triple Index**: Vectors are stored in three parallel indices (BM25 for keyword search, FAISS for dense retrieval, Hybrid for fused results).
- **Metadata**: Each chunk is tagged with `doc_id`, `chunk_idx`, `total_chunks`, and `source`.

Visual Indexing Stream

The PDF output undergoes a visual encoding process:

- **Image Strategy**: The system applies a "1 Page = 1 Image" strategy, converting every PDF page into a high-resolution JPEG.
- **Visual Embedding**: Page images are encoded using **ColQwen2** (based on Qwen2-VL), producing multi-vector representations that preserve layout semantics.
- **Visual Index**: Multi-vector tensors are stored in the ColQwen Index, enabling late-interaction retrieval.
- **Metadata**: Each entry is tagged with `source`, `source_path`, `page` number, and `total_pages`.

4.4.3.3 Advanced Visual Retrieval with ColQwen

The visual indexing stream deserves special attention due to its innovative approach to document retrieval. To overcome the limitations of traditional OCR-based retrieval which often discards the semantic value of document layout, charts, and figures the pipeline integrates **ColQwen**. This state-of-the-art Visual Language Model (VLM) represents an evolution in retrieval architecture, transitioning from text-only processing to holistic visual understanding.

Theoretical Foundation: From ColBERT to ColPali

The system's visual retrieval engine is built upon the "Late Interaction" paradigm, which contrasts sharply with traditional dense retrieval methods.

ColBERT (Contextualized Late Interaction over BERT)

Traditional embedding models (like SBERT) compress an entire document into a single vector. While efficient, this "bottleneck" loses fine-grained details. ColBERT introduced the concept of **Late Interaction**, where documents are encoded into a *bag of embeddings* (one vector per token). Matching is performed by comparing every query token vector against every document token vector (MaxSim), preserving distinct semantic features.

ColPali (ColBERT for Paligemma)

ColPali extends this mechanism to the visual domain. Instead of tokenizing text, it utilizes a Vision Transformer (ViT) to patchify a document image into visual tokens. Each patch is projected into a multi-vector space, allowing the model to perform "Late Interaction" retrieval on visual features directly, effectively treating an image patch exactly like a textual word.

The pipeline specifically utilizes **ColQwen2** (based on Qwen2-VL), which optimizes the ColPali architecture for document-heavy tasks.

Visual Encoding Pipeline

The process follows a strict transformation workflow:

1. **Rasterization**: The system adheres to a "1 Page = 1 Image" strategy. Input PDFs are rendered into high-fidelity JPEG images, preserving fonts, layout, and diagrams.
2. **Patch Embedding**: The `vidore/colqwen2-v1.0` (or `v2.5`) model processes these images. Unlike standard CLIP embeddings which create one vector per image, ColQwen generates a distinct embedding for every 16×16 pixel patch of the page.
3. **Tensor Storage**: The resulting multi-vector representation is stored in the ColQwen Index. This allows the system to retrieve a specific page because a diagram in the top-right corner semantically matches the user's query, even if no text describes it.

Index Metadata Schema

To facilitate precise retrieval and citation, every visual embedding is tightly coupled with a metadata object, as defined in the system architecture:

```
{
  "source": "filename.pdf",        # Origin document
  "source_path": "/path/to/file",  # System path for retrieval
  "page": int,                     # Specific page number (1-indexed)
  "total_pages": int               # Contextual length of document
}
```

Advantages over OCR

By adopting ColQwen, the pipeline achieves capabilities that standard text-RAG cannot match:

- **Visual Semantics**: It can retrieve a page based on the visual content of a bar chart or the structure of a molecule diagram.
- **Layout Awareness**: It understands that text in a "Header" is more important than text in a "Footer" based on visual hierarchy.
- **Multilingual Agnosticism**: It processes visual glyphs rather than relying on language-specific OCR dictionaries.

4.4.3.4 Generative Synthesis Layer

The final operational layer orchestrates user interaction and answer generation, as illustrated in Figure 4.2. This layer implements the retrieve-then-generate paradigm central to RAG architectures.

Query Execution Workflow

When a user submits a natural language query via the chatbot interface, the system executes the following pipeline:

1. **Parallel Retrieval**: The query is simultaneously processed by the Text Retrieval Engine (Hybrid Search combining BM25 + Dense retrieval) and the Visual Retrieval Engine (ColQwen late-interaction search).
2. **Context Aggregation**: Top-K text chunks and Top-K image pages are collected. An optional Cross-Encoder Reranking step (using BGE-Large) refines the text results to improve precision.
3. **Just-in-Time Rendering**: Retrieved PDF pages are rendered on-demand into high-resolution images, preserving charts, diagrams, and layout information for visual understanding.
4. **LLM Generation**: The aggregated context (text chunks + rendered page images) is passed to a Multimodal Large Language Model. The system is provider-agnostic, supporting OpenAI GPT-4o, Google Gemini, Anthropic Claude, and local models via Ollama (Llama, Mistral, Qwen).
5. **Answer Synthesis with Citations**: The LLM generates a natural language answer grounded in the retrieved context. The response includes precise citations referencing source documents and page numbers, enabling full traceability.

System Integration

The complete Multimodal RAG architecture integrates all three layers seamlessly. The system provides a FastAPI-based REST interface with asynchronous I/O support, exposing endpoints for document upload (`/api/upload`), processing (`/api/process`), querying (`/api/query`), and status monitoring (`/api/status`).

Office Document Formats

Legacy office formats (DOC, DOCX, PPT, PPTX, XLS) are often difficult to parse directly. The system utilizes **LibreOffice** in headless mode (server-side). These documents are converted into standard PDF format. This ensures that layout, fonts, and slide structures are preserved visually for the image retrieval engine.

Multimedia (Audio and Video)

Video (MP4, AVI) and Audio (WAV, MP3) files contain rich semantic information that must be serialized into text. **MoviePy** is employed to separate audio tracks from video containers. The **OpenAI Whisper** model (Automatic Speech Recognition) processes the audio stream to generate high-fidelity transcripts. These transcripts are saved as Markdown text files, time-stamped to allow for future temporal retrieval.

Structural Parsing

For complex documents like scientific PDFs, HTML pages, or Table-heavy reports, simple text extraction is insufficient. The pipeline integrates **Docling**, a specialized document parsing library. Docling analyzes the document structure to distinguish between headers, paragraphs, and tables. It outputs structured **Markdown** that preserves table boundaries (CSV-style) and document hierarchy. This allows the text chunker to respect semantic boundaries rather than cutting sentences mid-table.

Output Artifacts

Every input file results in a unified set of artifacts in the rag ready directory:

1. **PDF**: Used for visual embedding (ColQwen) and user preview.
2. **Structured Markdown**: Used for text embedding (BM25/Dense) and LLM context injection.
3. **Extracted Metadata**: JSON sidecars containing file provenance, page counts, and processing timestamps.

Dual-Stream Indexing Layer

Following normalization, the data is split into two specialized indexing streams to handle the unique requirements of text and images.

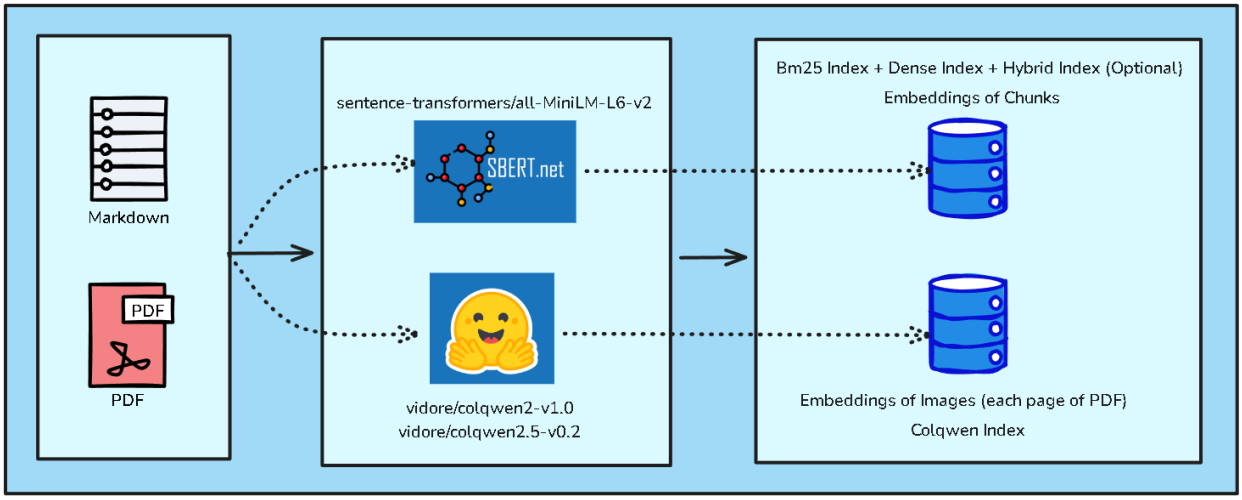

Figure 4.1: Dual-Stream Indexing Layer.

Textual Indexing Stream

The Markdown output undergoes a semantic vectorization process:

- **Chunking Strategy**: The system employs a `RecursiveCharacterTextSplitter` configured with a chunk size of **1024** characters and an overlap of **200** characters to maintain context across boundaries.
- **Embedding Model**: Chunks are encoded using sentence-transformers/all-MiniLM-L6-v2 via the SBERT framework.
- **Storage**: Vectors are stored in a Hybrid Index supporting both BM25 (sparse) and Dense retrieval.
- **Metadata**: Each chunk is tagged with `doc_id`, `chunk_idx`, `total_chunks`, and `source`.

Visual Indexing Stream

The PDF output undergoes a visual encoding process:

- **Image Strategy**: The system applies a "1 Page = 1 Image" strategy, converting every PDF page into a high-resolution JPEG.
- **Vision Model**: These images are processed by **ColQwen** (specifically vidore/colqwen2-v1.0 or v2.5). This model generates visual embeddings that capture layout and diagrammatic information.
- **Storage**: Embeddings are stored in a specialized ColQwen Index.
- **Metadata**: Each entry is tagged with `source`, `source_path`, `page` number, and `total_pages`.

Generative Synthesis Layer

The final layer orchestrates the user interaction and answer generation.

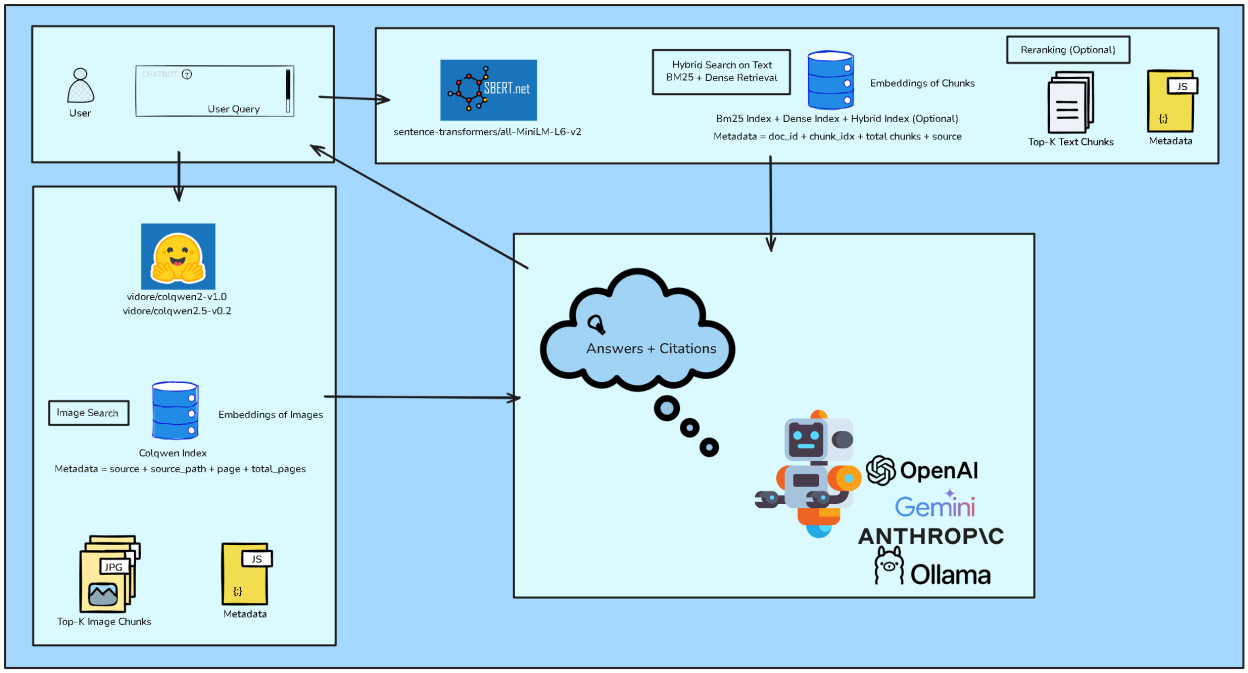

Figure 4.2: Overall System Architecture.

- **Hybrid Retrieval**: A user query triggers parallel searches against the Text Index (BM25 + Dense) and the Visual Index (Image Search).
- **Reranking**: The system supports an optional Reranking step to refine the retrieved Top-K Text Chunks before context injection.
- **LLM Integration**: The aggregated context (Text Chunks + Image Pages) is sent to a Large Language Model. The system is provider-agnostic, supporting OpenAI, Gemini, Anthropic, and Ollama.
- **Output**: The final output includes the generated answer with precise citations linking back to the specific source documents.

Chapter 5

Initial Implementation

Phase 1 Scope and Objectives: This chapter presents the initial implementation phase of the Multimodal Retrieval-Augmented Generation (mRAG) system. In Phase 1, the primary focus is on **establishing and validating the core system architecture**, including the data processing pipeline, embedding strategies, and retrieval mechanisms. The implementation prioritizes architectural feasibility and functional correctness. Advanced considerations such as production-grade application development, optimized database storage solutions, and comprehensive system optimization are deliberately deferred to Phase 2. This phased approach allows for iterative refinement based on empirical evaluation of the foundational architecture before committing to infrastructure-level optimizations.

This chapter details the practical implementation of the core system architecture. We describe the system design, data processing pipeline, and key implementation components that enable efficient document processing and retrieval.

5.1 Architecture Design

The Unified mRAG Pipeline is architected as a modular, resource-optimized system designed to handle the complete lifecycle of document intelligence from raw ingestion to semantic generation. The system deviates from traditional monolithic designs by adopting a **Layered Service Architecture**, decoupling the retrieval logic from the generation logic, and utilizing a file-system-based state management approach for portability.

The architecture is composed of four primary layers:

1. Interface Layer (API Gateway)

The entry point to the system is a high-performance RESTful API built on **FastAPI**. This layer is responsible for request validation, concurrency management, and protocol serialization.

- **Asynchronous I/O**: To prevent thread blocking during heavy file uploads, the `/api/upload` and `/api/process` endpoints utilize Python's `async/await` pattern

combined with FastAPI's `BackgroundTasks`. This allows the server to immediately acknowledge receipt of a request while heavy computation (ETL) occurs in a separate thread pool.

- **State Management**: To minimize latency during search operations, the system utilizes a *Lifespan Event Handler*. Upon application startup, heavy models (embedding models, cross-encoders, and ColQwen weights) are pre-loaded into GPU memory and stored in the application state (`app.state`), eliminating initialization overhead per request.

2. Orchestration Layer

At the core of the system lies the `UnifiedRAGPipeline` class. This component acts as the central controller, implementing the **Facade Pattern** to abstract the complexities of the underlying subsystems.

- **Pipeline Configuration**: The behavior of the orchestrator is dictated by a dynamic YAML configuration (e.g., `config/default.yaml`). This allows the system to switch between "Text-Only," "Image-Only," or "Multimodal" modes without code changes.
- **Dependency Injection**: The orchestrator initializes and injects dependencies into the sub-modules, ensuring that the *RetrieverManager* and *Generator* share the same configuration context (e.g., chunk sizes and overlaps).

3. Data Processing and Persistence Layer

Unlike cloud-native solutions that rely on external vector databases (e.g., Pinecone, Milvus), this architecture implements a self-contained **File-System Artifact Store**. This design choice maximizes data privacy and portability. The storage architecture is organized into a rigid directory hierarchy:

4. Logical Subsystems

The system logic is divided into two specialized engines that operate in parallel during query execution:

Text Retrieval Engine

This subsystem implements a "Retrieve-then-Rerank" pipeline:

1. **Sparse Retrieval**: Uses BM25Okapi to capture keyword-specific relevance.
2. **Dense Retrieval**: Uses `all-MiniLM-L6-v2` to generate 384-dimensional embeddings for semantic search via FAISS (IndexFlatIP).
3. **Hybrid Fusion**: A weighted algorithm combines scores ($Score = \alpha \cdot Dense + (1 - \alpha) \cdot BM25$).
4. **Reranking**: A Cross-Encoder (`bge-large`) re-evaluates the top K candidates to ensure high precision.

Visual Retrieval Engine (ColQwen)

This subsystem handles non-textual data. Instead of extracting text from images (OCR), it utilizes a Vision-Language Model (ColQwen2).

1. **Rasterization**: PDFs are converted to high-resolution images (150 DPI).
2. **Visual Embedding**: The model generates a tensor representation of the visual layout.
3. **Just-in-Time Rendering**: During generation, the specific retrieved page is rendered on-the-fly and passed to the Multimodal LLM, allowing the AI to "see" charts and diagrams directly.

5.2 Data Normalization

Data normalization is a critical preprocessing step designed to reduce the entropy of input data formats. As illustrated in Figure **??**, the system accepts a wide variety of unstructured and semi-structured inputs ranging from multimedia files to office documents and funnels them into a standardized "RAG-ready" format. The normalization pipeline is governed by the `Processor` module and utilizes a suite of open-source libraries to perform format unification.

Upon upload via the API, raw files undergo immediate sanitization:

- **Filename Sanitization**: To prevent filesystem collisions and encoding errors, filenames are normalized to ASCII characters, and duplicate uploads are handled via an incremental suffix strategy.
- **Type Validation**: The system acts as a gatekeeper, filtering files based on magic numbers to ensure only supported formats proceed to the processing stage.

To facilitate both Visual (ColQwen) and Textual (Dense/Sparse) retrieval, all inputs are converted into two canonical formats: **PDF** (for visual representation) and **Markdown** (for semantic text content).

The specific normalization strategies per file type are as follows:

Multimedia Processing

To handle temporal media, the system implements a two-step transformation:

- **Video Extraction**: Raw video files (e.g., MP4) are processed using **MoviePy**. This extractor separates the visual stream from the audio stream, outputting a `.WAV` file for transcription and an optional image thumbnail.

Figure 5.1: MoviePy Extraction Process.

- **Audio Transcription**: The extracted audio or raw `.WAV` files are passed to **OpenAI Whisper** ASR engine. Whisper transcribes the speech to text, which is immediately serialized into **Markdown** format to preserve semantic structure.

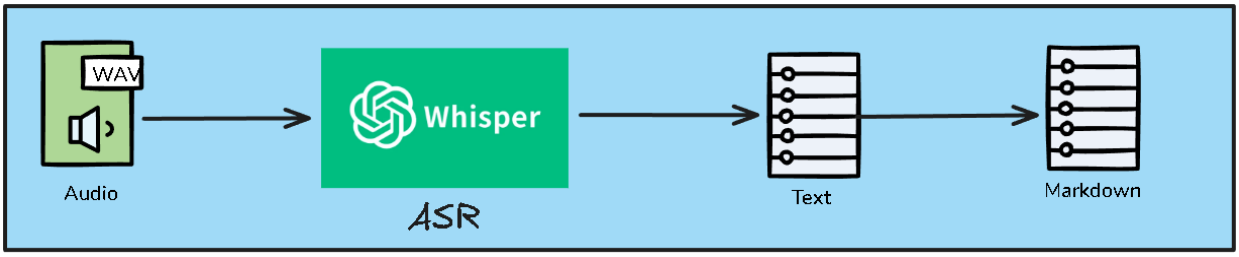

Figure 5.2: OpenAI Whisper Audio Transcription Process.

Office Document Normalization

Legacy office formats require conversion before structural analysis. The system utilizes **LibreOffice** in a headless server environment.

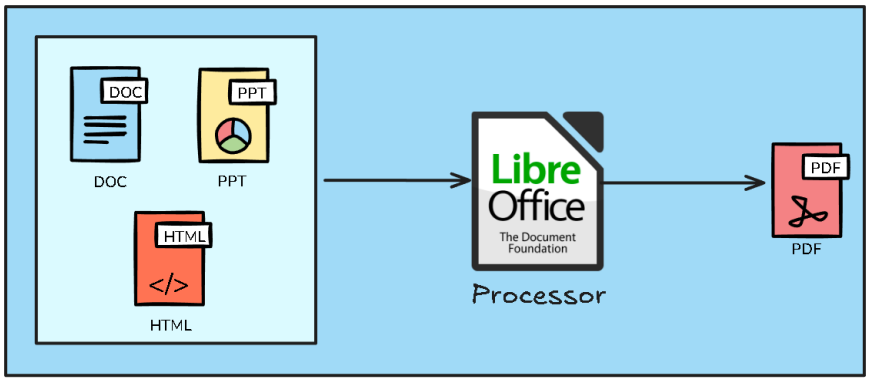

Figure 5.3: LibreOffice Document Conversion Process.

Formats such as Microsoft Word (.DOC), PowerPoint (.PPT), and HTML are normalized into standard **PDF** documents to freeze layout and fonts.

Complex Document Parsing

For data-dense documents, the pipeline integrates **Docling** as the central parsing engine:

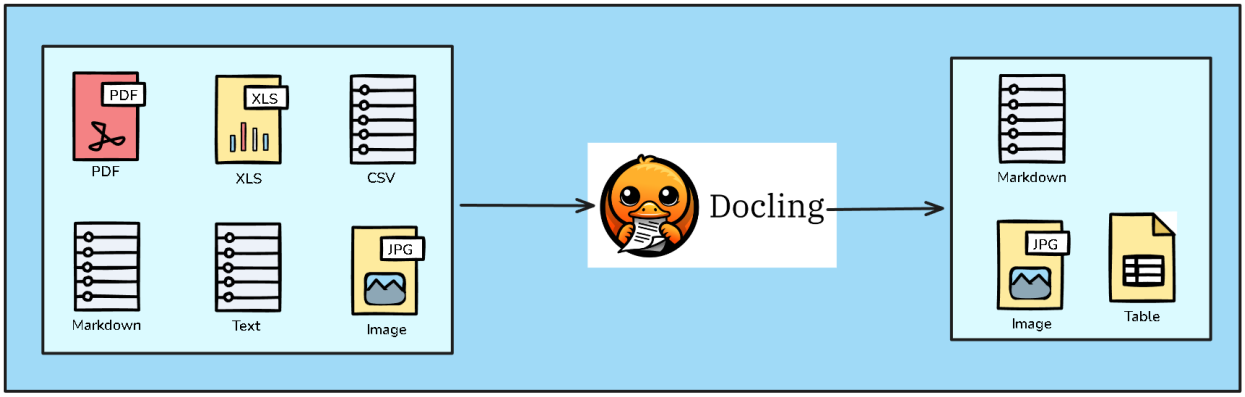

Figure 5.4: Docling Document Parsing Process.

- **Inputs**: Handles PDF, Excel (.XLS), CSV, Markdown, Text, and Images.
- **Outputs**: Docling deconstructs these inputs into three distinct normalized artifacts:
  1. **Structured Markdown**: For semantic text analysis.
  2. **Table Objects**: For preserving row/column relationships.
  3. **Isolated Images**: For downstream visual indexing.

The output of this step is a standardized set of **PDF** files (for visual processing) and **Markdown** files (for textual processing).

5.3 Data Preprocessing

The preprocessing module transforms raw files into machine-understandable formats. The pipeline splits the workflow into two parallel streams to handle multimodal data:

Text Stream:

The primary input for this stream is the structured **Markdown** files generated during the normalization phase. The preprocessing logic focuses on preparing this text for semantic vectorization: Removal of artifacts introduced during OCR or file conversion. The system utilizes the Markdown headers to maintain context, ensuring that section titles remain associated with their respective paragraphs during the splitting process.

Visual Stream:

A critical component of this pipeline is the handling of visual data within **PDF** files. The system utilizes **pdf2image** to render PDF pages into high-resolution images (defaulting to 150 DPI). This rasterization step is essential for the visual retriever (ColQwen), which treats

document pages as images to preserve layout and semantic structure that text-only extractors often lose.

5.4 Chunking and Embedding Strategies

The system employs a dual-pathway embedding architecture. This approach allows for the simultaneous retrieval of granular text segments and holistic document pages.

Textual Chunking and Embedding

To process the Markdown inputs, the system utilizes a high-precision splitting strategy designed to maximize retrieval relevance.

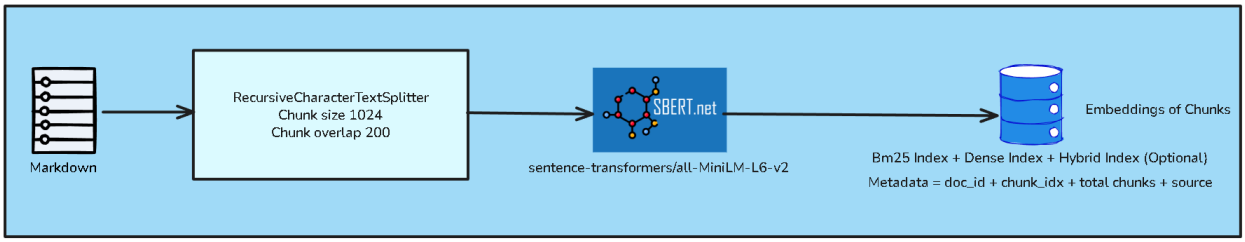

Figure 5.5: Textual Chunking and Embedding.

Textual data is segmented into chunks to fit within the context window of the embedding models. The system employs the `RecursiveCharacterTextSplitter` with the configuration using a chunk size of 1024 characters with an overlap of 200 characters. This overlap ensures semantic continuity across chunk boundaries, preventing context loss for sentences that straddle the split point.

Text Vectorization (SBERT)

Processed chunks are mapped to a high-dimensional vector space using the **Sentence-BERT (SBERT)** framework.

- **Model**: `sentence-transformers/all-MiniLM-L6-v2`.
- **Output**: A 384-dimensional dense vector representing the semantic meaning of the text chunk.
- **Index Structure**: The embedding is stored alongside a composite metadata object containing:

```
{
  "doc_id": "unique_document_identifier",
  "chunk_idx": "integer_sequence_number",
  "total_chunks": "total_count_for_document",
  "source": "original_filename"
```

```
}
```

Visual Embedding (ColQwen)

For documents rich in diagrams, charts, and complex layouts, the pipeline utilizes JPG images derived from the canonical PDF files. These intermediate image files are stored temporarily to facilitate the vision-based embedding process used by ColQwen, ensuring the model "sees" the document exactly as a human would.

The visual pathway processes the page images generated to enable multimodal retrieval.

Figure 5.6: Visual Embedding (ColQwen).

Vision-Language Model

The system integrates the **ColQwen** architecture (specifically `vidore/colqwen2-v1.0` or `v2.5`). Unlike standard OCR which strips layout, ColQwen encodes the visual semantics of the entire page.

Visual Indexing

- **Embedding Process**: Each rasterized page image is passed through the ColQwen vision encoder to produce visual tokens.
- **Storage**: These embeddings are stored in a specialized ColQwen Index.
- **Metadata Schema**: Visual embeddings are tagged with precise location data:

```
{
  "source": "original_filename",
  "source_path": "file_system_path_to_pdf",
  "page": "page_number",
  "total_pages": "document_length"
}
```

Hybrid Retrieval Integration

The retrieval unifies these two streams. The Text and Visual indices can be queried simultaneously to return both relevant text snippets and their corresponding visual page contexts.

The index supports **Hybrid**, allowing for keyword precision and semantic recall, combining two distinct algorithms:

1. **Sparse Retrieval (BM25)**: Utilizes keyword matching to capture exact terms and domain-specific jargon.
2. **Dense Retrieval**: Utilizes vector embeddings (stored in FAISS indices) to capture semantic meaning and context. The default embedding model is `all-MiniLM-L6-v2`.

Scores from both retrievers are combined using Min-Max normalization and a weighted sum.

Chapter 6

Evaluation Plan

This chapter describes the metrics and procedures used to evaluate the effectiveness, efficiency, and usability of the proposed system. The evaluation is conducted at three levels: retriever, generator, and overall system performance.

6.1 Evaluation Metrics

6.1.1 Retriever Metrics

Recall@K measures how many relevant documents are successfully retrieved within the top K results for a given query:

$$\text{Recall@K} = \frac{|\text{Relevant Documents} \cap \text{Top-}K \text{ Retrieved Documents}|}{|\text{Relevant Documents}|}$$

This metric reflects the ability of the retriever to cover all relevant information.

Precision@K evaluates the proportion of retrieved documents in the top K that are actually relevant:

$$\text{Precision@K} = \frac{|\text{Relevant Documents} \cap \text{Top-}K \text{ Retrieved Documents}|}{|\text{Top-}K \text{ Retrieved Documents}|}$$

It indicates how accurate the retrieved results are.

Retrieval Latency represents the time required for the retriever to return results:

$$\text{Latency (ms)} = \text{Result Returned Time} - \text{Query Submission Time}$$

Coverage measures how often the system retrieves at least one relevant document across all queries:

$$\text{Coverage} = \frac{\text{Number of Queries with Relevant Results}}{\text{Total Number of Queries}} \times 100$$

6.1.2 Generator Metrics

ROUGE Score quantifies textual similarity between generated outputs and reference answers. For ROUGE-N:

$$\text{ROUGE-N} = \frac{\sum_{\text{ref}} \sum_{\text{ngram} \in \text{generated}} \text{Count(ngram)}}{\sum_{\text{ref}} \sum_{\text{ngram} \in \text{reference}} \text{Count(ngram)}}$$

Relevance is assessed through human judgment, rating how well generated responses align with user intent. The final score is computed as:

$$\text{Relevance} = \frac{1}{N} \sum_{i=1}^{N} \text{Relevance Score}_i$$

Factual Accuracy measures the proportion of generated responses that are factually correct based on retrieved evidence:

$$\text{Factual Accuracy} = \frac{\text{Number of Correct Outputs}}{\text{Total Number of Outputs}} \times 100$$

Generation Latency captures the time taken by the generator to produce an output after retrieval completes:

$$\text{Latency (ms)} = \text{Output Generation Time} - \text{Retrieval Completion Time}$$

6.1.3 System-Level Metrics

Usability is evaluated via user feedback, typically collected using a Likert scale. The overall usability score is averaged across tasks:

$$\text{Usability} = \frac{1}{N} \sum_{i=1}^{N} \text{User Rating}_i$$

Task Success Rate measures the percentage of tasks that users complete successfully:

$$\text{Task Success Rate} = \frac{\text{Number of Completed Tasks}}{\text{Total Number of Tasks}} \times 100$$

End-to-End Latency reflects the total response time of the system from user input to final output delivery:

$$\text{Integration Latency (ms)} = \text{Output Delivery Time} - \text{Query Submission Time}$$

6.2 Evaluation Methodology

6.2.1 Retriever Evaluation

The retriever is evaluated using a benchmark dataset with predefined relevance judgments. Performance is analyzed across different query categories:

- **Simple Queries**: Direct queries requiring straightforward document retrieval.
- **Complex Queries**: Queries involving multiple constraints or ambiguous conditions.
- **Context-Aware Queries**: Queries that depend on contextual knowledge of the repository.

6.2.2 Generator Evaluation

The generator receives outputs from the retriever and is evaluated using:

- **Automatic Metrics**: ROUGE scores to measure overlap with reference answers.
- **Human Evaluation**: Expert assessments of relevance, correctness, and response quality.

Special attention is given to challenging cases, such as incomplete or ambiguous retrieved information, to test robustness.

6.2.3 System Evaluation

System-level evaluation is conducted through controlled user studies where participants interact with the VSCode extension to perform predefined programming tasks. Feedback is collected through questionnaires and interviews to assess usability, responsiveness, and overall user experience.

Chapter 7

Conclusion

This chapter summarizes the achievements and learning outcomes from Phase 1 of the project, and outlines the strategic execution plan for Phase 2. The project is divided into two distinct phases: Phase 1 focused on foundational research, feasibility studies, and the development of a baseline Retrieval-Augmented Generation (RAG) system with emphasis on establishing the core architecture. Phase 2 will shift toward multimodal integration, system scaling, production-grade application development, and rigorous performance evaluation.

The project follows an iterative development methodology, allowing for parallel progress in data engineering and algorithmic refinement to ensure a cohesive final system.

7.1 Phase 1: Foundation and Baseline Development (September – December)

Phase 1 was characterized by the transition from theoretical research to a functional baseline. The primary goal was to establish a robust data pipeline capable of processing heterogeneous lecture materials, including video, audio, and slides.

During the initial weeks, the focus was on concurrent research and infrastructure setup. While the literature review identified state-of-the-art Vision Language Models (VLMs) and RAG frameworks like LangChain and LlamaIndex, the repository and environment were simultaneously configured. This parallel approach allowed for immediate experimentation with ASR (Automatic Speech Recognition) systems like PhoWhisper and OCR (Optical Character Recognition) tools such as Tesseract and Mistral OCR.

A key milestone was reached in the mid-phase with the deployment of **Pipeline v1**. This version integrated basic $BM25$ and dense retrieval methods (utilizing $all - MiniLM - L6 - v2$ and $BGE - M3$), providing a benchmark for all subsequent optimizations. The team also conducted comparative analysis between frameworks to select the most efficient one for the project's specific needs. The latter half of the phase focused on stabilizing these components into a unified code repository, transitioning from experimental notebooks to

a structured backend API architecture. Additionally, user feedback was solicited through surveys to refine the system design and focus.

7.1.1 Phase 1 Learning Milestones and Key Achievements

Throughout Phase 1, the team achieved several critical learning milestones that informed system design:

- **Multimodal Data Processing Integration:** Successfully integrated ASR, OCR, and text extraction pipelines into a unified workflow, demonstrating the feasibility of processing heterogeneous lecture content without significant performance degradation.
- **Retrieval Architecture Selection:** Empirical evaluation revealed that hybrid retrieval (BM25 + dense embeddings + reranking) significantly outperforms single-method approaches, with BM25 capturing keyword relevance while dense methods capture semantic similarity.
- **Framework Evaluation:** Comparative analysis of LangChain and LlamaIndex demonstrated that LlamaIndex offers better modularity for custom RAG pipelines, leading to its selection for Phase 2.
- **Model Selection Insights:** Testing across multiple embedding models (all-MiniLM-L6-v2, BGE-M3, CLIP) revealed that task-specific embeddings (e.g., BGE-M3 for retrieval) consistently outperform general-purpose models.
- **User Feedback Integration:** Early feedback collection through surveys highlighted the importance of providing context-aware summaries alongside search results, influencing the Phase 2 feature roadmap.
- **Scalability Considerations:** Processing of 10 lectures identified memory and latency bottlenecks that inform the design of Phase 2's multimodal indexing strategy.

7.1.2 Phase 1 Implementation Journey and Accomplishments

Phase 1 successfully established a comprehensive foundation for the multimodal RAG system through systematic development across four key stages:

- **Foundation and Research Infrastructure:**
  - Established a complete development environment with Git-based version control, enabling collaborative development and systematic experimentation tracking.
  - Curated an initial dataset of 10 lectures encompassing diverse formats (videos, audio, slides) to serve as the foundation for pipeline development and testing.

  - Conducted comprehensive literature review covering RAG architectures, sparse/dense/hybrid retrieval methodologies, and multimodal embedding techniques, establishing the theoretical foundation for system design.
  - Performed comparative analysis of RAG frameworks (LangChain vs. LlamaIndex) and evaluated prompt engineering strategies, ultimately selecting LlamaIndex for its superior modularity.
  - Surveyed state-of-the-art Vision Language Models (VLMs), OCR technologies (Tesseract, Mistral OCR), and ASR systems (PhoWhisper, Whisper), informing technology selection decisions.

- **Multimodal Processing Pipeline Development:**
  - Successfully implemented audio extraction and integrated ASR capabilities using PhoWhisper, achieving high-quality transcription of Vietnamese lecture content.
  - Deployed and benchmarked multiple OCR solutions on lecture slides, establishing optimal processing configurations for diverse slide formats and content densities.
  - Developed initial retrieval components including BM25 sparse retrieval and dense retrievers using embedding models (all-MiniLM-L6-v2, BGE-M3).
  - Created "Hello World" RAG prototypes in both LangChain and LlamaIndex frameworks, enabling hands-on comparison of architectural patterns and API designs.

- **Baseline System Integration and Optimization:**
  - Constructed a unified evaluation framework supporting systematic comparison of BM25 and dense retrievers using standard metrics (MRR, NDCG), enabling data-driven architecture decisions.
  - Conducted extensive reranker experiments and explored multi-embedding approaches (CLIP, BLIP), discovering that hybrid retrieval with reranking significantly improves result quality.
  - Completed comprehensive benchmarking of OCR and ASR alternatives, selecting optimal tools based on accuracy, processing speed, and resource requirements.
  - Architected and implemented a robust multi-format document processor capable of handling text, audio, and video inputs with unified output schemas.
  - Designed the initial system architecture including component interactions, data flow patterns, and database schema supporting multimodal content indexing.

- **System Refinement and Documentation:**
  - Refined data preprocessing and embedding pipelines, optimizing chunking strategies and metadata extraction to improve retrieval accuracy.
  - Transformed experimental Jupyter notebooks into a production-ready code repository with modular architecture, proper error handling, and comprehensive logging.

- Developed an initial evaluation dataset with carefully crafted query-answer pairs, establishing baseline performance metrics for future comparison.
- Designed UI wireframes and implemented a functional backend API (`/query` endpoint), demonstrating end-to-end system integration.
- Gathered preliminary user feedback via Google Forms surveys, identifying key usability requirements and feature priorities for Phase 2.
- Completed comprehensive system documentation including architecture diagrams, API specifications, and implementation details.
- Finalized chunking strategies and metadata management approaches, ensuring optimal balance between retrieval granularity and semantic coherence.

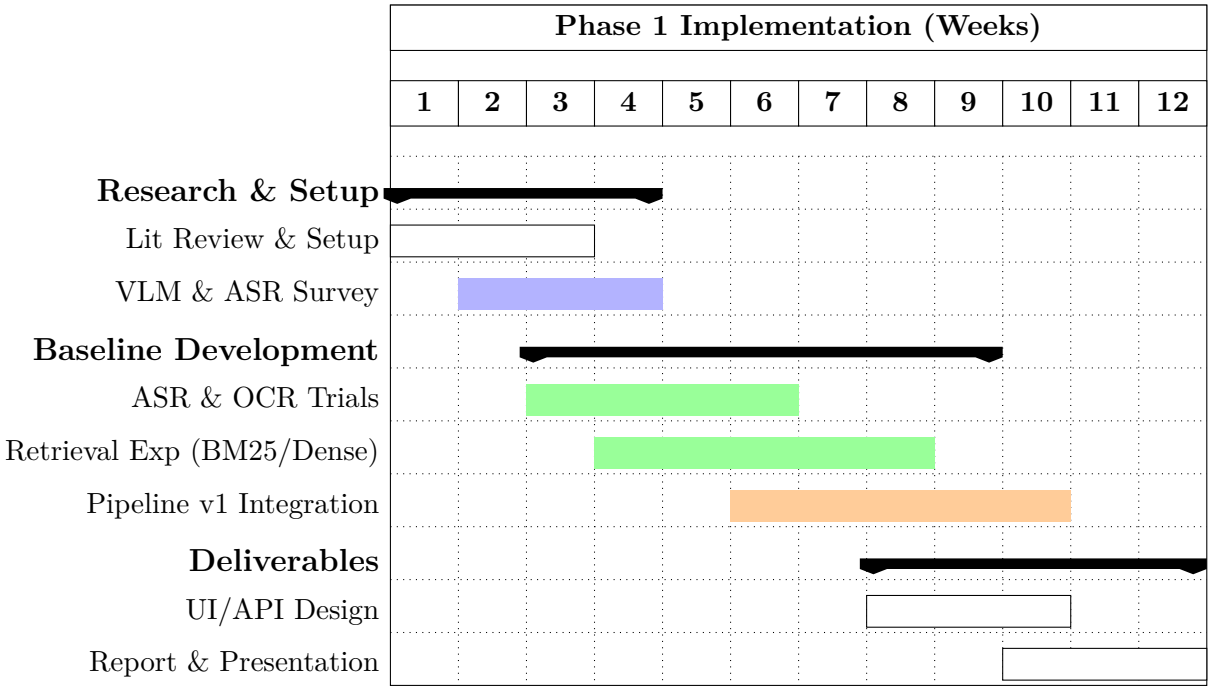

Figure 7.1: Phase 1 Implementation Timeline: Completed work showing the parallelization of research, processing, and experimental tracks across the 12-week development period.

7.2 Phase 2: Future Work – Multimodal Integration and Scaling (January – May)

Building upon the solid foundation established in Phase 1, Phase 2 will focus on evolving the system from a text-centric baseline to a fully multimodal, production-ready platform. The planned workload is organized into three complementary streams: Multimodal Engineering, Interface Development, and Academic Evaluation.

The primary technical challenge for Phase 2 is implementing temporal synchronization of video segments with corresponding slide content. This will require developing a custom alignment algorithm (such as MaViLS) capable of linking ASR timestamps with OCR-detected visual transitions. Concurrently, the system will be scaled to process and index over 50 lectures, necessitating efficient multimodal embedding pipelines (potentially utilizing VISTA) and advanced RRF (Reciprocal Rank Fusion) based retrieval logic.

The concluding weeks of Phase 2 will be dedicated to rigorous system evaluation through comprehensive stress testing and systematically answering the core Research Questions (RQs) regarding retrieval accuracy, generation faithfulness, and overall system quality. Phase 2 will culminate in the final thesis defense.

7.2.1 Phase 2 Planned Work Schedule

- **Phase 2 Kick-off and Integration Planning (Week 13):**
  - Plan data scaling to 50+ lectures.
  - Research video–slide synchronization techniques.
  - Design hybrid retrieval architecture with multimodal embeddings and RRF fusion.
- **Core Module Implementation and Dataset Construction (Week 14):**
  - Implement video–slide synchronization prototype.
  - Construct multimodal embedding and vector indexing pipeline.
  - Collaborative creation of a large-scale evaluation dataset (50+ Q&A pairs).
- **Hybrid RAG Integration and Frontend Development (Weeks 15–17):**
  - Integration of sparse, dense, and multimodal retrievers.
  - Delivery of a unified retrieval function to the application layer.
  - Backend–frontend connection and synchronized result display development.
- **System Evaluation and Thesis Writing (Weeks 18–20):**
  - Full benchmark evaluation and metric analysis for RQ2.
  - Experiments on faithfulness and grounded generation for RQ3.
  - Writing of core thesis chapters on data pipeline, retrieval, and generation.
- **Final Report and Presentation Preparation (Weeks 21–23):**
  - Finalization of processing and retrieval scripts.
  - UI/UX refinement and demo script preparation.
  - Completion of thesis draft and presentation slides.
- **Defense Preparation and Thesis Defense (Weeks 24–25):**

– Presentation rehearsals and Q&A preparation covering data, retrieval, and architecture.

– Final thesis defense.

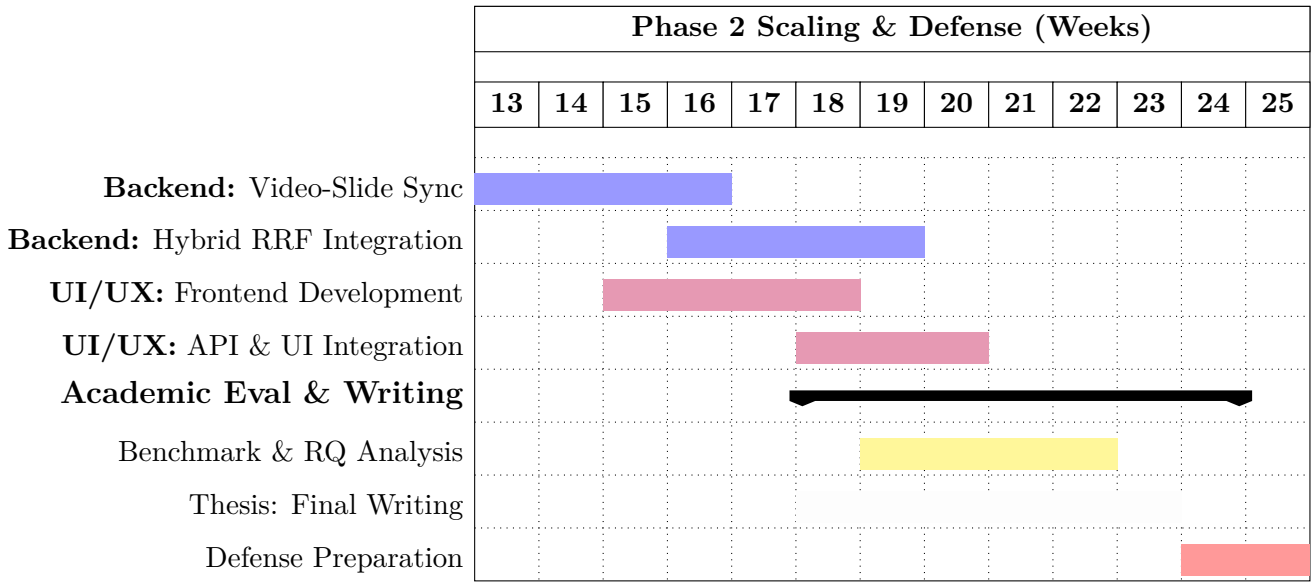

Figure 7.2: Phase 2 Planned Timeline: Proposed schedule highlighting the strategic overlap between multimodal backend development, frontend integration, and academic evaluation across the 13-week implementation period.

References

[1] M. J. Eppler and J. Mengis. "The Concept of Information Overload: A Review of Literature from Organization Science, Accounting, Marketing, MIS, and Related Disciplines". In: *The Information Society* 20.5 (2004), pp. 325–344.

[2] J. Sweller, P. Ayres, and S. Kalyuga. *Cognitive Load Theory*. Springer, 2011.

[3] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber. "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks". In: *Proceedings of the 23rd International Conference on Machine Learning (ICML)*. 2006, pp. 369–376.

[4] W. Chan, N. Jaitly, Q. V. Le, and O. Vinyals. "Listen, attend and spell". In: *arXiv preprint arXiv:1508.01211* (2015).

[5] A. Baevski, H. Zhou, A. Mohamed, and M. Auli. "wav2vec 2.0: A framework for self-supervised learning of speech representations". In: *arXiv preprint arXiv:2006.11477* (2020).

[6] A. Radford et al. "Robust speech recognition via large-scale weak supervision". In: *arXiv preprint arXiv:2212.04356* (2022). Whisper.

[7] G. Hinton et al. "Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups". In: *IEEE Signal Processing Magazine* 29.6 (2012), pp. 82–97.

[8] L. R. Rabiner. "A tutorial on hidden Markov models and selected applications in speech recognition". In: *Proceedings of the IEEE* 77.2 (1989), pp. 257–286.

[9] D. Povey et al. "The Kaldi speech recognition toolkit". In: *IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*. 2011.

[10] J. Godfrey, E. Holliman, and J. McDaniel. "Switchboard: A telephone speech corpus for research and development". In: *ICASSP*. 1992.

[11] D. B. Paul and J. M. Baker. "CSR-I (WSJ0) Complete LDC93S6A". In: *Linguistic Data Consortium (LDC)*. 1993.

[12] A. Graves. "Sequence transduction with recurrent neural networks". In: *arXiv preprint arXiv:1211.3711* (2012).

[13] Y. He et al. "Streaming end-to-end speech recognition for mobile devices". In: *ICASSP*. 2019.

[14] A. Gulati et al. “Conformer: Convolution-augmented Transformer for speech recognition”. In: *arXiv preprint arXiv:2005.08100* (2020).

[15] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur. “LibriSpeech: An ASR corpus based on public domain audio books”. In: *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2015.

[16] A. Rousseau, P. Deléglise, and Y. Estève. “TED-LIUM: an automatic speech recognition dedicated corpus”. In: *LREC*. 2012.

[17] R. Ardila et al. “Common voice: A massively-multilingual speech corpus”. In: *arXiv preprint arXiv:1912.06670* (2020).

[18] R. Smith. “An Overview of the Tesseract OCR Engine”. In: *Proceedings of the 9th International Conference on Document Analysis and Recognition (ICDAR)*. 2007, pp. 629–633.

[19] X. Chen, L. Jin, Y. Zhu, and C. Luo. “Text Recognition in the Wild: A Survey”. In: *ACM Computing Surveys* 54.2 (2021), pp. 1–35.

[20] V. Govindaraju and S. N. Srihari. “Handwritten OCR: Challenges and Opportunities”. In: *Proceedings of the IEEE* 80.7 (1993), pp. 1029–1038.

[21] S. Mori, C. Y. Suen, and K. Yamamoto. “Historical Review of OCR Research and Development”. In: *Proceedings of the IEEE* 80.7 (1992), pp. 1029–1058.

[22] R. G. Casey and E. Lecolinet. “A Survey of Methods and Strategies in Character Segmentation”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 18.7 (July 1996), pp. 690–706.

[23] R. Plamondon and S. N. Srihari. “On-line and Off-line Handwriting Recognition: A Comprehensive Survey”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 22.1 (Jan. 2000), pp. 63–84.

[24] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (Nov. 1998), pp. 2278–2324.

[25] A. Graves, M. Liwicki, S. Fernández, R. Bertolami, H. Bunke, and J. Schmidhuber. “A novel connectionist system for unconstrained handwriting recognition”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 31.5 (May 2009), pp. 855–868.

[26] M. Li, Y. Sun, Y. Wang, B. Li, and T.-Y. Liu. “TrOCR: Transformer-based optical character recognition with pre-trained models”. In: *Proceedings of the 37th AAAI Conference on Artificial Intelligence*. 2023, pp. 11–19.

[27] B. Shi, X. Bai, and C. Yao. “An End-to-End Trainable Neural Network for Image-Based Sequence Recognition and Its Application to Scene Text Recognition”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2016).

[28] J. Baek, G. Kim, et al. “What is Wrong with Scene Text Recognition Model Comparisons? Dataset and Model Analysis”. In: *ICCV*. 2019.

[29] M. Li et al. “TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models”. In: *NeurIPS* (2021).

[30] J. Baek, G. Lee, J. Han, S. Yun, S. Lee, J. Na, and H. Kim. "What is wrong with scene text recognition model comparisons? Dataset and model analysis". In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2019, pp. 4715–4723.

[31] R. Holley. "How good can it get? Analysing and improving OCR accuracy in large scale historic newspaper digitisation programs". In: *D-Lib Magazine* 15.3/4 (2009).

[32] V. Govindaraju. "Handwritten digit recognition: Benchmarks, state of the art, and directions". In: *Pattern Recognition Letters* 16.1 (1994), pp. 1–9.

[33] Y. Zhou, H. Hu, X. Li, and J. Song. "Automatic OCR of medical prescriptions using deep convolutional networks". In: *Journal of Biomedical Informatics* 127 (2022), p. 103998.

[34] C. N. E. Anagnostopoulos, I. E. Anagnostopoulos, V. Loumos, and E. Kayafas. "A license plate-recognition algorithm for intelligent transportation system applications". In: *IEEE Transactions on Intelligent Transportation Systems* 7.3 (Sept. 2006), pp. 377–392.

[35] S. Du, M. Ibrahim, and M. Shehata. "Automatic license plate recognition (ALPR): A state-of-the-art review". In: *IEEE Transactions on Circuits and Systems for Video Technology* 23.2 (Feb. 2013), pp. 311–325.

[36] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. "Attention Is All You Need". In: *arXiv preprint arXiv:1706.03762* (2017). URL: https://arxiv.org/pdf/1706.03762.

[37] S. Liu, W. Quan, M. Zhou, S. Chen, J. Kang, Z. Zhao, C. Chen, and D. Yan. "M2HF: Multi-level Multi-modal Hybrid Fusion for Text-Video Retrieval". In: *CoRR* abs/2208.07664 (2022). URL: https://arxiv.org/abs/2208.07664.

[38] Zhu et al. "Reranking Multi-Modal Retrievers for Video Retrieval". In: Unpublished draft / preprint. 2024. URL: https://www.mlmi.eng.cam.ac.uk/files/2023-2024/zhu_re-ranking_2024_reduced.pdf.

[39] G. Geigle, J. Pfeiffer, N. Reimers, and I. Vulić Iryna Gurevych. "Retrieve Fast, Rerank Smart: Cooperative and Joint Approaches for Improved Cross-Modal Retrieval". In: *arXiv preprint* (2021). eprint: 2103.11920.

[40] A. Radford et al. "Learning Transferable Visual Models From Natural Language Supervision". In: *arXiv preprint* (2021). eprint: 2103.00020.

[41] S.-C. Lin, C. Lee, M. Shoeybi, J. Lin, B. Catanzaro, and W. Ping. "MM-Embed: Universal Multimodal Retrieval with Multimodal LLMs". In: *International Conference on Representation Learning (ICLR)*. 2025. URL: https://proceedings.iclr.cc/paper/2025/hash/6d5d6afa9957cfc9142ba60e78a467e9-Abstract.html.

[42] R. Girdhar, A. El-Nouby, Z. Liu, M. Singh, K. V. Alwala, A. Joulin, and I. Misra. "ImageBind: One Embedding Space To Bind Them All". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2023, pp. 15180–15190. URL: https://openaccess.thecvf.com/content/CVPR2023/papers/Girdhar_ImageBind_One_Embedding_Space_To_Bind_Them_All_CVPR_2023_paper.pdf.

[43] M. Günther, S. Sturua, M. K. Akram, I. Mohr, A. Ungureanu, S. Eslami, S. Martens, B. Wang, N. Wang, and H. Xiao. "jina-embeddings-v4: Universal Embeddings for Multimodal Multilingual Retrieval". In: *arXiv preprint* (2025). eprint: 2506.18902. URL: https://arxiv.org/abs/2506.18902.

[44] D. Chen, A. Fisch, J. Weston, and A. Bordes. "Reading Wikipedia to Answer Open-Domain Questions". In: *ACL*. 2017.

[45] K. Lee, M.-W. Chang, and K. Toutanova. "Open-Domain Question Answering with Pre-Training Information Retrieval". In: *ACL*. 2019.

[46] U. Khandelwal, A. Fan, D. Jurafsky, L. Zettlemoyer, and M. Lewis. "Generalization through Memorization: Nearest Neighbor Language Models". In: *ICLR*. 2020.

[47] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang. "REALM: Retrieval-Augmented Language Model Pre-Training". In: *ICML*. 2020.

[48] P. Lewis, B. Oguz, R. Rinott, S. Riedel, T. Rocktäschel, D. Lukovnikov, F. Petroni, M. Lewis, E. Perez, and V. Karpukhin. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *NeurIPS Datasets and Benchmarks Track*. 2020.

[49] Y. Qu et al. "RocketQA: An Optimized Training Approach to Dense Passage Retrieval for Open-Domain Question Answering". In: *NAACL*. 2021. URL: https://aclanthology.org/2021.naacl-main.466.pdf.

[50] S. Ren et al. "RocketQAv2: A Joint Training Method for Dense Passage Retrieval and Cross-Encoder Reranking". In: *arXiv* (2021). URL: https://arxiv.org/pdf/2110.07367.

[51] L. Gao et al. "Condenser: a Pre-training Architecture for Dense Retrieval". In: *EMNLP*. 2021. URL: https://aclanthology.org/2021.emnlp-main.75.pdf.

[52] G. Izacard et al. "Unsupervised Contrastive Learning for Dense Retrieval". In: *arXiv* (2021). URL: https://arxiv.org/pdf/2112.09118.

[53] J. Ni et al. "Text-to-Text Transfer for Dense Retrieval (GTR)". In: *arXiv* (2021). URL: https://arxiv.org/pdf/2112.07899.

[54] K. Santhanam et al. "ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction". In: *arXiv* (2021). URL: https://arxiv.org/pdf/2112.01488.

[55] T. Formal et al. "SPLADE: Sparse Lexical and Expansion Model for Information Retrieval". In: *arXiv* (2021). URL: https://arxiv.org/pdf/2107.05720.

[56] T. Formal et al. "SPLADE v2: Sparse Lexical and Expansion Model for IR". In: *arXiv* (2021). URL: https://arxiv.org/pdf/2109.10086.

[57] W. Xiong et al. “Learned Multi-Document Retrieval for QA (EMDR)”. In: *arXiv* (2021). URL: https://arxiv.org/pdf/2106.05346.

[58] W. Xiong et al. “Multi-hop Dense Retrieval for Open-Domain QA (MDR)”. In: (2020). URL: https://arxiv.org/pdf/2009.12756.

[59] G. Izacard et al. “Fusion-in-Decoder: Generative Multi-Passage QA”. In: (2020). URL: https://arxiv.org/pdf/2007.01282.

[60] X. Zhang et al. “KG-FiD: Fusion-in-Decoder with Graph Knowledge”. In: (2021). URL: https://arxiv.org/pdf/2110.04330.

[61] Y. Tay et al. “FiD-Light: Scaling Multi-Passage Readers with Runtime Constraints”. In: (2022). URL: https://arxiv.org/pdf/2209.14290.

[62] S. Borgeaud et al. “RETRO: Retrieval-Augmented Transformer”. In: (2021). URL: https://arxiv.org/pdf/2112.04426.

[63] O. Ram et al. “Retrieval as Attention”. In: (2022). URL: https://arxiv.org/pdf/2212.02027.

[64] K. Guu et al. “Re2G: Retrieval, Reranking, Generation in One Model”. In: *NAACL*. 2022. URL: https://aclanthology.org/2022.naacl-main.194.pdf.

[65] Y. Mao et al. “GAR: Generative Augmented Retrieval for Open-Domain QA”. In: *ACL*. 2021. URL: https://aclanthology.org/2021.acl-long.316.pdf.

[66] O. Press et al. “Self-Ask: Chain-of-Thought with Search”. In: (2022). URL: https://arxiv.org/pdf/2210.03350.

[67] S. Yao et al. “ReAct: Reason + Act with Search”. In: (2022). URL: https://arxiv.org/pdf/2210.03629.

[68] R. Nakano et al. “WebGPT: Browser-Assisted QA”. In: (2021). URL: https://arxiv.org/pdf/2112.09332.

[69] L. Gao et al. “HyDE: Hypothetical Documents for Retrieval”. In: (2022). URL: https://arxiv.org/pdf/2212.10496.

[70] F. Author and S. Author. “FLARE: Triggered Retrieval during Decoding”. In: (2023). URL: https://arxiv.org/pdf/2305.06983.

[71] A. Author and B. Author. “Adaptive-RAG: Learning to Route Retrieval Strategies”. In: (2024). URL: https://arxiv.org/pdf/2403.14403.

[72] C. Author and D. Author. “SEAKR: Selective Retrieval for Adaptive RAG”. In: (2024). URL: https://arxiv.org/pdf/2406.19215.

[73] E. Author and F. Author. “DRAGIN: Dynamic Retrieval and Grounding in Interactive NLU”. In: *Proceedings of ACL 2024*. 2024. URL: https://aclanthology.org/2024.acl-long.702.pdf.

[74] G. Author and H. Author. “SELF-RAG: Alternating Retrieve, Generate, and Critique”. In: (2023). URL: https://arxiv.org/pdf/2310.11511.

[75] I. Author and J. Author. “CRAG: Corrective Retrieval-Augmented Generation”. In: (2024). URL: https://arxiv.org/pdf/2401.15884.

[76] K. Author and L. Author. “CoV-RAG: Chain-of-Verification for Retrieval-Augmented Generation”. In: *Findings of EMNLP 2024*. 2024. URL: https://aclanthology.org/2024.findings-emnlp.607.pdf.

[77] M. Author and N. Author. “On LM Priors vs Retrieved Evidence”. In: (2024). URL: https://arxiv.org/pdf/2404.10198v1.

[78] O. Author and P. Author. “RankRAG: Instruction-Tuned Ranking for Context Selection”. In: (2024). URL: https://proceedings.neurips.cc/paper_files/paper/2024/file/db93ccb6cf392f352570dd5af0a223d3-Paper-Conference.pdf.

[79] Q. Author and R. Author. “LLMLingua: Compressing and Generalizing Multilingual Prompts”. In: *EMNLP 2023*. 2023. URL: https://aclanthology.org/2023.emnlp-main.825.pdf.

[80] S. Author and T. Author. “Contextual Compression for Retrieval-Augmented Generation”. In: (2024). URL: https://arxiv.org/pdf/2409.13385.

[81] U. Author and V. Author. “AU-RAG: Source-Aware Routing among Heterogeneous Corpora”. In: (2024). URL: https://dl.acm.org/doi/pdf/10.1145/3673791.3698416.

[82] W. Author and X. Author. “MSPR: Multi-Source Policy Retrieval”. In: (2024). URL: https://arxiv.org/pdf/2411.00689v1.

[83] Y. Han et al. “GraphRAG: Retrieval-Augmented Generation with Graph-Based Retrieval”. In: (2024). URL: https://arxiv.org/pdf/2404.16130.

[84] L. Hu et al. “GRAG: Graph Retrieval-Augmented Generation”. In: (2024). URL: https://arxiv.org/pdf/2405.16506.

[85] Y. Author and Z. Author. “LongRAG: Retrieval with Long-Context LLMs”. In: (2024). URL: https://arxiv.org/pdf/2406.15319.

[86] A. Author and B. Author. “Self-Route: Routing between Long-Context Reading and Targeted RAG”. In: (2024). URL: https://arxiv.org/pdf/2407.16833.

[87] C. Author and D. Author. “Survey on Agentic RAG: Reflection, Planning, and Tool Use”. In: (2025). URL: https://arxiv.org/pdf/2501.09136v2.

[88] E. Author and F. Author. “Survey on Reasoning-Agentic RAG”. In: (2025). URL: https://arxiv.org/pdf/2506.10408v1.

[89] G. Author and H. Author. “HopRAG: Logic-Guided Hops over Passage Graphs”. In: (2025). URL: https://arxiv.org/pdf/2502.12442.

[90] I. Author. “Knowledge-Oriented RAG Survey”. In: (2025). URL: https://arxiv.org/pdf/2503.10677.

[91] J. Author. “RAG: Architectures, Enhancements, and Robustness Survey”. In: (2025). URL: https://arxiv.org/pdf/2506.00054v1.

[92] K. Author and L. Author. “FaithfulRAG: Enforcing Evidence in Generation”. In: *arXiv preprint*. 2025. URL: https://arxiv.org/pdf/2506.08938.

[93] M. Author. “ReCLAIM: Constraining Decoding with Per-Sentence Citations”. In: (2024). URL: https://arxiv.org/pdf/2407.01796.

[94] N. Author. “CiteFix: Post-hoc Citation Repair and Validation”. In: (2025). URL: https://arxiv.org/pdf/2504.15629.

[95] O. Author. “Source Attribution in RAG”. In: (2025). URL: https://arxiv.org/pdf/2507.04480.

[96] P. Author. “CiteEval / CiteBench: Benchmarks for Citation Quality”. In: (2025). URL: https://arxiv.org/pdf/2506.01829v1.

[97] Q. Author. “ARES: Automated Relevance and Faithfulness Evaluation Suite”. In: *NAACL 2024*. 2024. URL: https://aclanthology.org/2024.naacl-long.20.pdf.

[98] R. Author. “Survey of RAG Evaluation Metrics and Datasets”. In: (2025). URL: https://arxiv.org/pdf/2504.14891.

[99] S. Author. “RAGtifier / LiveRAG Challenge”. In: (2025). URL: https://arxiv.org/pdf/2506.14412v2.

[100] T. Author. “HippoRAG-2: Associative Graph-Traversal Memory for RAG”. In: (2025). URL: https://arxiv.org/pdf/2502.14802.

[101] U. Author. “MemoRAG v3: Dual-System Memory for Retrieval-Augmented Generation”. In: (2024). URL: https://arxiv.org/pdf/2409.05591v3.

[102] V. Author. “Mem0: Persistent Memory for LLMs”. In: (2025). URL: https://arxiv.org/pdf/2504.19413.

[103] W. Author. “MIRIX: Agent-Memory Systems for Retrieval-Augmented Agents”. In: (2025). URL: https://arxiv.org/pdf/2507.07957.

[104] X. Author. “MemOS: Operating Systems for Agentic Memory”. In: (2025). URL: https://arxiv.org/pdf/2507.03724.

[105] K. Marino, M. Rastegari, A. Farhadi, and R. Mottaghi. “OK-VQA: A Visual Question Answering Benchmark Requiring External Knowledge”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019. URL: https://openaccess.thecvf.com/content_CVPR_2019/papers/Marino_OK-VQA_A_Visual_Question_Answering_Benchmark_Requiring_External_Knowledge_CVPR_2019_paper.pdf.

[106] F. Gao, Q. Ping, G. Thattai, A. Reganti, Y. N. Wu, and P. Natarajan. “Transform-Retrieve-Generate: Natural Language-Centric Outside-Knowledge Visual Question Answering”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022. URL: https://paperswithcode.com/paper/transform-retrieve-generate-natural-language.

[107] J. Wu and R. J. Mooney. “Entity-Focused Dense Passage Retrieval for Outside-Knowledge Visual Question Answering”. In: *arXiv preprint* (2022). URL: https://arxiv.org/abs/2210.10176.

[108] Z. Hu, A. Iscen, C. Sun, Z. Wang, K.-W. Chang, Y. Sun, C. Schmid, D. A. Ross, and A. Fathi. “REVEAL: Retrieval-Augmented Visual-Language Pre-Training with Multi-Source Multimodal Knowledge Memory”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2023. URL: https://reveal-cvpr.github.io/.

[109] S. Yu et al. "VisRAG: Vision-based Retrieval-augmented Generation on Multi-modality Documents". In: *arXiv preprint* (2024). eprint: 2410.10594. URL: https://arxiv.org/abs/2410.10594.

[110] V. N. Rao, S. Choudhary, A. Deshpande, R. K. Satzoda, and S. Appalaraju. "RAVEN: Multitask Retrieval Augmented Vision-Language Learning". In: *arXiv preprint* (2024). eprint: 2406.19150. URL: https://arxiv.org/abs/2406.19150.

[111] C. Wen, Y. Lin, X. Qu, N. Li, Y. Liao, H. Lin, and X. Li. "RS-RAG: Bridging Remote Sensing Imagery and Comprehensive Knowledge with a Multi-Modal Dataset and Retrieval-Augmented Generation Model". In: *arXiv preprint* (2025). eprint: 2504.04988. URL: https://arxiv.org/abs/2504.04988.

[112] N. Drushchak, N. Polyakovska, M. Bautina, T. Semenchenko, J. Koscielecki, W. Sykala, and M. Wegrzynowski. "Multimodal Retrieval-Augmented Generation: Unified Information Processing Across Text, Image, Table, and Video Modalities". In: *Proceedings of the Workshop on Multimodal Augmented Generation via Multimodal Retrieval (MAGMAR), ACL / EMNLP 2025*. 2025. URL: https://aclanthology.org/2025.magmar-1.5.pdf.

[113] L. Mei, S. Mo, Z. Yang, and C. Chen. "A Survey of Multimodal Retrieval-Augmented Generation". In: *arXiv preprint* (2025). eprint: 2504.08748. URL: https://arxiv.org/abs/2504.08748.

[114] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih. "Dense Passage Retrieval for Open-Domain Question Answering". In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2020, pp. 6769–6781. DOI: 10.18653/v1/2020.emnlp-main.550.

[115] G. Izacard and E. Grave. "Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering". In: *arXiv preprint* (2021). eprint: 2112.04426.

[116] Y. Luan, J. Eisenstein, K. Toutanova, and M. Collins. "Sparse, Dense, and Attentional Representations for Text Retrieval". In: *Transactions of the Association for Computational Linguistics* 9 (2021), pp. 329–345. DOI: 10.1162/tacl_a_00369.

[117] G. Izacard and E. Grave. "Sequence-to-Sequence Retrieval-Augmented Generation". In: *arXiv preprint* (2022). eprint: 2201.11487.

[118] Y. Gao et al. "Retrieval-Augmented Generation for Large Language Models: A Survey". In: *arXiv preprint arXiv:2312.10997* (2023). URL: https://arxiv.org/abs/2312.10997.

[119] C.-W. Hu, Y. Wang, S. Xing, C.-J. Chen, and Z. Tu. "mRAG: Elucidating the Design Space of Multi-modal Retrieval-Augmented Generation". In: *arXiv preprint* (2025). eprint: 2505.24073.

[120] P. Liu, X. Liu, R. Yao, J. Liu, S. Meng, D. Wang, and J. Ma. "HM-RAG: Hierarchical Multi-Agent Multimodal Retrieval-Augmented Generation". In: *arXiv preprint* (2025). eprint: 2504.12330.

[121] T. Zhang et al. "mR^{2}AG: Multimodal Retrieval-Reflection-Augmented Generation for Knowledge-Based VQA". In: *arXiv preprint*. 2024. eprint: 2411.01920.

[122] S. E. Robertson and H. Zaragoza. "The Probabilistic Relevance Framework: BM25 and Beyond". In: *Foundations and Trends in Information Retrieval* 3.4 (2009), pp. 333–389. DOI: 10.1561/1500000019.

[123] T. Formal, C. Lassance, B. Piwowarski, and S. Clinchant. "SPLADE v2: Sparse Lexical and Expansion Model for Information Retrieval". In: *arXiv preprint* (2021). eprint: 2109.10086.

[124] A. Singh, M. D'Arcy, A. Cohan, D. Downey, and S. Feldman. "SciRepEval: A Multi-Format Benchmark for Scientific Document Representations". In: *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2022.

[125] Y. A. Malkov and D. A. Yashunin. "Efficient And Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs". In: *arXiv preprint* (2016). eprint: 1603.09320.

[126] P. Lewis, B. Oguz, R. Rinott, S. Riedel, T. Rocktäschel, D. Lukovnikov, F. Petroni, M. Lewis, E. Perez, and V. Karpukhin. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Proceedings of the 2020 Conference on Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*. 2020.

[127] H. Iida and N. Okazaki. "Unsupervised Domain Adaptation for Sparse Retrieval by Filling Vocabulary and Word Frequency Gaps". In: *ACL 2022*. 2022.

[128] J. Chen, Z. Zhang, T. Ge, and F. Wei. "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation". In: *arXiv preprint arXiv:2402.03216* (2024). URL: https://arxiv.org/abs/2402.03216.

[129] K. Santhanam, O. Khattab, L. Tang, Z. Sun, X. Zhao, B. Kraft, and M. Zaharia. "ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction". In: *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2022.

[130] R. Nogueira and J. Lin. "MonoT5: Text-to-Text Transfer Transformer for Information Retrieval". In: *Proceedings of the 44th European Conference on Information Retrieval (ECIR)*. arXiv:2003.06713. 2021.

[131] J. Ni, A. Yu, M. Ghazvininejad, B. V. Durme, and J. Balhan. "TART: A Retrieval-Augmented Transformer for Zero-Shot Information Retrieval". In: *arXiv preprint arXiv:2305.15383* (2023).

[132] H. Sun, H. Tang, Y. Song, and J. He. "RankGPT: Instruction Tuning Large Language Models for Information Retrieval". In: *arXiv preprint arXiv:2303.11366* (2023).

[133] C. Yoon, T. Lee, H. Hwang, M. Jeong, and J. Kang. "Compact: Compressing Retrieved Documents Actively for Question Answering". In: (2024).

[134] A. Asai et al. "SELF-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection". In: *arXiv preprint arXiv:2310.11511* (2023). URL: https://arxiv.org/abs/2310.11511.

[135] P. He et al. “Exploring Demonstration Retrievers in RAG for Coding Tasks: Yeas and Nays!”. In: *arXiv preprint arXiv:2410.09662* (2024). URL: https://arxiv.org/abs/2410.09662.

[136] Z. Zhang et al. “LLM Hallucinations in Practical Code Generation: Phenomena, Mechanism, and Mitigation”. In: *arXiv preprint arXiv:2409.20550* (2024). URL: https://arxiv.org/abs/2409.20550.

[137] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, et al. “Language Models are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems* (2020).

[138] L. Reynolds and K. McDonell. “Prompt Programming for Large Language Models: Beyond the Few-Shot Paradigm”. In: *arXiv preprint arXiv:2102.07350* (2021).

[139] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, et al. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. In: *Advances in Neural Information Processing Systems* (2022).

[140] X. Wang, J. Wei, D. Schuurmans, M. Bosma, E. Chi, Q. Le, and D. Zhou. “Self-Consistency Improves Chain-of-Thought Reasoning in Language Models”. In: *International Conference on Learning Representations* (2023).

[141] Z. Shi et al. “TRIPLE: Bandit-based Prompt Selection Under Query Budget”. In: *ICLR* (2024).

[142] X. L. Li and P. Liang. “Prefix-Tuning: Optimizing Continuous Prompts for Generation”. In: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*. 2021.

[143] T. Shin, Y. Razeghi, M. Alzantot, X. Ren, and P. Liu. “AutoPrompt: Eliciting Knowledge from Language Models with Automatically Generated Prompts”. In: *Proceedings of EMNLP* (2020).

[144] A. Prasad, S. Arora, et al. “GrIPS: Gradient-free Instruction Prompt Search”. In: *arXiv preprint arXiv:2305.00318* (2023).

[145] S. e. a. Zhou. “Large Language Models are Human-Level Prompt Engineers”. In: *EMNLP*. 2022.

[146] S. e. a. Yao. “Large Language Models as Optimizers”. In: *arXiv preprint arXiv:2309.03409* (2023).

[147] Y. Deng et al. “RLPrompt: Optimizing Discrete Text Prompts with Reinforcement Learning”. In: *arXiv preprint arXiv:2205.12548* (2022).

[148] S.-C. Lin and J. Lin. “Densifying Sparse Representations for Passage Retrieval by Representational Slicing”. In: *arXiv preprint* (2021). eprint: 2112.04666.

[149] J. Sun, X. Zhong, S. Zhou, and J. Han. “DynamicRAG: Leveraging Outputs of Large Language Model as Feedback for Dynamic Reranking in Retrieval-Augmented Generation”. In: *arXiv preprint* (2025). eprint: 2505.07233.

[150] M. Mortaheb, M. A. Khojastepour, S. T. Chakradhar, and S. Ulukus. "Re-ranking the Context for Multimodal Retrieval Augmented Generation". In: *arXiv preprint* (2025). eprint: 2501.04695.

[151] D. W. Lee, C. Ahuja, P. P. Liang, S. Natu, and L.-P. Morency. "Lecture Presentations Multimodal Dataset: Towards Understanding Multimodality in Educational Videos". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2023, pp. 20087–20098. URL: https://openaccess.thecvf.com/content/ICCV2023/html/Lee_Lecture_Presentations_Multimodal_Dataset_Towards_Understanding_Multimodality_in_Educational_Videos_ICCV_2023_paper.html.

[152] H. Alawwad, U. Naseem, A. Alhothali, A. Alkhathlan, and A. Jamal. "Beyond Retrieval: Joint Supervision and Multimodal Document Ranking for Textbook Question Answering". In: *arXiv preprint* (2025). eprint: 2505.13520.

[153] Z. Chen, H. Liu, W. Yu, G. Sun, H. Liu, J. Wu, C. Zhang, Y. Wang, and Y. Wang. "M^{3}AV: A Multimodal, Multigenre, and Multipurpose Audio-Visual Academic Lecture Dataset". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024, pp. 9041–9060. DOI: 10.18653/v1/2024.acl-long.489.